Project: Test SQL Injection, On Your Website

University Of Europe for Applied Sciences

Software Engineering

Kimia Sadat Karbasi - 60393958

Civan Arda Özel - 45824733

Arsilda Qato - 20540071

Andres Sabillon - 79887864

Senem Türkaydın Elmalı - 79873076

Sorasith Chormalee - 53500842

Abstract- This project focuses on the security evaluation of a custom-developed website by testing it against various forms of SQL Injection (SQLi) attacks. SQLi is a common and dangerous vulnerability that can allow attackers to manipulate backend databases through unsanitized inputs. To assess the robustness of our website's input handling mechanisms, we implemented and executed a series of 10 structured test cases, each targeting a different SQLi technique. These include classic comment injection, union-based extraction, blind SQL injection (both boolean and time-based), and attacks via cookies, headers, stored procedures, and URL parameters. By simulating real-world attack scenarios directly on our own platform, we were able to identify potential weaknesses and better understand how to secure web applications against injection-based threats. This hands-on testing not only enhanced the security of our application but also provided practical experience in secure software development and ethical hacking.

Introduction

SQL Injection (SQLi) is one of the most widespread and severe security vulnerabilities in modern web development. It occurs when attackers are able to insert malicious SQL code into input fields or request parameters, often gaining unauthorized access to sensitive data or manipulating database contents. Despite being a well-documented issue, many web applications still suffer from poor input validation, making them susceptible to such attacks.

In this project, we created our own web application to serve as the target for SQLi testing. Our goal was to evaluate the security of this website by simulating realistic SQL injection attacks using a set of 10 detailed test cases. These test cases covered a wide range of attack vectors, including comment injection, error-based injection, blind injection, second-order injection, and more. Each case was designed to test a specific method of bypassing authentication, retrieving unauthorized data, or triggering unexpected behavior in the backend.

The motivation behind this project was twofold: first, to strengthen our website by identifying and addressing SQL injection vulnerabilities; second, to gain hands-on experience with ethical hacking methodologies and secure coding practices. Through this work, we aim to emphasize the importance of proactive security testing in the software development lifecycle.

Secure-WebPage

Security is one of the essential components in information technology, which should be considered during online platforms such as student portals. Having said that, we have to maintain students' information, professors' details, and other necessary things, which must be kept secure by the Database and other security mechanisms.

Therefore, based on this, our goal is to enhance the security of the student portal by taking specific steps from initialization to implementation. This portal has a few pages consisting of a login page with the features of forget password, reset password, home page by showing course, exams, and student profile which is filled by class schedule and each of them were built by security options. Moreover, we are going to do SQL injection, checking test scenarios to ensure that attackers are unable to gain access to the portal and obtain sensitive details, and there is no vulnerability to be seen.

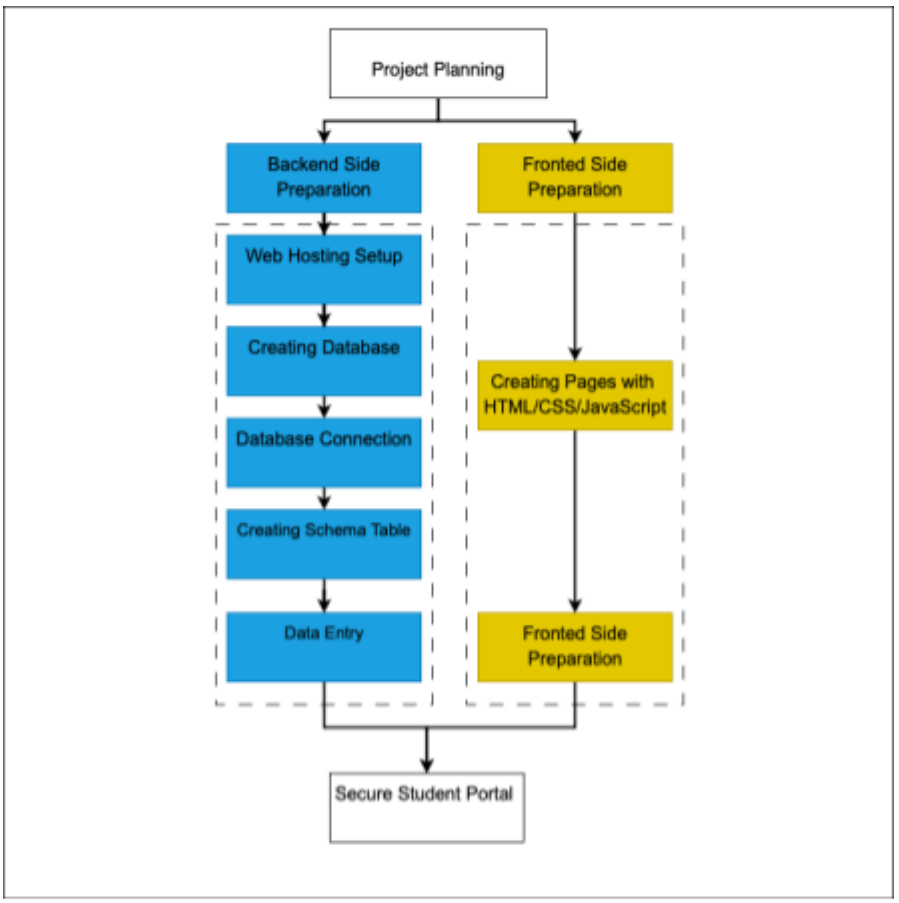


Fig.1: Workflow

Here is the steps are taken to make the platform more secure:

1. **Backend Side Preparation**
    - Web Hosting Setup
    - Creating Database
    - Database Connection
    - Creating Schema Table
    - Data Entry

2. **Fronted Side Preparation**
    - Creating Pages with HTML/CSS/JavaScript
    - Security Mechanisms

1.Backend Side Preparation

In this section, we focused on preparing the server-side foundation, which ultimately enable us to serve our pages, such as the login page, reset password, etc. The backend is responsible for storing, processing, and retrieving data securely.

1.1 Web Hosting Setup

We‘ve created an account in AlwaysData, which provides us with some features such as accessing databases by using phpMyAdmin, allowing permission to our source codes by using FTP, and Apache for serving our web pages through the internet. Moreover we added os.php to check the backend bone of AlwaysData environment to be sure about its backbone.

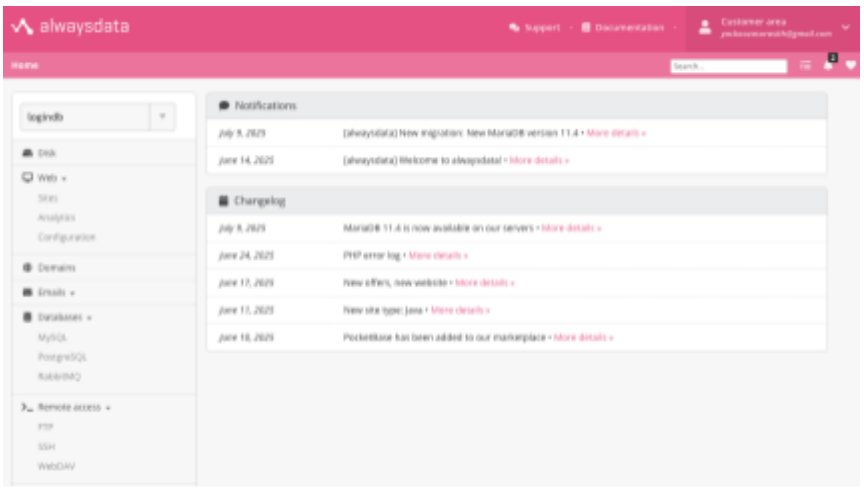


Fig.2: Alwaysdata Web Hosting

- **phpMyAdmin:**
  It consists of the mariadb version = 10.11.13 , php version = 7.4.33

Fig.3: phpMyAdmin

PhpMyadmin link : https://phpmyadmin.alwaysdata.com/
User: 421207_ajaska
Password: !WaaUt@TUM3#IdMwWcF$IESWS@1

- **FTP account:**

Creating an account To be accessed to our source codes by creating one account with required informations **hostname + Username + password + default port 21, then will be acc**

Here is the permission for accessing source codes:
Hostname: ftp-studentportal.alwaysdata.net
Username: studentportal
Password: Lifett!5!6
Port: 21

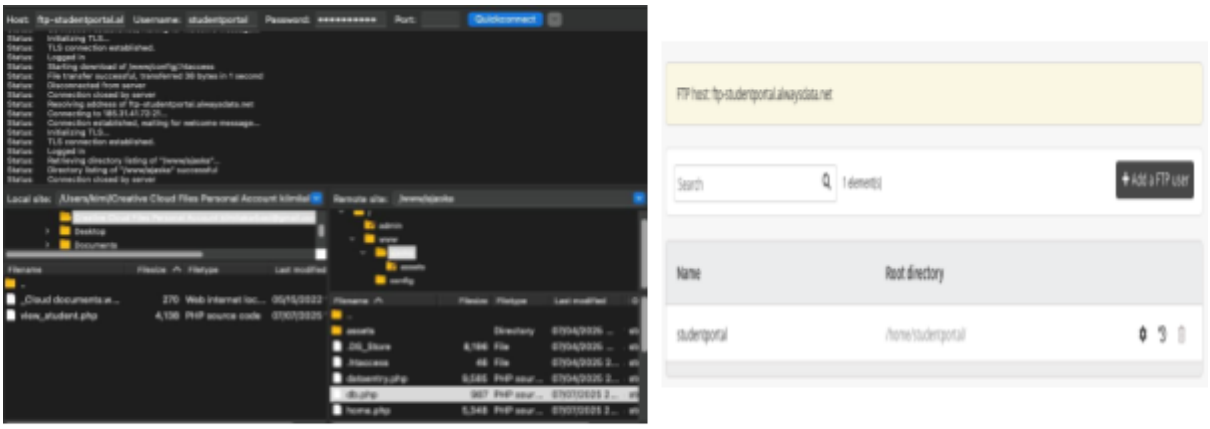


Fig.4: FTP Account

- **Checking Apache:**

Apache HTTP Server does have the responsibility to server pages on port 443 (HTTPS), so if it wants to make a connection it will pass .php files to the PHP engine (PHP-FPM) for handling web requests.

Fig.5: Apache HTTP Server

- **Checking Operating System:**

  To ensure the web hosting is build on Linux which environment which has lower vulnerability against to windows operating system

  https://studentportal.alwaysdata.net/ajaska/osinfo.php

1.2 Preparing Database

Some key points must be considered while creating database and its user:

- **Preventing from saving password on chrome:**using last pass to keep the password of database connection in a secure environment.

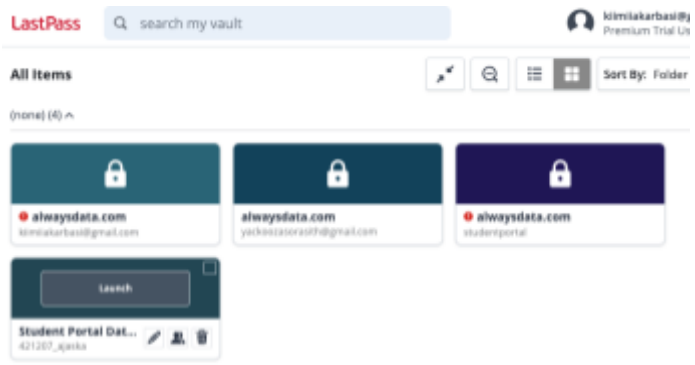


Fig.6: LastPass

- **Strong Password:** Using one Sentence and select first of each word and change some of them with special characters
  We are a united team, and to us, unity means everything. It doesn't matter where we come from – what truly matters is our togetherness.In every state, we stand as one.

  Password Example : !WaaUt@TUM3#IdMwWcF$IESWS@1

1.3 Database Connection

- **Creating Config.php** ⇒ put the file out of the original directory /www/ajaska, so we pull it into /config/config.php

```
<?php
return [
   'DB_HOST' => 'mysql-logindb.alwaysdata.net',
   'DB_PORT' => '3306',
   'DB_NAME' => 'studentportal_ajaska',
   'DB_USER' => '421207_ajaska',
   'DB_PASSWORD' =>
'!WaaUt@TUM3#IdMwWcF$IESWS@1'
];
```

Fig.7: Config.php

- **Adding .htaccess** ⇒ which allows us to block the config.php to not be accessible while sending requests to this page.

.htaccess

```
Deny from all
```

After importing .htaccess file in the path of /config/ , the result on the browser will show us following error message:

http://studentportal.alwaysdata.net/config/config.php

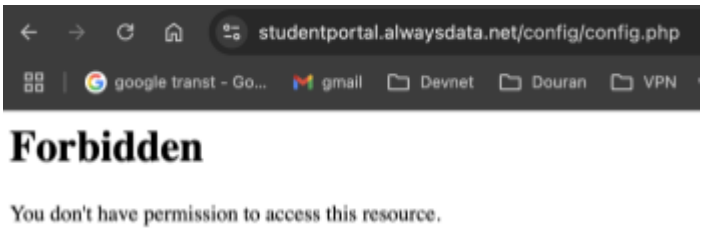


Fig.8: Error Message

- **Creating db.php** ⇒ by loading Credential from a file outside the public directory, this keeps sensitive information like DB passwords away from public access. The db.php will be putted on /var/www/ajaska

```php
$config = require __DIR__ . '/../config/config.php';
```

Fig.9: Creating db.php

1.4 Creating Schema Table

After gathering data about what is supposed to show on the student portal, it's necessary to create /www/ajaska/schema.sql which has all 9 tables with necessary fields.

```sql
-- Create student profiles table
CREATE TABLE student_profiles (
    user_id INT UNSIGNED PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    family_name VARCHAR(100) NOT NULL,
    matriculation_number VARCHAR(50),
    program_id INT UNSIGNED,
    date_of_birth DATE,
    FOREIGN KEY (user_id) REFERENCES portal_users(id) ON DELETE CASCADE,
    FOREIGN KEY (program_id) REFERENCES academic_programs(id) ON DELETE SET NULL
);
```

Fig.10: Schema Table

And there are 2 ways to be imported:

1. Adding tables by running sql queries
2. Import them Manually

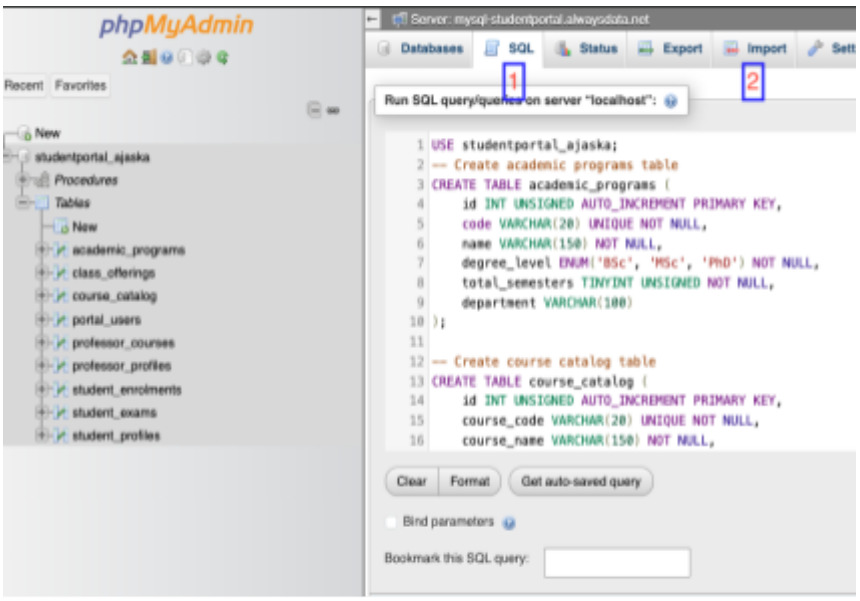


Fig.11: Adding Table-SQL Query

1.5 Data Entry

Queries are added into the dataentry.php and will be placed in this directory /www/ajaska/dataentry.php by loading db.php. Then we can call them to be inserted in our tables, or we can do them manually.
https://studentportal.alwaysdata.net/ajaska/dataentry.php
Warning: If you have entered sample once, It will return you back an error:

> Failed to insert data: SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicate entry 'BCS-CS' for key 'code'

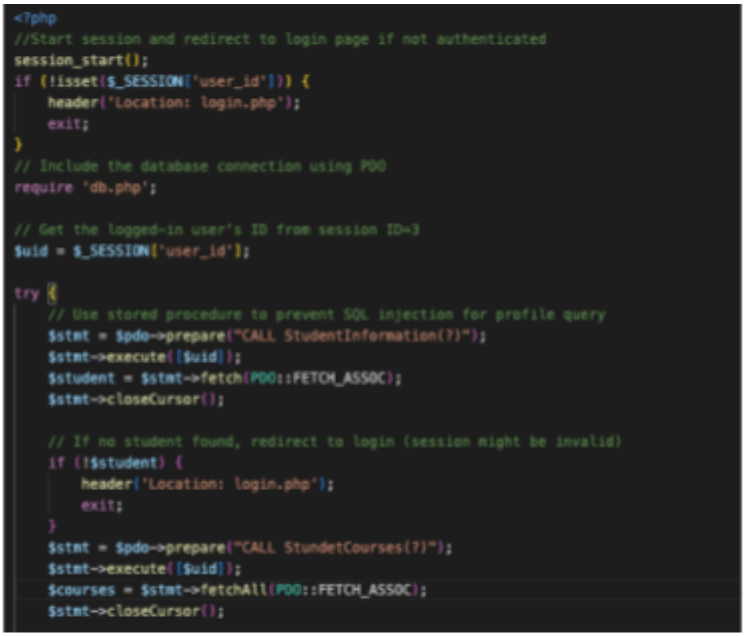
2.Front Side Preparation

After completing the backend setup and database structure, we moved on to building the frontend, the part of the system which users interact with directly. In this stage, we connected each page to the database in a secure way using db.php. This resulted in the possibility to access information within our PHP code to make forms, tables, and transfer contents to form dynamically. Furthermore, we focused on building pages with security options which gave us a reliable and secure space which should not be broken into by attackers.

2.1 Creating Pages with HTML/CSS/JavaScript

According to our created tables and student portal scenario, we have built 7 pages, each of which contributes to displaying and managing key student information through the portal with user-friendly interfaces. All of these pages together represent the Student Portal website, and they are served in a consistent structure. Every page follows the same architectural pattern, which is built on three core layers:

- PHP - Operational and Functional Layer
  This layer is in charge of running sql queries,fetching and receiving data between frontend and databases, handling users, and performing security checks.



```php
<?php
//Start session and redirect to login page if not authenticated
session_start();
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}
// Include the database connection using PDO
require 'db.php';

// Get the logged-in user's ID from session ID=3
$uid = $_SESSION['user_id'];

try {
    // Use stored procedure to prevent SQL injection for profile query
    $stmt = $pdo->prepare("CALL StudentInformation(?)");
    $stmt->execute([$uid]);
    $student = $stmt->fetch(PDO::FETCH_ASSOC);
    $stmt->closeCursor();

    // If no student found, redirect to login (session might be invalid)
    if (!$student) {
        header('Location: login.php');
        exit;
    }
    $stmt = $pdo->prepare("CALL StundetCourses(?)");
    $stmt->execute([$uid]);
    $courses = $stmt->fetchAll(PDO::FETCH_ASSOC);
    $stmt->closeCursor();
```

Fig.12: PHP -Operational and Functional Layer

- **HTML & CSS - Structural Layer**

  This structure will build forms, tables, headers, sections in the correct layout. It means HTML & CSS establish a framework with a stylist layout for each page in the student portal. Also we added two type of styles in this route : One of them is my customized style, stored at:
  /www/ajaska/assests/css/style.css
  And accessible via:
  https://studentportal.alwaysdata.net/ajaska/assets/css/style.css
  And another style is a prebuilt form bootstrap included via CDN:
  https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css

  These two styles work together and Bootstrap provides general UI styling, while the custom CSS file handles project-specific design.

```
<!doctype html>
<html lang='en'>
<head>
    <meta charset='utf-8'>
    <title>Login</title>
    <link href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css' rel='stylesheet'>
    <link href="/assets/css/style.css" rel="stylesheet" />
</head>
<body class='d-flex justify-content-center align-items-center vh-100 bg-light'>
<div class='card p-4 shadow rounded-4' style='min-width:22rem'>
    <h3 class='text-center mb-3'>Student Portal</h3>

    <!-- Display logout success message -->
    <?php if ($message): ?>
        <div class="alert alert-success" role="alert">
            <?= htmlspecialchars($message) ?>
        </div>
    <?php endif; ?>

    <!-- Display login error messages -->
    <?php if ($error): ?>
        <div class='alert alert-danger' role='alert'><?= htmlspecialchars($error) ?></div>
    <?php endif; ?>
    <!-- Login form -->
    <form method='post' autocomplete="off">
        <!-- CSRF token for security -->
        <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
        <!-- Email field -->
        <div class='mb-3'>
            <label class='form-label'>Email</label>
            <input name='email' class='form-control' type='text' autofocus>
        </div>
```

Fig.13: HTML & CSS Structural Layer

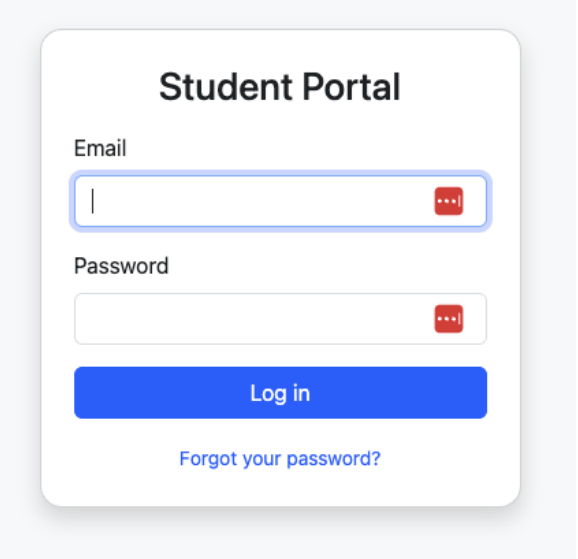
- **Bootstrap - UI/UX Layer**
  This layer was utilized for completing a set of prebuilt CSS and Javascript components to help us build responsive and user-friendly interfaces.
  We have two Javascript sources in the project:
  Custom Javascript which is stored at:
  /www/ajaska/assets/js/app.js
  And accessible via:
  https://studentportal.alwaysdata.net/ajaska/assets/js/app.js
  This file contains custom scripts for form handling.
  Prebuilt JavaScript from Bootstrap, loaded via CDN:
  https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js
  This file provides built-in functionality for dynamic components such as modals, dropdowns, tooltips, and etc.

```
                    </tr>
                <?php endforeach; ?>
                <?php if (!$exams): ?>
                    <tr><td colspan='4' class='text-center text-muted'>No exams</td></tr>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>
</div>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
    <script src="/assets/js/app.js"></script>
</body>
</html>
```

Fig.14: UI/UX Layer

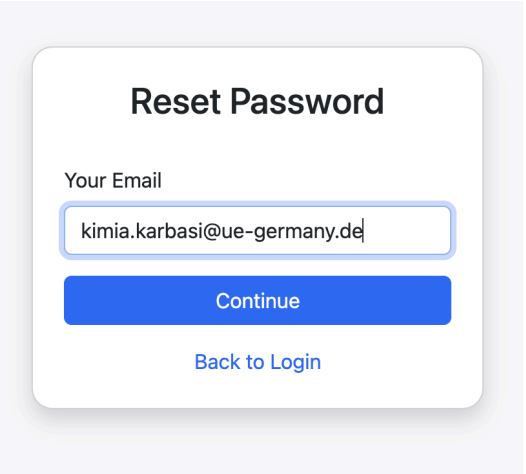
After understanding important components, we've implemented seven functional pages and all of them are stored at the path : /www/ajaska:

1.login.php

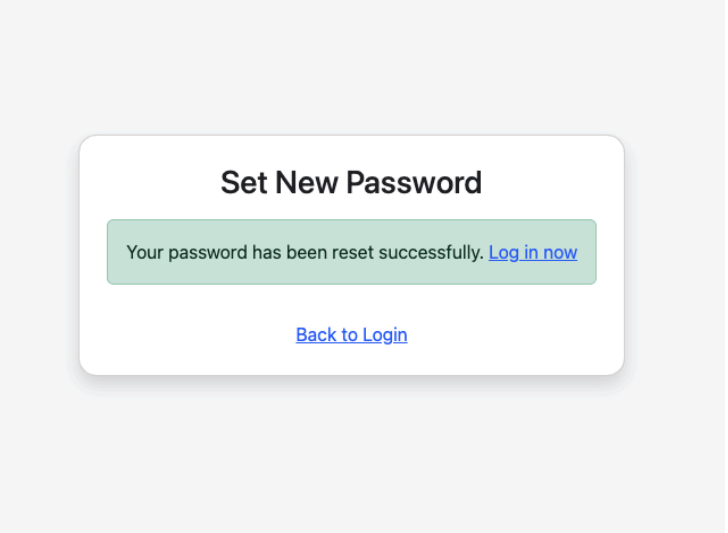


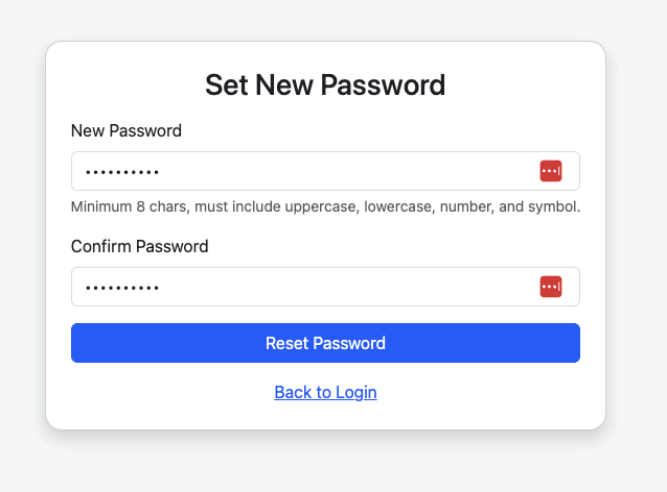
2.logout.php

3.reset_new_password.php

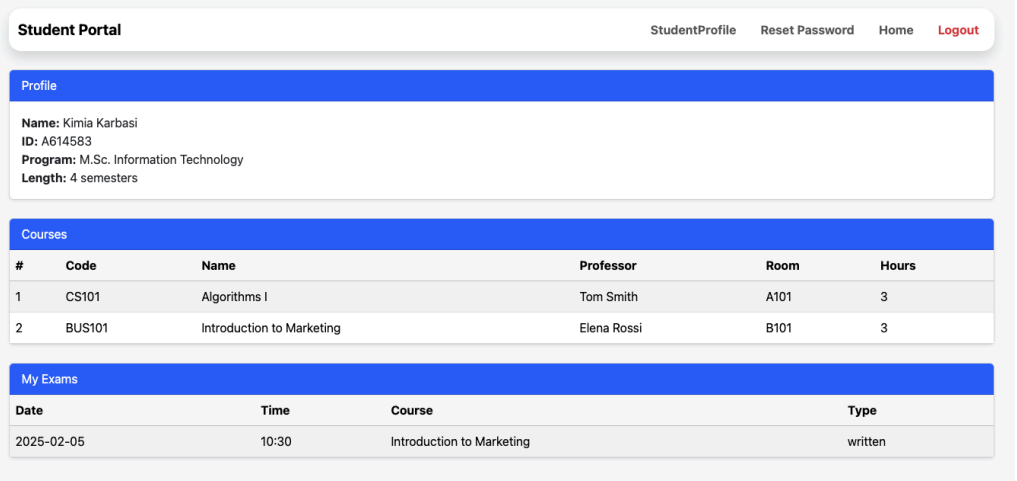


4.Set_new_password.php

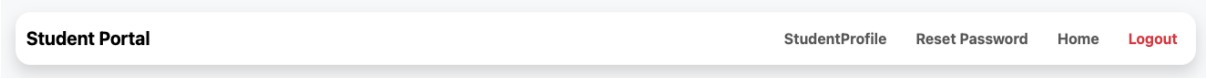




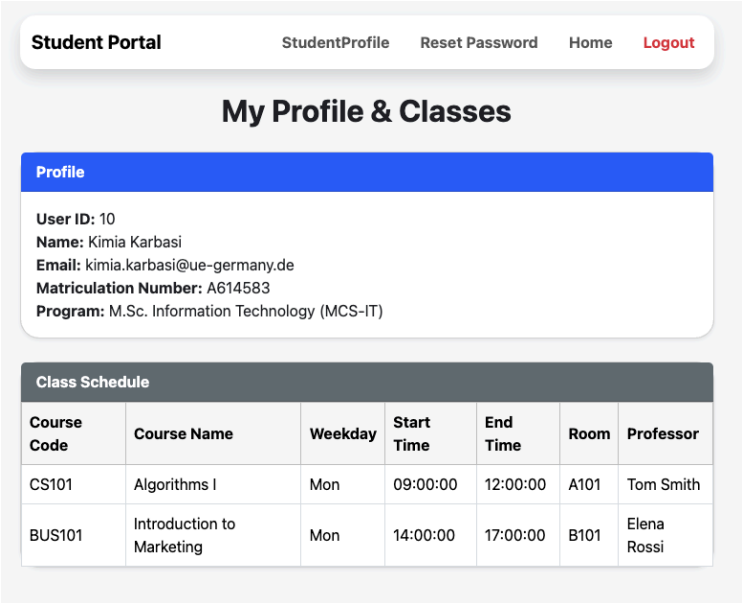
1. home.php



2. **nav.inc.php**



3. **view_students.php**



All of the source codes are added on the github:
https://github.com/cardaozel/Student_Portal

Login Credential:

https://studentportal.alwaysdata.net/ajaska/

Email: kimia.karbasi@ue-germany.de

Password: Lifett!5!6

2.2 Security Mechanisms

As a way of maintaining security and integrity of the student portal, several security features were added on the php pages. This combination of measures offers prevention of unauthorized access, identity theft, SQL injection, and typical web vulnerabilities.
The following core security mechanisms are implemented in the main pages:

Session Management:

The secure session handling is used to manage user authentication and session status on each of the secured pages.

```php
session_start();
require 'db.php';
```

```php
//Check if user is logged in. If not, redirect to login page.
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}
```

Fig. 17: Authentication Syntax

Secure Session Cookies:

Whenever a sensitive action is taking place, flags like HttpOnly, Secure and Strict Mode are applied on session cookies to reduce the chances of the session hijacking and fixation.

```php
// Start a secure session with recommended settings before session_start()
ini_set('session.cookie_httponly', 1);  // Prevent JavaScript access to session cookie
ini_set('session.use_strict_mode', 1);  // Prevent session fixation
ini_set('session.cookie_secure', 1);      // Only send session cookie over HTTPS

session_start();
```

Fig.16: Secure Session Cookies Syntax

Authentication and Authority checks:

Each page checks to make sure that the user is authenticated and, in case of doubt, further step-based authentication and it will return back to the login.php. Moreover, Authentication alone is not enough to prevent from **IDOR** happening, we must also check authorization for every sensitive resource or action.

```php
// Get the logged-in user's ID from session ID=3
$uid = $_SESSION['user_id'];
```

Fig.15: Session Management Syntax

```php
//Check if user is logged in. If not, redirect to login page.
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}

//Get the current user's ID from session.
$id = $_SESSION['user_id'];
$student = null;
$result = [];
$error = null;
```

Fig. 17: Authentication Syntax

So, we don't have access to other details by adding various user IDs.

https://studentportal.alwaysdata.net/ajaska/view_students.php?id=10

PDO with Prepared Statements:

The database queries are all prepared statements and against SQL injection. We added some PDO options, and created an object to get connected to our database. Also all of the queries can be added in a prepared statement without inserting them directly.

```php
$options = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,       // Errors as exceptions
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  // Results as associative array
    PDO::ATTR_EMULATE_PREPARES => false                // Real prepared statements
];

try {
    $pdo = new PDO(
        "mysql:host={$config['DB_HOST']};port={$config['DB_PORT']};dbname={$config['DB_NAME']};charset=utf8mb4",
        $config['DB_USER'],
        $config['DB_PASSWORD'],
        $options                // Apply secure PDO options
    );
```

Fig.18: PDO Syntax

Stored Procedures:

This feature is pre-compiled SQL routines which are routines in the database server by passing parameters instead of including queries in the PHP files(Calling Procedures).

```php
try {
    //Calling the stored procedure to fetch profile & class info for the student.
    $stmt = $pdo->prepare("CALL StudentProfiles(?)");
    $stmt->execute([$id]);
    $result = $stmt->fetchAll(PDO::FETCH_ASSOC); //Fetch all rows as associative arrays
    $student = $result[0] ?? null;               //Use the first row as the main student profile

    //Release the connection
    $stmt->closeCursor();

} catch (PDOException $e) {
    //If a database error occurs, show a generic message
    $student = null;
    $error = "Database error. Please try again later.";
```

Fig.19: Stored Procedure Syntax

StudentProfile: https://studentportal.alwaysdata.net/ajaska/view_students.php

Error Logging and User Errors:

All errors discovered within the application are recorded and presented to the administrator but in case of users, only general and non-sensitive messages are returned to avoid loss of information. (try, Catch)

```php
    //Use prepared statement to prevent SQL injection ---
    try {
        $stmt = $pdo->prepare("SELECT id FROM portal_users WHERE email=? LIMIT 1");
        $stmt->execute([$email]);
        $user = $stmt->fetch();
    } catch (PDOException $e) {
        // Log the database error and show only a generic message to the user
        error_log("DB error in reset_password: " . $e->getMessage());
        $error = "Internal Server Error. Please try again later.";
    }
```

Fig.20: Logging & User Errors Syntax

XSS Protection:

Cross site scripting is avoided as all dynamic information is safely avoided using escaping in advance to rendering which means the portal not be exposed by malicious and strange scripts.

```
<td><?= $i + 1 ?></td>
<td><?= htmlspecialchars($c['course_code']) ?></td>
<td><?= htmlspecialchars($c['course_name']) ?></td>
<td><?= htmlspecialchars($c['prof_fn'].' '.$c['prof_ln']) ?></td>
<td><?= htmlspecialchars($c['room_number']) ?></td>
<td><?= htmlspecialchars($c['duration_hours']) ?></td>
```

Fig.21: XSS Protection Syntax

Character Encoding:

Every page is encoded with the UTF-8 code set that blocks character encoding and display problems, So, we are able to identify which characters are included or not.

```
<meta charset='utf-8'>
```

Fig.22: Character encoding Syntax

CSRF Protection:

CSRF tokens act like shields for sensitive forms to prevent cross-site requests. This process will be runned from generating, validation and applying on html form. This will check that our session id is validated or not. (randomizing token, verifying before processing the request POST)

```
// Generate a CSRF token if it does not exist yet for the session
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32)); // Prevent CSRF attacks
}
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Validate CSRF token to prevent Cross-Site Request Forgery attacks
    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
        die('Invalid CSRF token'); // Stop if CSRF token doesn't match
    }
```

```
    <!-- CSRF token for security -->
    <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
```

Fig.23: CSRF Syntax

Input validation:

The validation of user input is done on the server side (checking an email format, password complexity) before processing.

```php
//validate email format
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $error = "Please enter a valid email address.";
} else {
```

Fig.24: Input Validation Syntax

Password Hashing:

passwords stored or updated by all users are securely hashed, which are checked during login.

```php
function hashPassword($password) {    # This function turns plain text passwords into hashed
    return password_hash($password, PASSWORD_DEFAULT);
}
```

```php
// Verify password hash and check login attempts
if ($user && password_verify($password, $user['password'])) {
    // Reset login attempts on successful login
    $_SESSION['login_attempts'] = 0;
```

Fig.25: Password Hashing Syntax

https://studentportal.alwaysdata.net/ajaska/reset_password.php

Rate Limiting:

Rate limiting is involved in the process of logging (user-attempts) to prevent from being attacked by the **brute force** process.

```php
// Initialize login attempts counter if not set
if (!isset($_SESSION['login_attempts'])) {
    $_SESSION['login_attempts'] = 0;
}
```

```php
    $_SESSION['login_attempts']++; // Increase failed attempt count

    // Lockout after 5 failed attempts - simple example
    if ($_SESSION['login_attempts'] >= 5) {
        $error = "Too many login attempts. Please try again later.";
    } else {
        $error = "Invalid credentials"; // Show login error
    }
}
```

Fig.26: Rate Limiting Syntax

Database Transactions:

Transactions are also used in the case of the initialization script to make certain data integrity during creation or update lots of records and then if there was one missing record, the transaction will be rolled back.

```
try {
    $pdo->beginTransaction(); # This makes sure that either everything works, or nothing is saved, ensuring database consistency.
```

```
$pdo->rollBack();
echo "Failed to insert data: " . $e->getMessage();
```

Fig.27: Database Transitions Syntax

Security Mechanism	Home Page	Logout	Reset Password	Data entry	Login	Set New Password	View_students
Session Management	X	X	X	-	X	X	X
Session Cookie Flags	-	X	X	-	X	X	-
Session Fixation Protection	-	-	-	-	X	-	-
Authentication/Authorization	X	-	X	-	X	X	X
PDO Prepared Statements	X	-	X	X	X	X	X
Stored Procedures	X	-	-	-	-	X	X
Error Logging and Handling	X	-	X	-	X	X	X
Generic User-Facing Errors	X	-		X	X	X	X
XSS Prevention (Output Escaping)	X	-	X		X	X	X
CSRF Protection	-	-	X		X	X	-
Input Validation	-	-	X		X	X	-
Character Encoding (UTF-8)	X	-	X		X	X	X
Rate Limiting	-	-	X		X	X	-
Password Hashing/Verification	-	-	-	X	verify	X	-
Database Transactions	-	-	-	X	-	-	-

As it can be seen in the above table, session management, database security (PDO), output escaping, error handling, and character encoding all run on every page and the rest of the protections (CSRF tokens, password hashing, rate limiting etc.) are applied onto pages that transmits authentication or password related activities or sensitive actions.

Moreover there are some general security features that can be used for the security mechanisms:

- **Session Initialization and Database Connection:** Initializing a PHP session, enabling the page to be tracked and including the database connection where the configuration file is located outside the original route.

Fig.28: Session Initialization

- **Input Retrieval and Sanitization:** retrieving user input from POST requests and trimming whitespace

Fig.29: Input Retrieval and Sanitization Syntax

- **Secure Session Destruction:** removing all traces of user session when a user logout

Fig.30: Secure Session Destruction Syntax

Having said that, in all our pages these core security components are used constantly: session management, secure database connections, PDO, stored procedures, input validation, output escaping, and error handling to ensure that all our pages are covered against the most common vulnerabilities on the web. This common standard structure has been published throughout our pages and database calls, which ensures that our application is resistant to dangerous attacks like SQL injection and keeps its overall security to a desirable level.

UnSecure Web Page

Non-Secure Student Portal

Project Folder

File contain :

- `index.php` – Index
- `home.php` – Main dashboard/home after login
- `logout.php` – Logout handle
- `db.php` – Database connection
- resetpassword.php - Reset Password Page
- `nav.inc.php` – Navigation include
- view_Student.php - Show Student
- `schema.sql` – Database schema(maria DB Connect with Port 3306)

Methodology

1. **Alwaysdata - Hosting And Database**
2. **Upload Non-Secure Student Portal**
3. **Create DB https://phpmyadmin.alwaysdata.com/ - With SQL Schema to Update Information in table adding with sample data insert - for student enrollment and professor**
4. **PHP Credential :**
   https://phpmyadmin.alwaysdata.com/
   user: logindb_user
   password: Hope@5@6

1.Non-Secure Student Portal - Code Summary & Documentation

Overview

This student portal allows users to log in, view a dashboard, and log out. It connects to a database using PHP and provides basic functionality - **without security measures** such as input validation, password hashing, or prepared statements.

User Flow(Normal Flow)

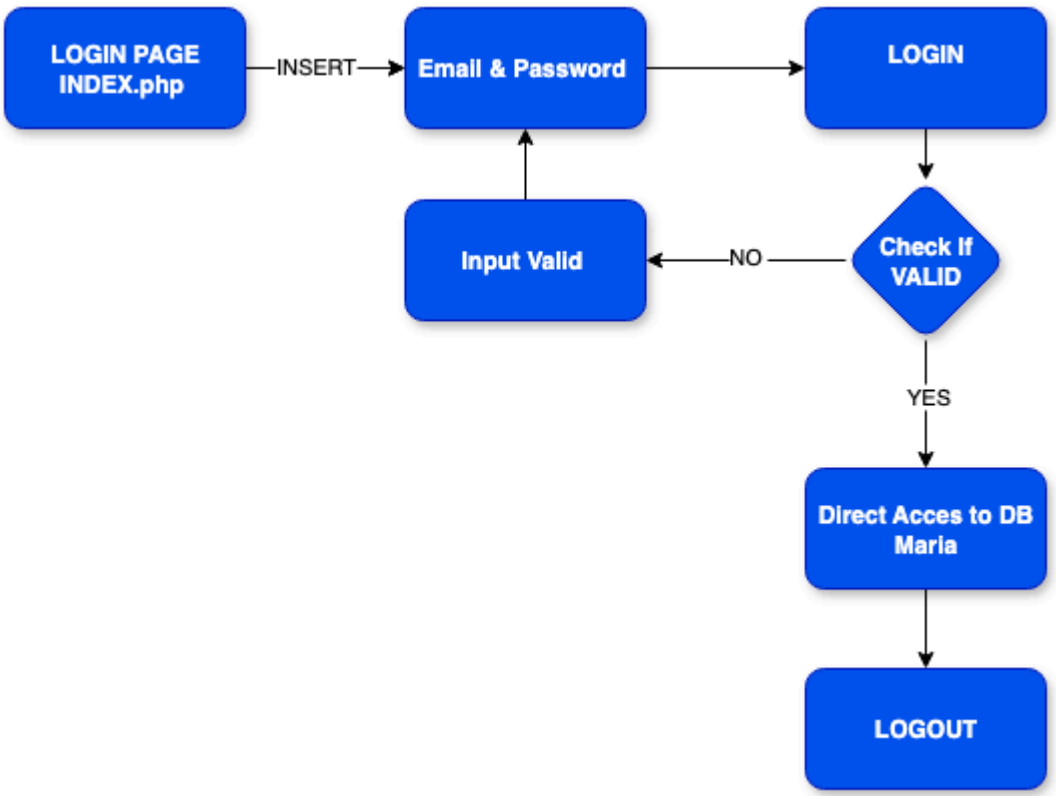
1. **User opens `index.php`** Sees the login form.

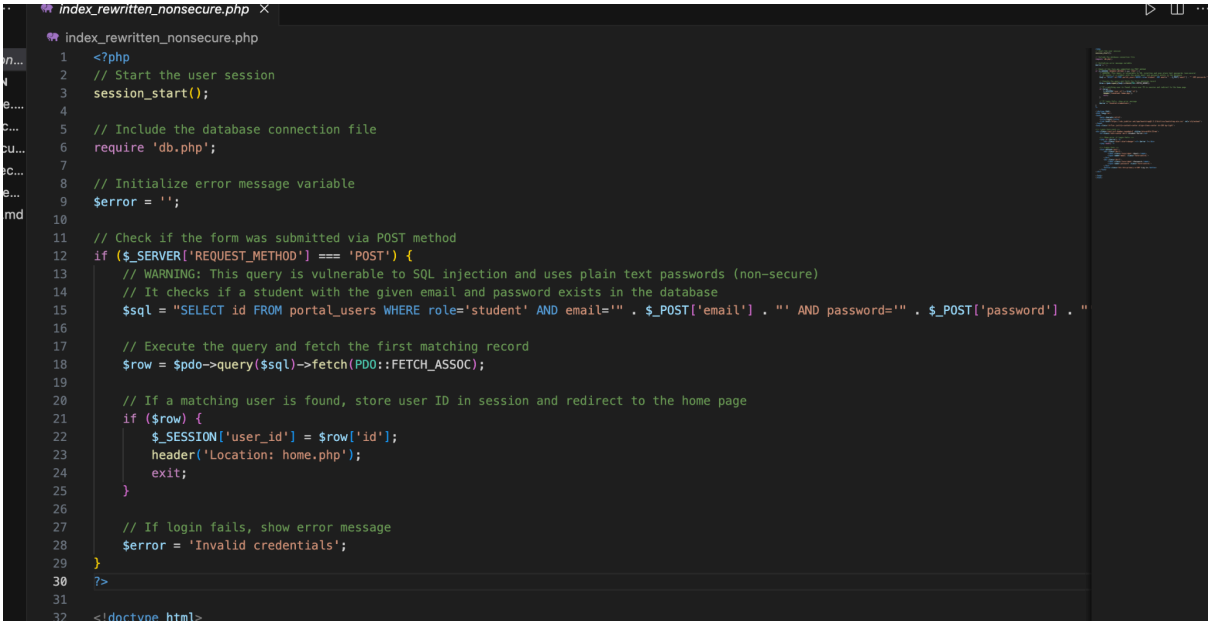
2. **Enters email & password** Credentials are checked directly (non-secure).

3. **If valid**, session is started Redirect to `home.php`.

4. **Dashboard (`home.php`)** greets the user by name.

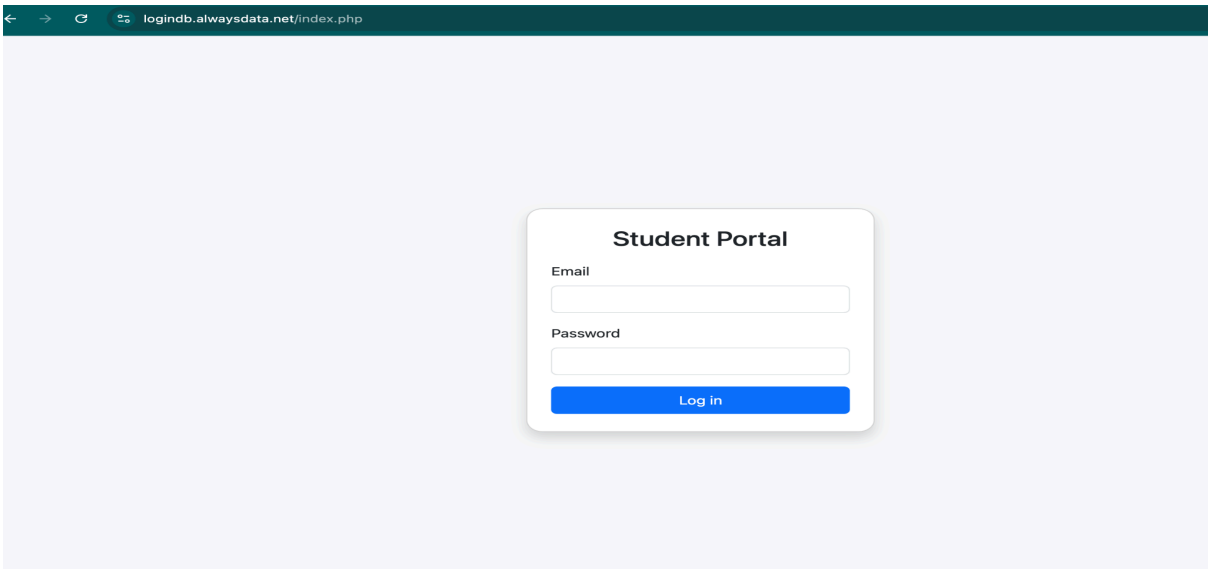
5. **User can log out** via `logout.php`





```php
<?php
// Start the user session
session_start();

// Include the database connection file
require 'db.php';

// Initialize error message variable
$error = '';

// Check if the form was submitted via POST method
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // WARNING: This query is vulnerable to SQL injection and uses plain text passwords (non-secure)
    // It checks if a student with the given email and password exists in the database
    $sql = "SELECT id FROM portal_users WHERE role='student' AND email='" . $_POST['email'] . "' AND password='" . $_POST['password'] . "'";

    // Execute the query and fetch the first matching record
    $row = $pdo->query($sql)->fetch(PDO::FETCH_ASSOC);

    // If a matching user is found, store user ID in session and redirect to the home page
    if ($row) {
        $_SESSION['user_id'] = $row['id'];
        header('Location: home.php');
        exit;
    }

    // If login fails, show error message
    $error = 'Invalid credentials';
}
?>
```

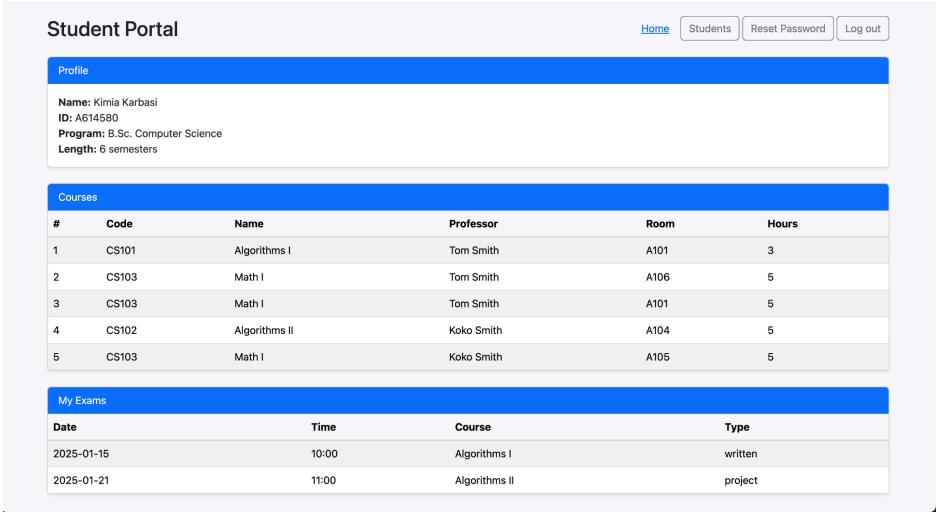
URL : https://logindb.alwaysdata.net/index.php



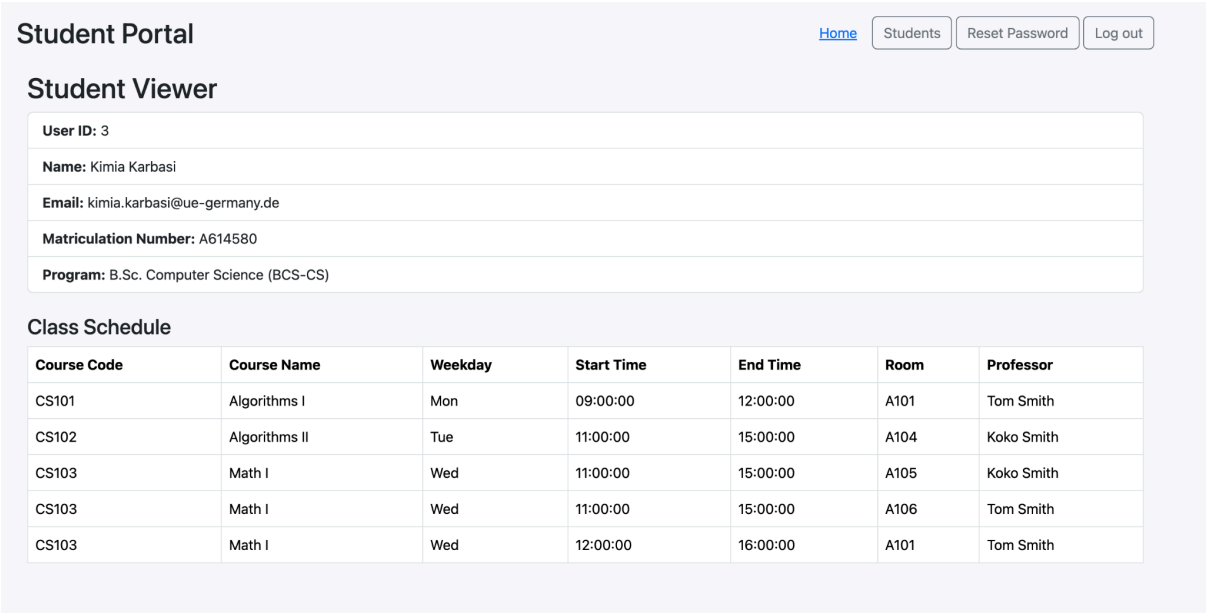
LOGIN Credential

user: kimia.karbasi@ue-germany.de
password: Robert

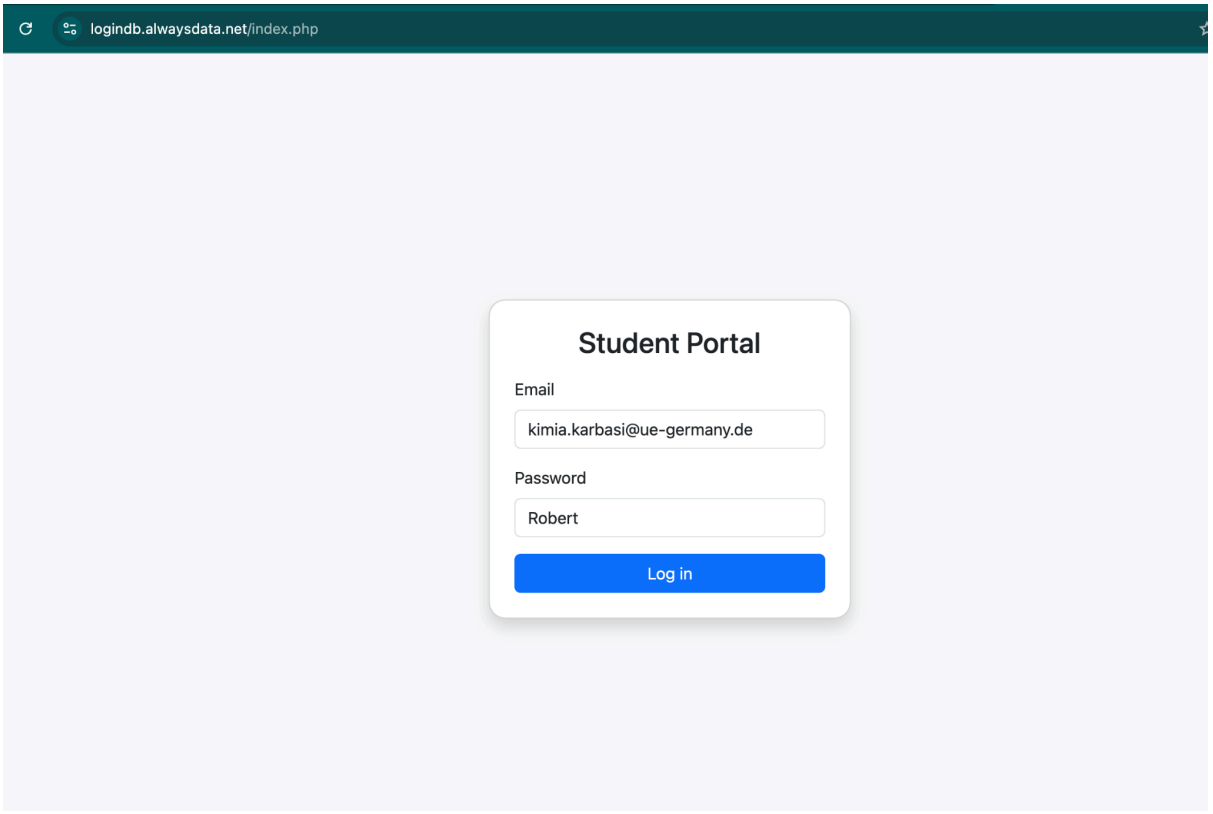
Student Portal : https://logindb.alwaysdata.net/home.php



Student Information : https://logindb.alwaysdata.net/view_student.php



1.1.Login State



Test Normal Flow : Without Injection SQL
Normal Login - Login Pass

Table Calling

EMAIL `SELECT * FROM `portal_users` ORDER BY `portal_users`.`email` ASC` From Table
Password `SELECT * FROM `portal_users` ORDER BY `portal_users`.`password` ASC`

1.2 Student Portal

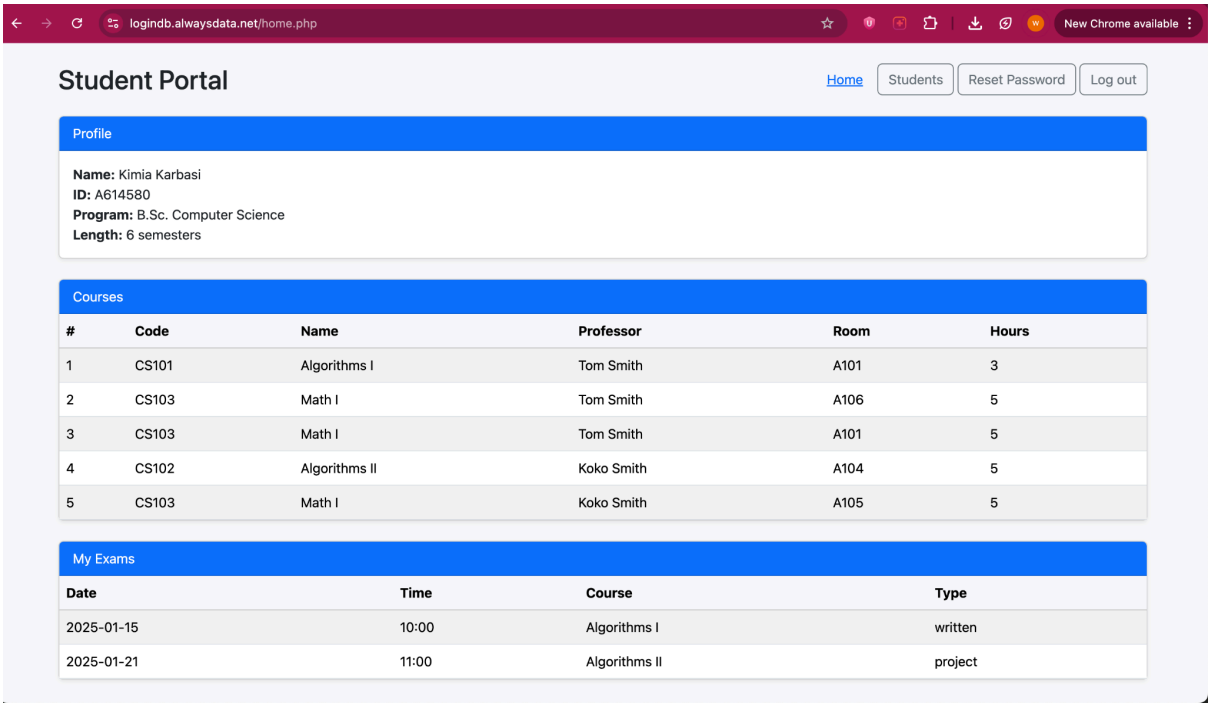


Table Calling

```
SELECT * FROM `academic_programs` - Academic Program
SELECT * FROM `class_offerings` - Class Offering
SELECT * FROM `course_catalog` - Course Catagories

SELECT * FROM `class_offerings` ORDER BY `class_offerings`.`room_number` ASC
- ROOM NUMBER
SELECT * FROM `class_offerings` ORDER BY `class_offerings`.`weekday` ASC -
WEEK OF TEACHING
SELECT * FROM `class_offerings` ORDER BY `class_offerings`.`start_time` DESC
- CLASS STARTIME
SELECT * FROM `class_offerings` ORDER BY `class_offerings`.`end_time` DESC -
END OF STUDY TIME
SELECT * FROM `class_offerings` ORDER BY `class_offerings`.`duration_hours`
ASC - DURATION TIME
SELECT * FROM `student_exams` ORDER BY `student_exams`.`exam_type` ASC - Exam
Type
```

1.3 Student Information Enrollment

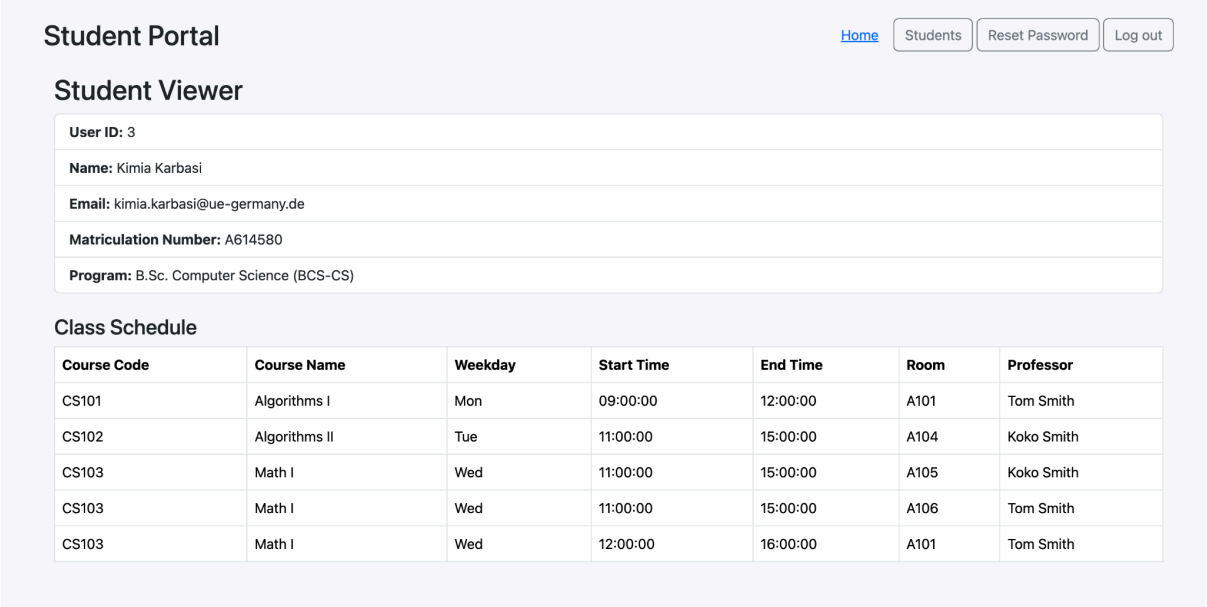


Table Calling

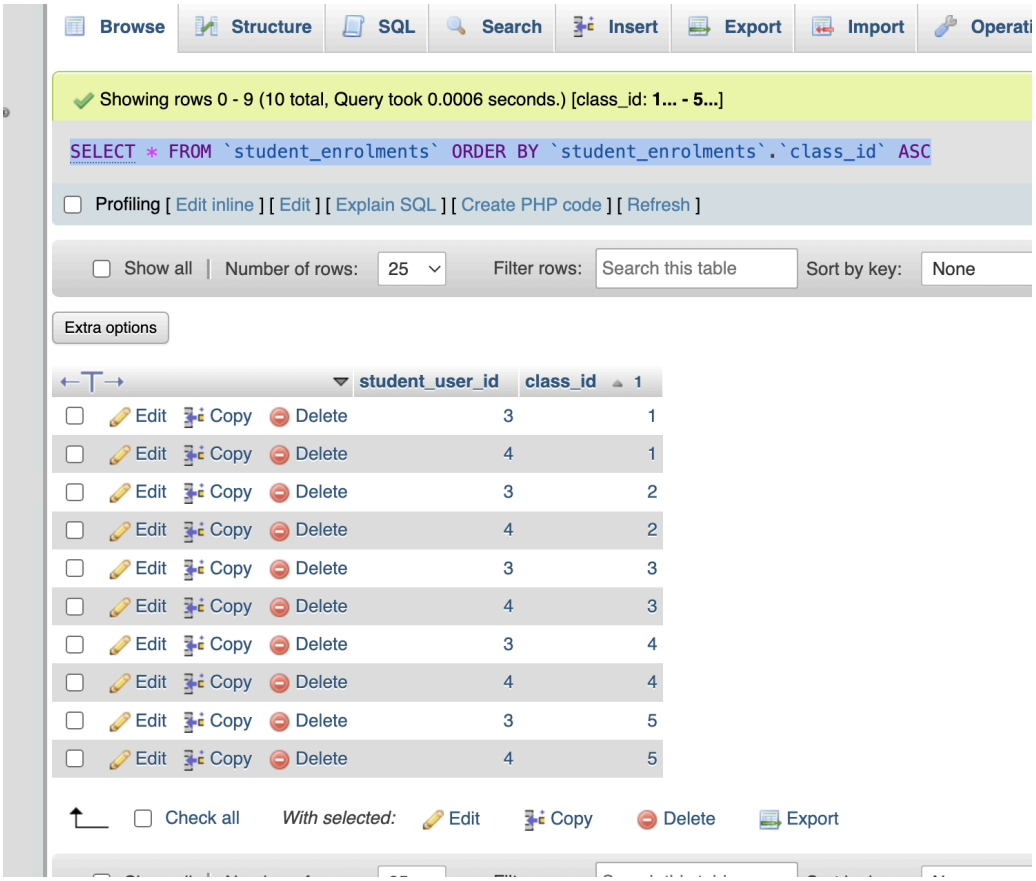
```
SELECT * FROM `portal_users` ORDER BY `portal_users`.`id` ASC - User ID
SELECT * FROM `student_profiles` ORDER BY `student_profiles`.`first_name`
ASC - Full name
SELECT * FROM `portal_users` ORDER BY `portal_users`.`email` ASC - Email
SELECT * FROM `student_profiles` ORDER BY
`student_profiles`.`matriculation_number` ASC - Matriculation Number
SELECT * FROM `academic_programs` - Program Info
```

Class Enrollment

```
SELECT * FROM `student_enrolments` ORDER BY `student_enrolments`.`class_id`
ASC - Class
```

```
SELECT * FROM `student_enrolments` ORDER BY
`student_enrolments`.`student_user_id` ASC - User ID from Student Portal
```

```
SELECT * FROM `student_enrolments` ORDER BY `student_enrolments`.`class_id`
ASC - Class ID
```



1.4 RESET PASSWORD

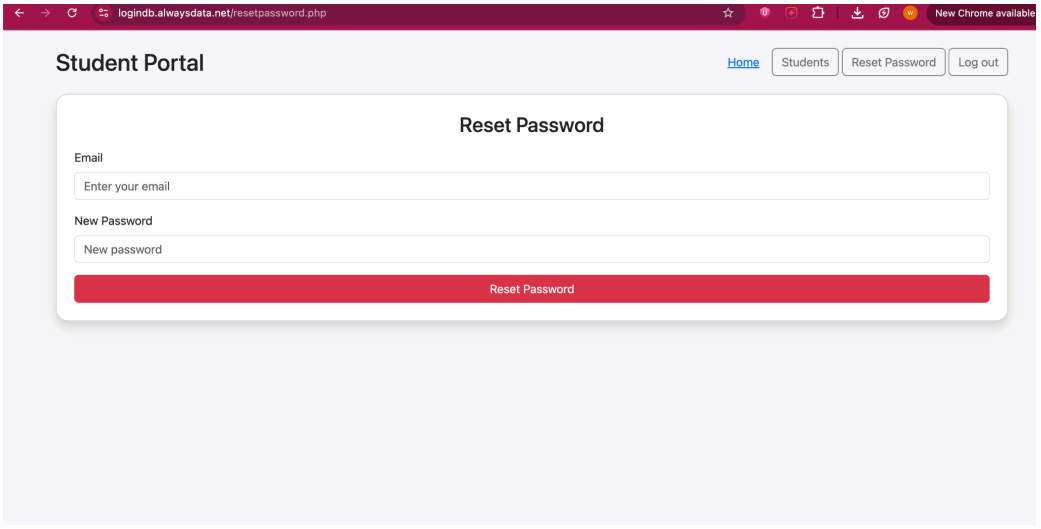


Table Calling

```
SELECT * FROM `portal_users` ORDER BY `portal_users`.`email` DESC - EMAIL
SELECT * FROM `portal_users` ORDER BY `portal_users`.`password` ASC
```

Sample of PHP ADMIN

Database Hosting :

https://phpmyadmin.alwaysdata.com/

user: logindb_user

password: Hope@5@6

File Descriptions

`index.php`

- Displays login form.
- On submission, runs **raw SQL query** with input from user (email, password).
- If match found, sets `$_SESSION['user_id']` and redirects to `home.php`.
- **Issues**:
- No password hashing. - REFER TO DATABASE
- Vulnerable to SQL injection.
- No CSRF protection.

`db.php`- Connects to MySQL using PDO.

- Sets error mode to Exception for better debugging.
- **Hardcoded credentials** (not secure for production).

`home.php`

- Checks if session is active (`$_SESSION['user_id']`).
- Retrieves user's name from DB and displays welcome message.
- Includes top navigation bar.

`nav.inc.php`

- Contains a simple Bootstrap-styled navbar.
- Provides a Logout button.

`logout.php`

- Ends the session with `session_destroy()`.
- Redirects back to login (`index.php`).

Major Non-Secure Issues Identified

Area	Issue	Risk Level
Authentication	Plain text password check	High
Database Query	Raw SQL with user input (SQL injection)	High
Session Handling	No secure/session timeout flags	Medium
CSRF	No token-based protection	Medium
Validation	No input sanitation	Medium

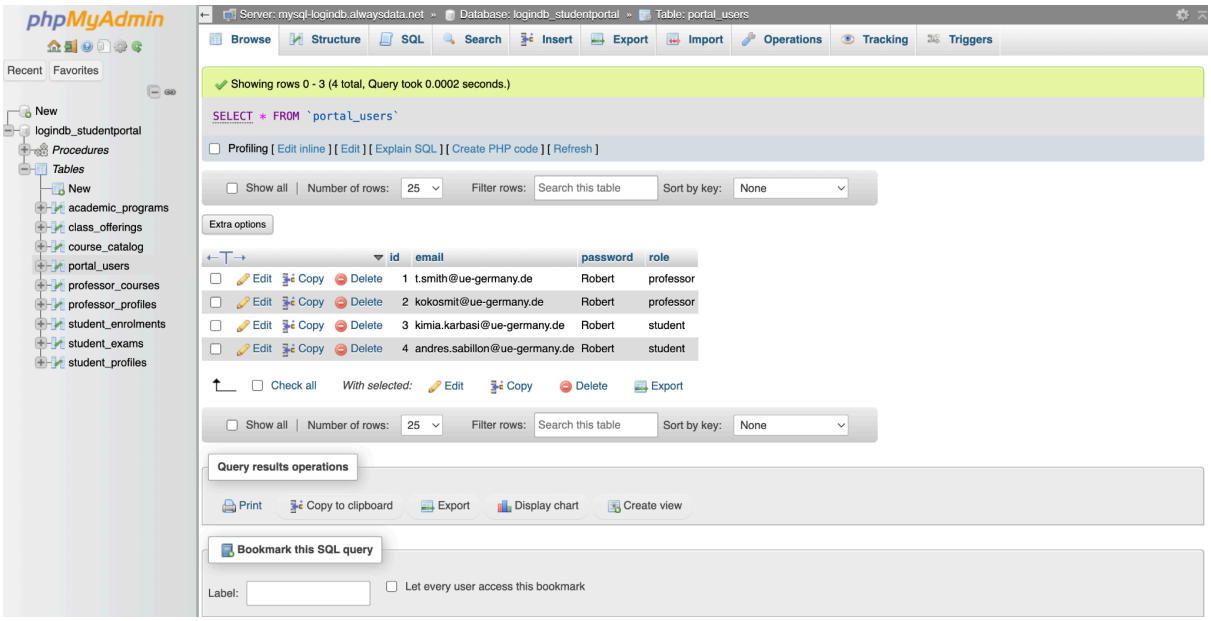
Authentication & Database Query - In Database using Plain text

```
$sql = "SELECT id FROM portal_users WHERE role='student' AND email='" .
$_POST['email'] . "' AND password='" . $_POST['password'] . "'";
```

Issue #1: It directly injects $_POST values into SQL

Issue #2: Compares plaintext password→ no `password_hash()` or `password_verify()` → **insecure login**.

Database Schema Documentation & Database Connection - Student Portal



schema.sql

```sql

-- Table storing academic programs (e.g., BSc, MSc)
CREATE TABLE academic_programs (
    id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    code            VARCHAR(20)  UNIQUE NOT NULL,
    name            VARCHAR(150) NOT NULL,
    degree_level    ENUM('BSc','MSc','PhD') NOT NULL,
    total_semesters TINYINT UNSIGNED NOT NULL,
    department      VARCHAR(100)
);

-- Table containing available courses and their module info
CREATE TABLE course_catalog (
    id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    course_code  VARCHAR(20)  UNIQUE NOT NULL,
    course_name  VARCHAR(150) NOT NULL,
    module_name  VARCHAR(150),
    credits      TINYINT UNSIGNED
);

-- Central users table (students, professors, admins) with plaintext passwords (non-secure)
CREATE TABLE portal_users (
    id       INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    email    VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role     ENUM('student','professor','admin') NOT NULL
);


-- Additional profile info for students (linked to portal_users)
CREATE TABLE student_profiles (
    user_id             INT UNSIGNED PRIMARY KEY,
    first_name          VARCHAR(100) NOT NULL,
    family_name         VARCHAR(100) NOT NULL,
    matriculation_number VARCHAR(50),
    program_id          INT UNSIGNED,
    date_of_birth       DATE,
    FOREIGN KEY (user_id)    REFERENCES portal_users(id)    ON DELETE CASCADE,
    FOREIGN KEY (program_id) REFERENCES academic_programs(id) ON DELETE SET NULL
);
```

Overview

This database schema is designed for a student portal system. It includes tables for managing academic programs, courses, users (students/professors), class schedules, enrollments, and exams.

Tables and Their Purpose

1. `academic_programs`

Stores degree program information (e.g., BSc Computer Science).

Fields: `code`, `name`, `degree_level`, `total_semesters`, `department`

2. `course_catalog`

Catalog of all courses offered by the university.

Fields: `course_code`, `course_name`, `module_name`, `credits`

3. `portal_users`

Main table for user login information;

Passwords stored in plaintext (non-secure)

Fields: `email`, `password`, `role`

4. `student_profiles` / `professor_profiles`

Additional personal information linked to `portal_users`.

- Students: `matriculation_number`, `program_id`

- Professors: `department`

5. `professor_courses`

Links professors to the courses they are qualified to teach.

6. `class_offerings`

Detailed schedule for each class, including room, weekday, and time.

7. `student_enrolments`

Tracks which student is enrolled in which class.

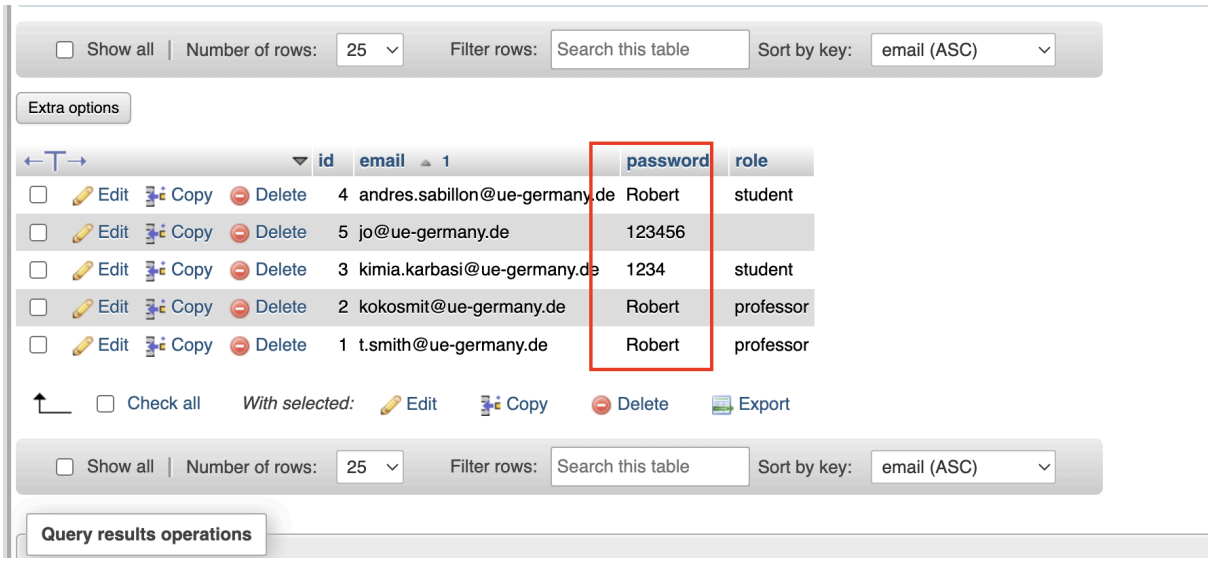
8. `student_exams`

Stores exam type (`written`, `project`), date, and time for students.

Security Notes

Issue	Description	Risk
Plaintext Passwords	Passwords are not hashed	High
No Input Validation	SQL expects valid input	Medium
Manual Relations	Referential integrity relies on app	Medium

PlainTex Password in DB Structure



No Input Validation in HTML

```php
// Check if the form was submitted using the POST method
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // WARNING: This SQL is vulnerable to SQL Injection and uses plaintext passwords
    // A secure version would use prepared statements and hashed passwords
    $sql = "SELECT id FROM portal_users
            WHERE email = '" . $_POST['email'] . "'
            AND password = '" . $_POST['password'] . "'";

    // Execute the query and fetch the result
    $row = $pdo->query($sql)->fetch(PDO::FETCH_ASSOC);

    // If a user with matching credentials is found
    if ($row) {
        // Store user ID in the session to track login state
        $_SESSION['user_id'] = $row['id'];

        // Redirect to the home page
        header('Location: home.php');
        exit;
    }

    // Set error message if login fails
    $error = 'Invalid credentials';
}
```

Manual Relations

1. $_GET['id'] in view_student.php

```
$id = $_GET['id'] ?? '3';
```

...

```
WHERE sp.user_id = $id
```

---

2. $_SESSION['user_id'] in home.php, etc.\

```
—--------------------------------
$uid = $_SESSION['user_id'];

...

WHERE sp.user_id = $uid
—--------------------------------
```

3. Foreign Keys Exist – but Not Always Enforced at Code Level

```
—--------------------------------
FOREIGN KEY (user_id) REFERENCES portal_users(id) ON DELETE CASCADE

—--------------------------------
```

Sample Data Inserted

- 1 student: `kimia.karbasi@ue-germany.de`

- 2 professors

- 5 courses in the catalog

- 5 class offerings scheduled

- Student enrolled in 5 classes

- 2 exam entries (written & project)

Recommendation (Methodology upgrade to SECURE)

- Use `password_hash()` and `password_verify()` for secure authentication.

- Add unique constraints and input validation at the application level.

- Apply prepared statements to prevent SQL injection

Injection Methodologies

1 - TC_SQLI_001: Login SQLi - Comment Injection

Test SQL Injection using comment termination to bypass authentication. The objective is to verify if the login form is vulnerable to '-- comment termination attack. The precondition is that the Login page must be accessible. The test data is Email: admin@domain.com and Password: anything. The expected results; Login should fail. If login succeeds, it indicates a vulnerability. The test steps are;

1. Open the login page
2. Enter Email: admin@domain.com
3. Enter Password: anything
4. Submit the form
5. Observe if login succeeds

The post condition is that the session should not be created if vulnerability is not present. The test status is Not Executed

2 - TC_SQLI_002: Test Union-based SQL injection to extract data

The test union-based SQL injection to extract data. The objective is to verify if UNION SELECT queries can be injected via email input. The precondition is that the Login page must be accessible. The test data is “Email: ' UNION SELECT creditCardNumber, 1, 'admin' FROM CreditCardTable – Password: irrelevant". The expected result Login should fail, and no credit card data should be exposed in the UI. The test steps are;

1. Open the login page
2. Enter Email: ' UNION SELECT creditCardNumber,1,'admin' FROM CreditCardTable --
3. Enter Password: irrelevant
4. Submit the form
5. Observe if sensitive data is returned.

The postconditions are that the database should not return or display credit card data. The test status is Not Executed.

3 - TC_SQLI_003: Login SQLi - Blind Injection

The test uses blind SQL injection to detect vulnerability based on timing. The objective is to Check for blind SQL injection via response delay. The precondition is that the login page must be accessible. The test data is "Email: admin@domain.com' AND SLEEP(5)--Password: any". The expected result if response is delayed significantly, it indicates SQL injection vulnerability. The test steps are;

1. Open the login page
2. Enter Email: admin@domain.com' AND SLEEP(5)--
3. Enter Password: any
4. Submit the form
5. Measure response time"

The postconditions are that no login should be successful; Response time should be normal. The test status is Not Executed

4 - TC_SQLI_04: Error-Based SQL Injection

The test attempts to trigger a SQL error via invalid input to expose backend database messages. The objective is to identify if the system leaks SQL error messages. The precondition is that the User is not logged in. The test data Email: ' OR 1=1 -- Password: any string. The expected result SQL error message is returned (e.g., syntax error, DB type, column name). The test steps are;

1. Go to login page
2. Enter payload in email field
3. Submit login
4. Observe response for error

The postconditions is User is not logged in. The test status is Pending.

5 - TC_SQLI_05: Boolean-Based Blind SQL Injection

The test uses true/false logic to infer backend behavior without visible error messages. The objective is to confirm if conditional logic affects system behavior. The precondition is that the user is not logged in. The test data Email: ' OR 1=1 -- and then ' OR 1=0 – . The expected result login success on true condition, failure on false — revealing data inference. The test steps are;

1. Submit valid condition
2. Submit false condition
3. Compare results

The postcondition is that the system should not expose any difference. The test status is Pending

6 - TC_SQLI_06: SQLi via HTTP Headers (User-Agent)

The test injects SQL in HTTP headers like User-Agent to test backend handling. The objective is to Identify if headers are unsafely passed into SQL queries. The precondition is that they use a tool like BurpSuite or Postman. The test data User-Agent: ' OR 1=1; -- . The expected result Server error or unexpected content from the backend. The test steps are;

1. Send request with custom User-Agent header
2. Observe server behavior

The postcondition is that No access should be granted. The test status is Pending.

7 - TC_SQLI_07: Second-Order SQL Injection

The test inserts a malicious payload that is stored and executed later in a different process. The objective is to Detect stored injection that executes upon retrieval. The precondition is that the access to a form that stores input data.The test data Full Name: Robert'); DROP TABLE students; -- . The expected result SQL is executed later when the field is loaded, causing potential damage. The test steps are;

1. Submit form with payload
2. Visit area that loads that data
3. Check for errors or anomalies

The postcondition is that the Data should be sanitized at storage & output. The test status is Pending.

8 - TC_SQLI_08: SQL Injection via URL Parameters

The test manipulates a GET parameter directly in the URL to trigger injection. The objective is to detect improper sanitization of URL query parameters. The precondition is that the endpoint must accept GET parameters. The test data URL: /view-student.php?id=1' OR '1'='1. The expected result is that extra data is shown or SQL error occurs. The test steps are;

1. Visit manipulated URL
2. Monitor page output or server response

The postcondition is that the Behavior should not change. The test status is Pending.

9 - TC_SQLI_09: SQLi via Stored Procedures

The test tries to exploit SQL stored procedures if they are being used by the backend. The objective is to Test if stored procedures can be maliciously invoked. The precondition is that the DB must use stored procedures. The test data Payload: '; CALL suspicious_proc(); -- The expected result procedure is called, or SQL error is thrown. The test steps are;

1. Inject into input
2. Trigger stored procedure logic
3. Observe results

The postcondition is that the procedures should never be invocable by input. The test status is Pending.

10 - TC_SQLI_10: Time-Based Blind SQL Injection

The test uses SQL conditional logic with delay to measure if a query was executed. The objective is toConfirm injection by observing delay in server response. The precondition is that the user is not logged in. The test data Email: ' OR IF(1=1, SLEEP(5), 0) -- Password: any string. The expected result If query is executed, response time increases noticeably (~5s). The test steps are;

1. Inject payload
2. Measure server response time
3. Compare with baseline

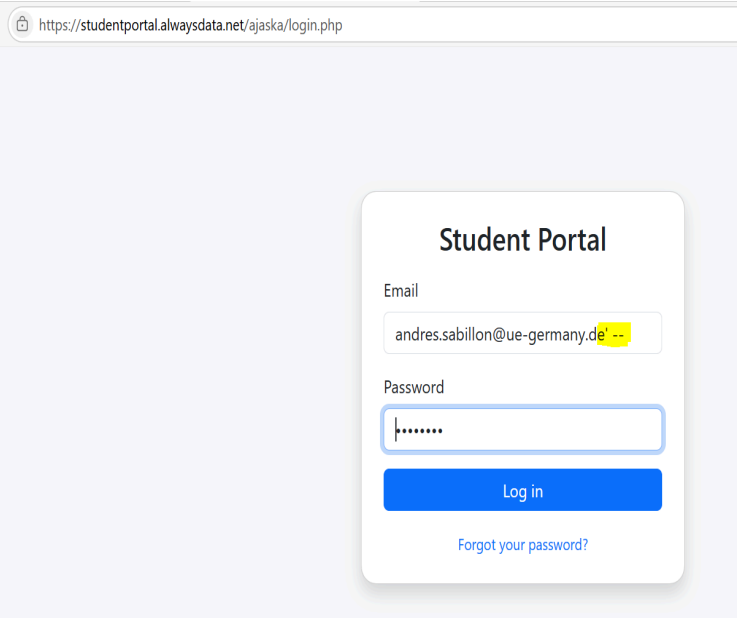
The postcondition is that the Response time should remain consistent. The test status is Pending

SQL Injection in Secure WEB Pages

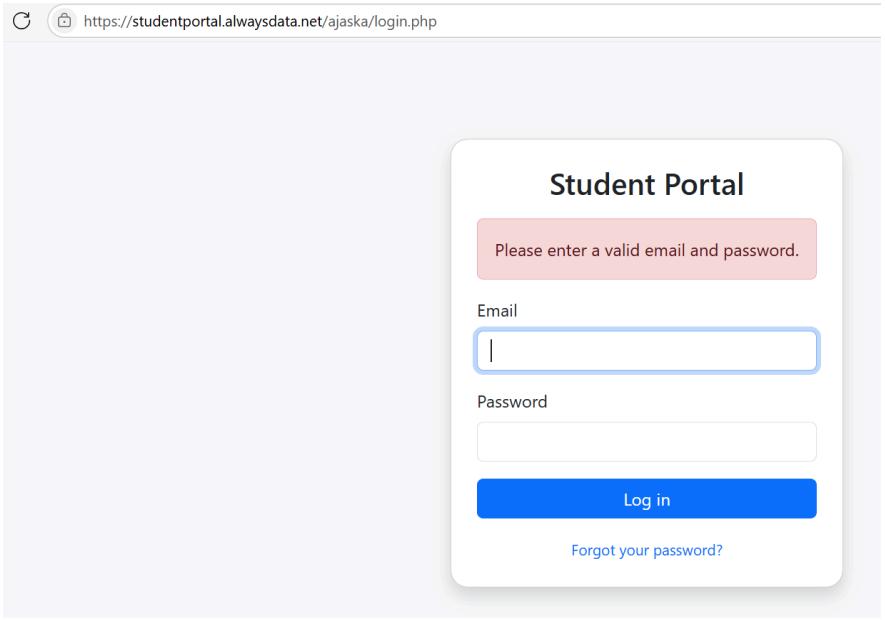
Manual Testing

1 - TC_SQLI_001: Login SQLi - Comment Injection

Username: andres.sabillon@ue-germany.de' –
Password: ' OR 1=1 --

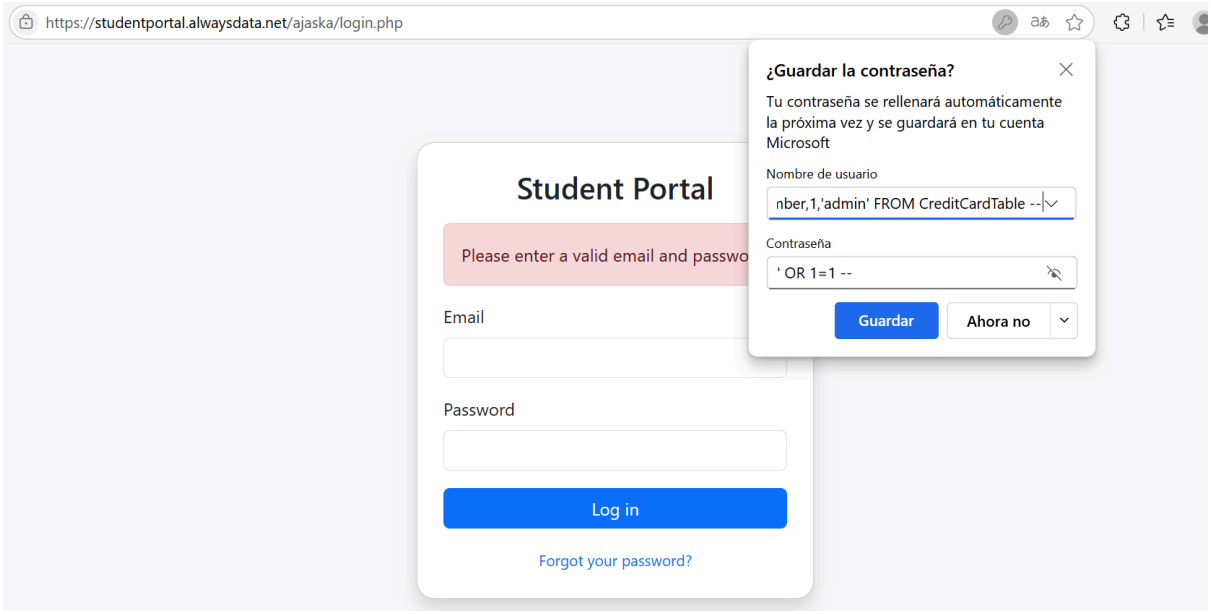


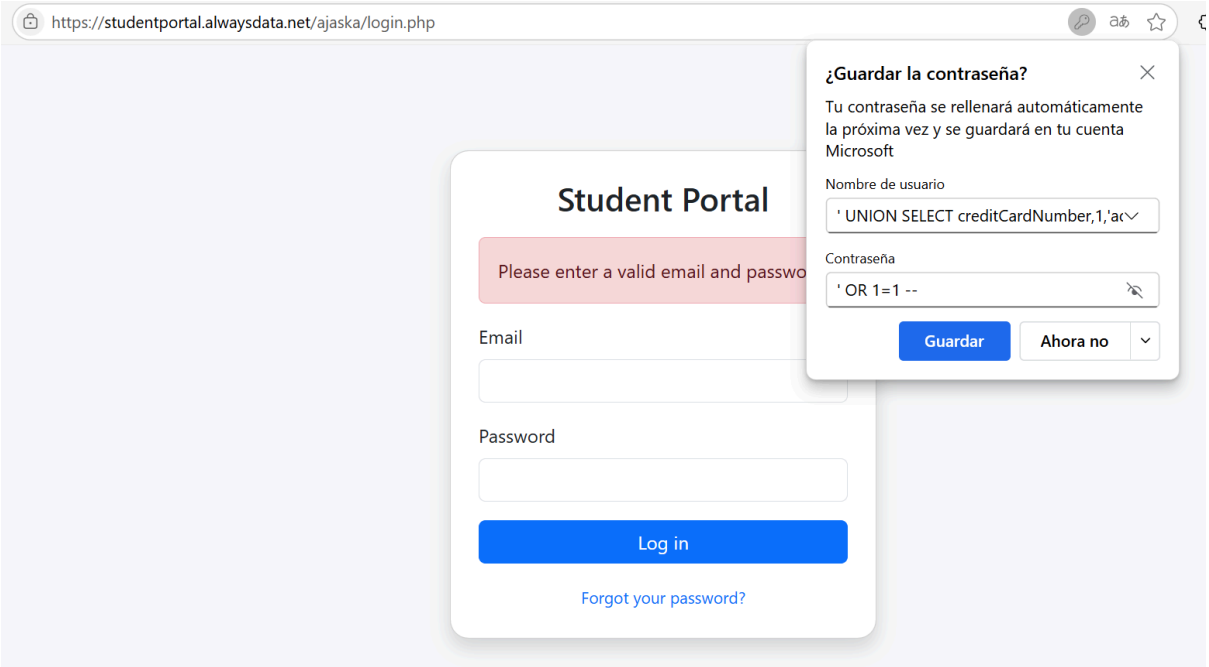
System Response



2 - TC_SQLI_002: Test Union-based SQL injection to extract data

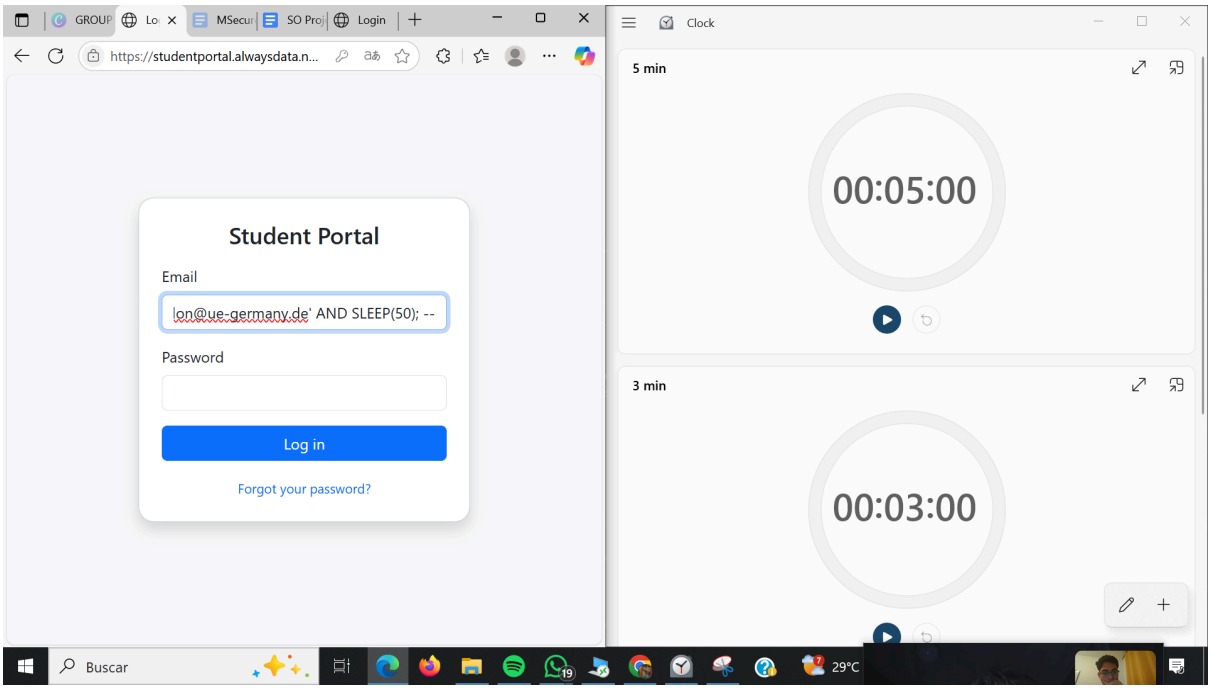
username: ' UNION SELECT creditCardNumber,1,'admin' FROM CreditCardTable –
Password:' OR 1=1 –



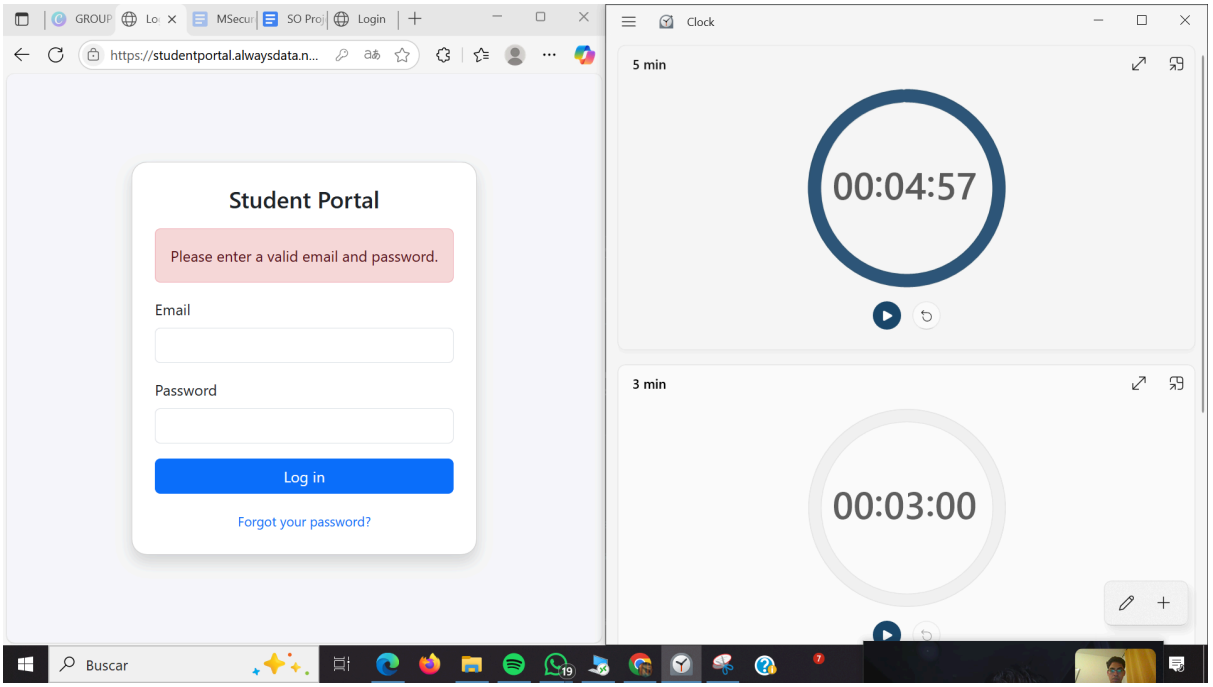


3 - TC_SQLI_003: Login SQLi - Blind Injection

Username: andres.sabillon@ue-germany.de' AND SLEEP(50); --
Password: irrelevant



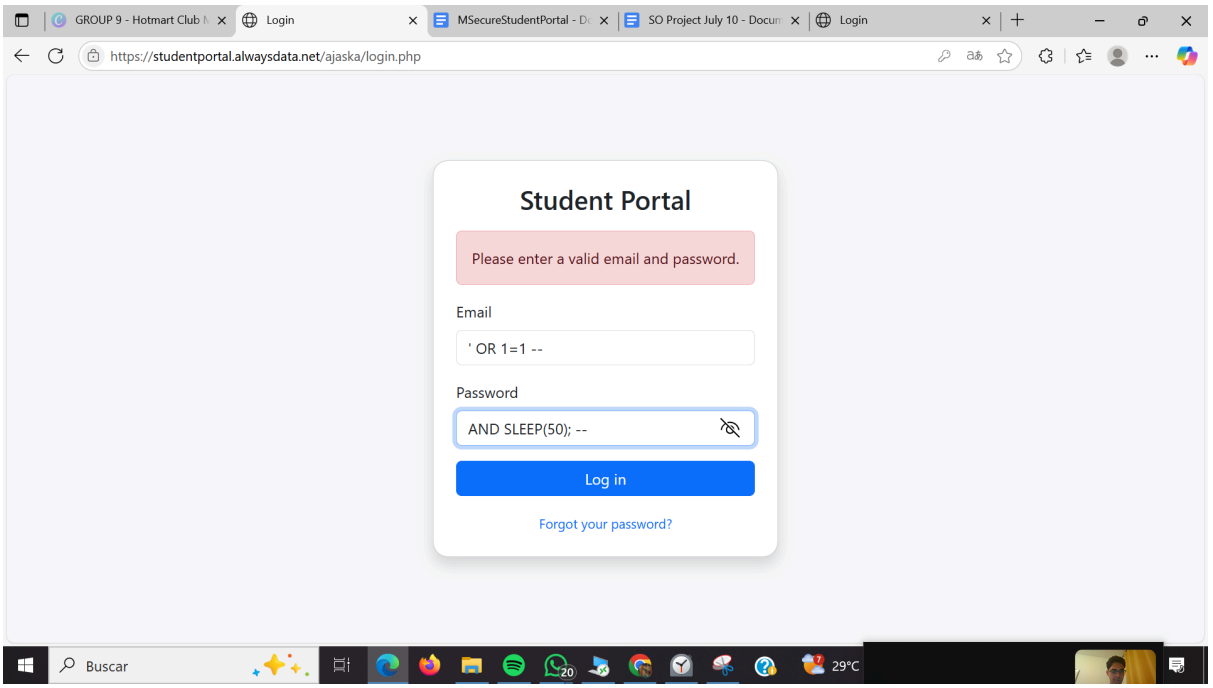
Response:



4 - TC_SQLI_04: Error-Based SQL Injection

username: ' OR 1=1 –

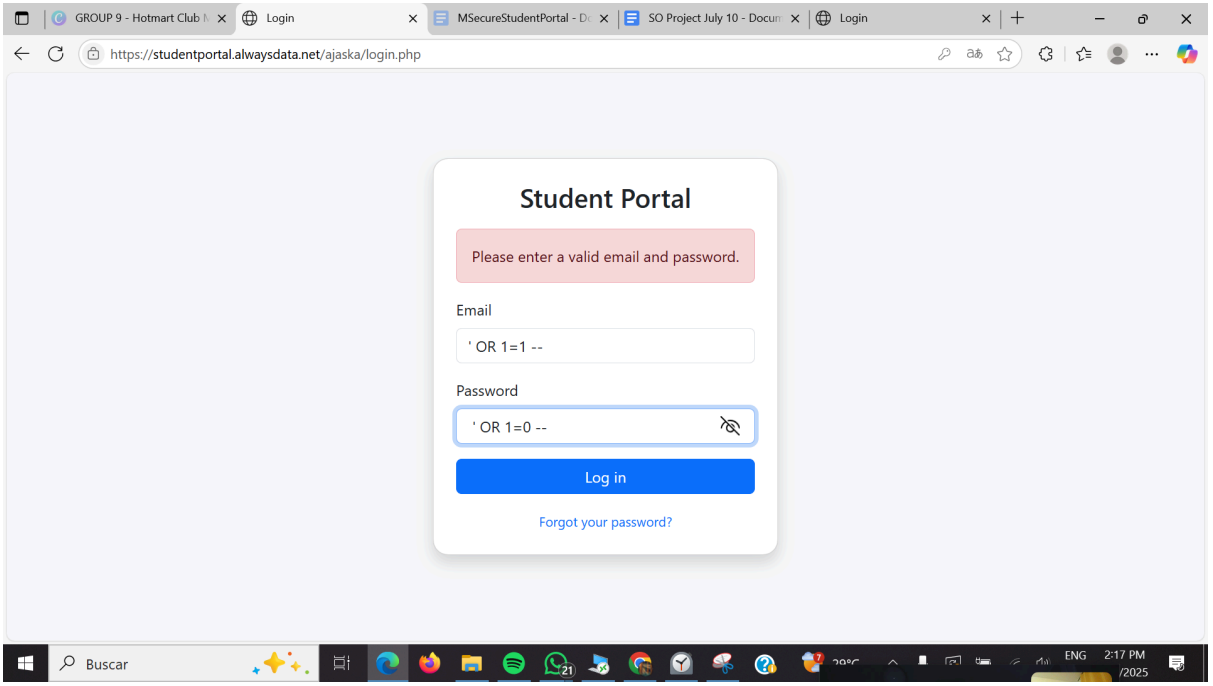
password: Any string

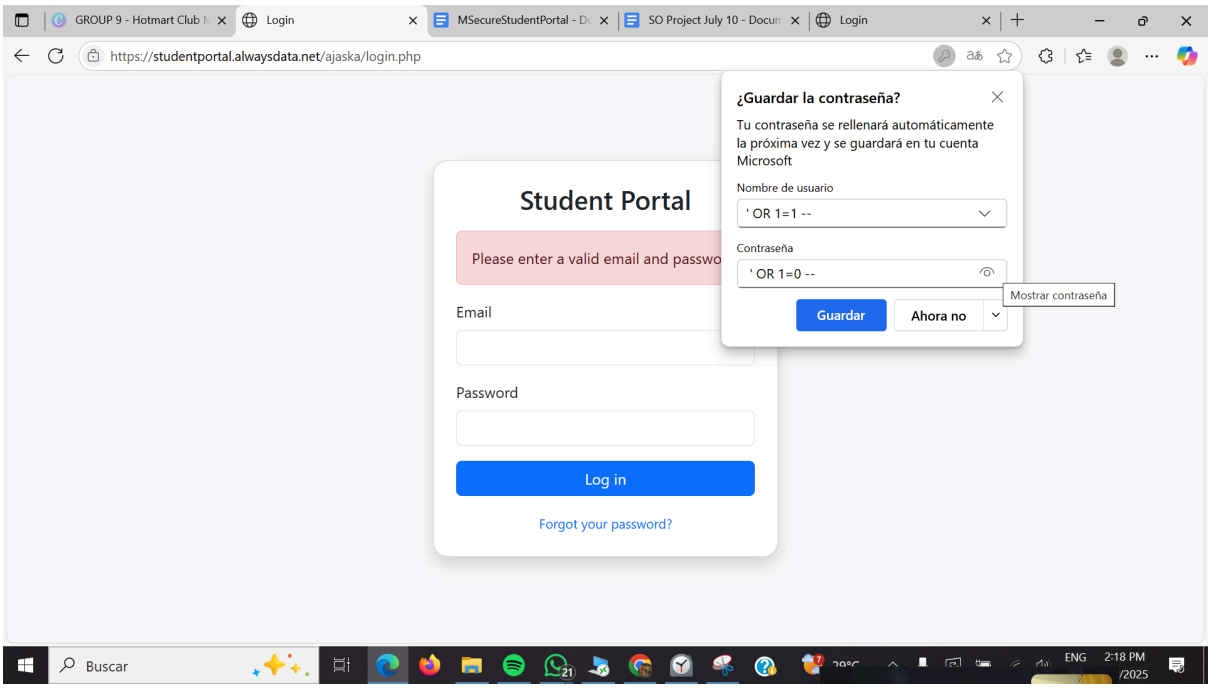


5 - TC_SQLI_05: Boolean-Based Blind SQL Injection

Username: Email: ' OR 1=1 -- and then ' OR 1=0 –

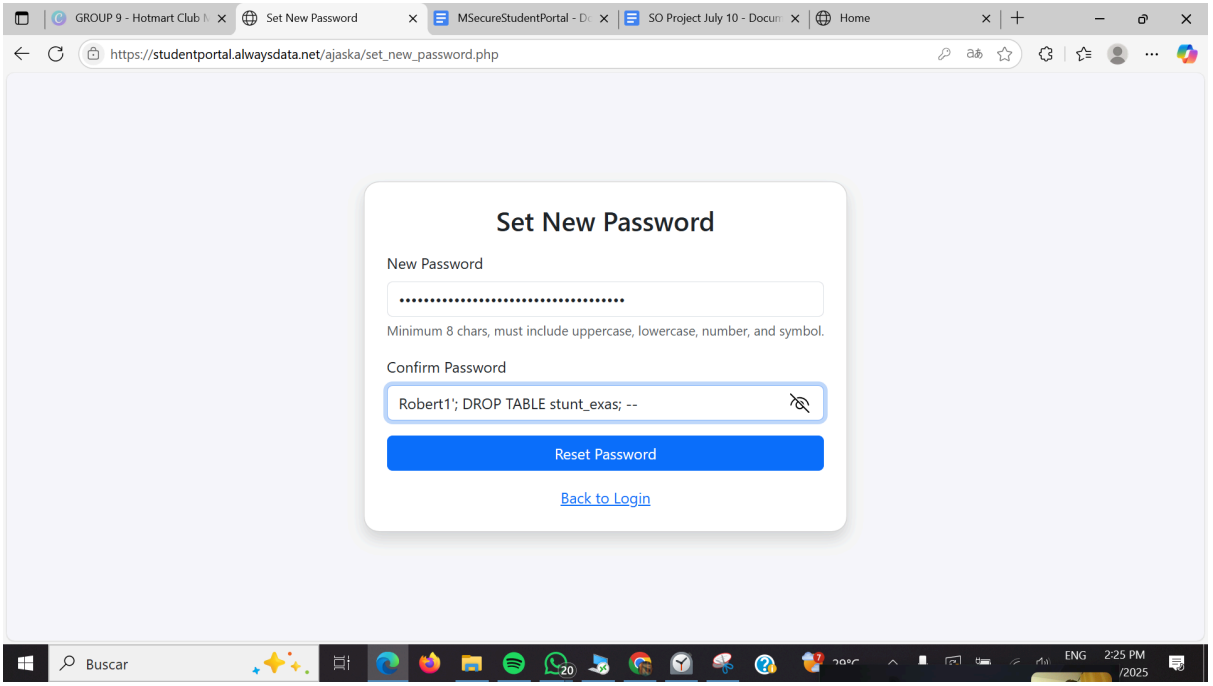
Password: ' OR 1=1 – and then ' OR 1=0 --



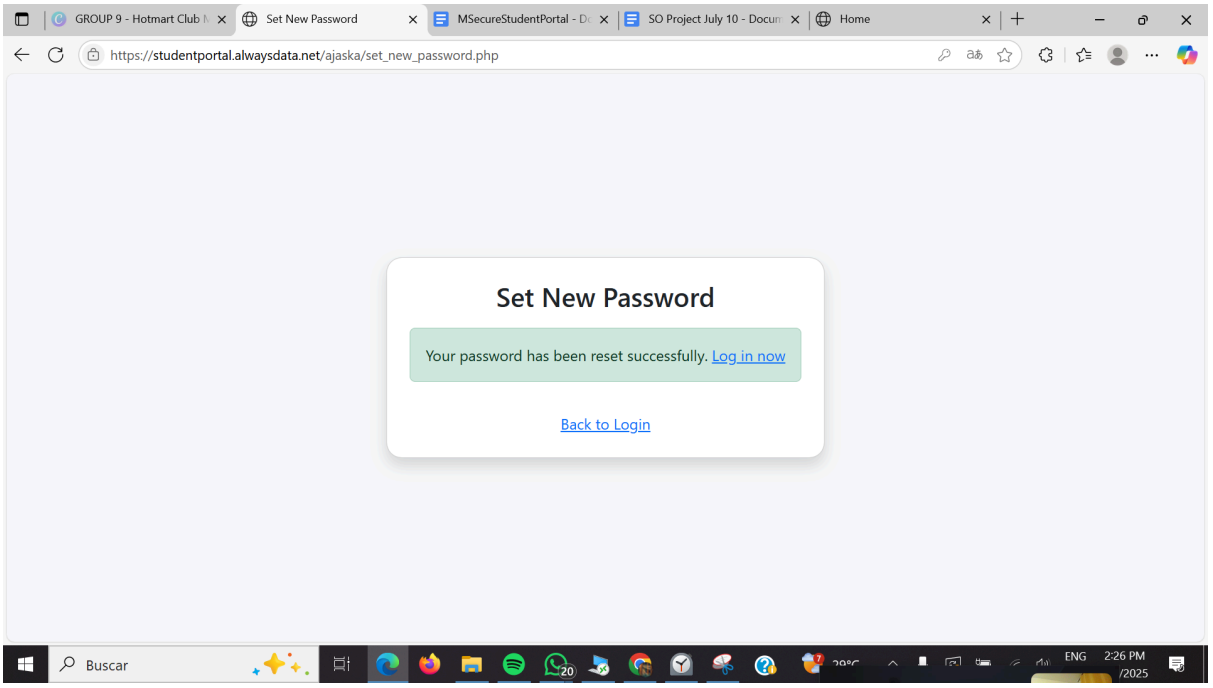


7 - TC_SQLI_07: Second-Order SQL Injection

Robert'; DROP TABLE student_exams; --
Also
Robert1'; DROP TABLE stunt_exas; --



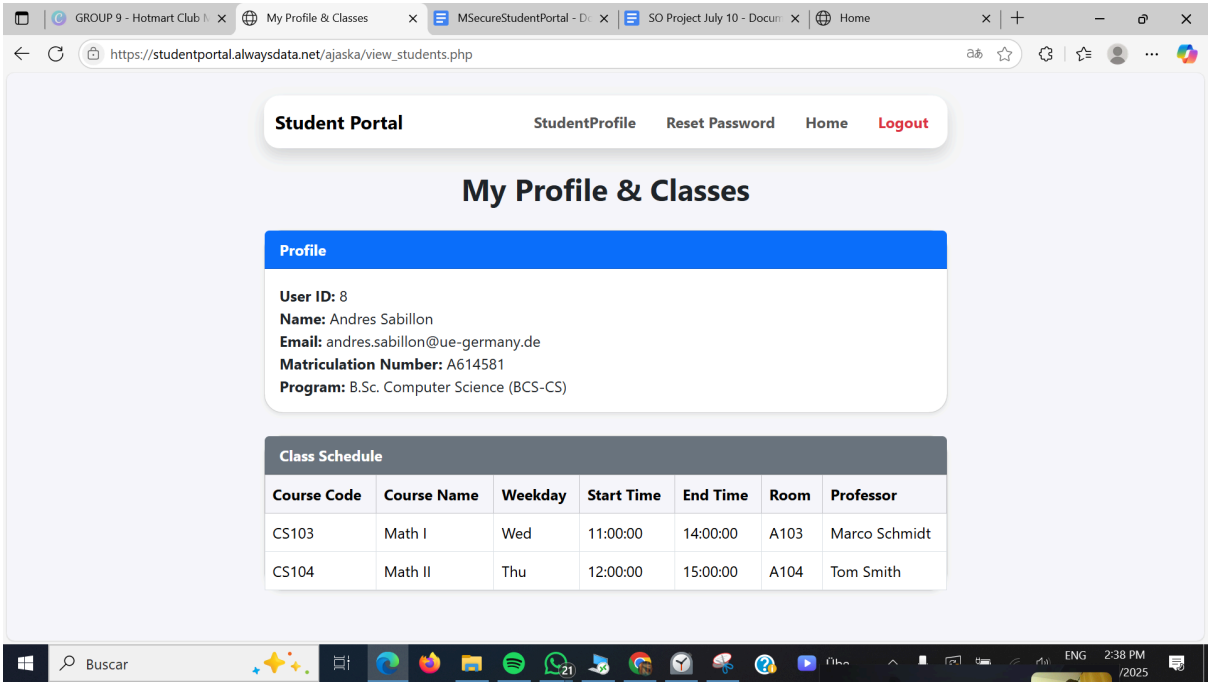
System Response: Does not take the SQL injection, instead take everything as the new password because all the message is sent in secure way.



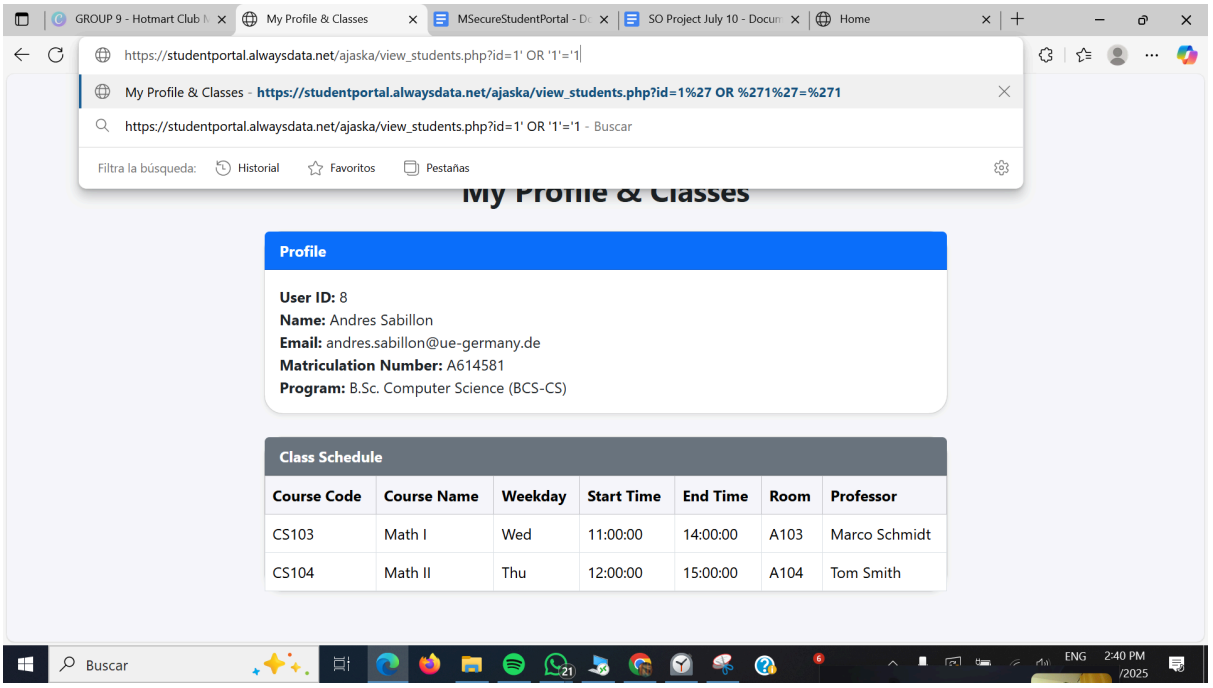
8 - TC_SQLI_08: SQL Injection via URL Parameters

Input in the URL example: /view-student.php?id=1' OR '1'='1

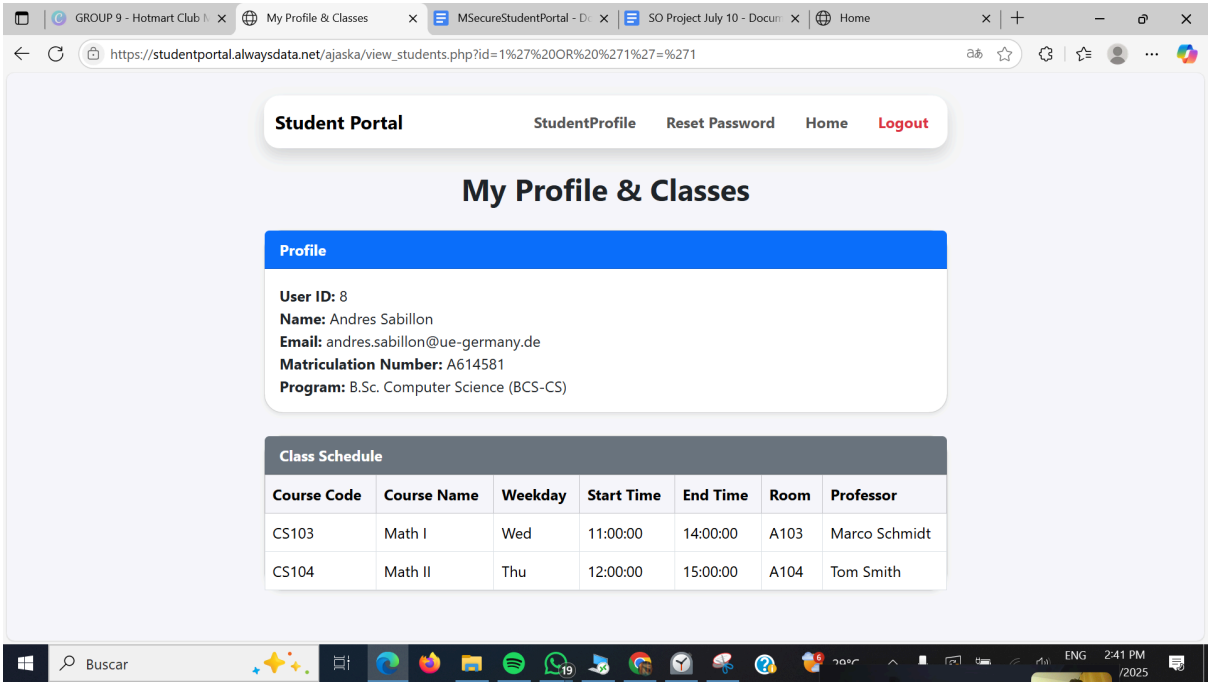
Before adding SQL injection in URL



Adding it:



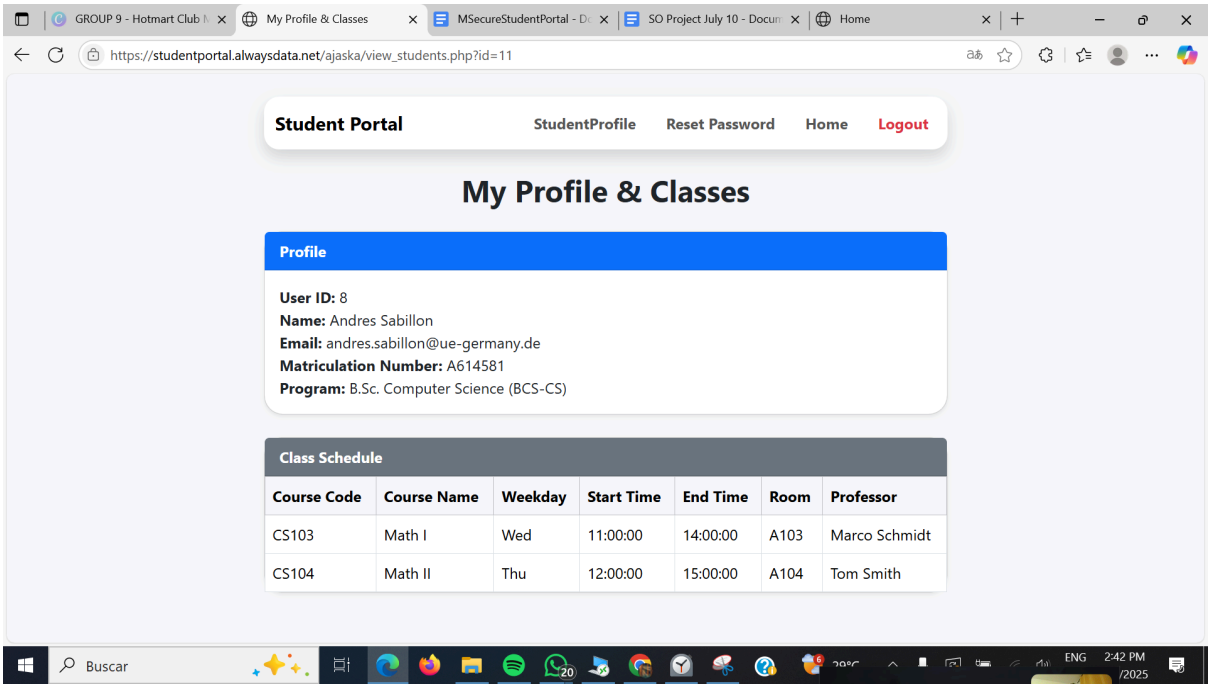
The page does not change:



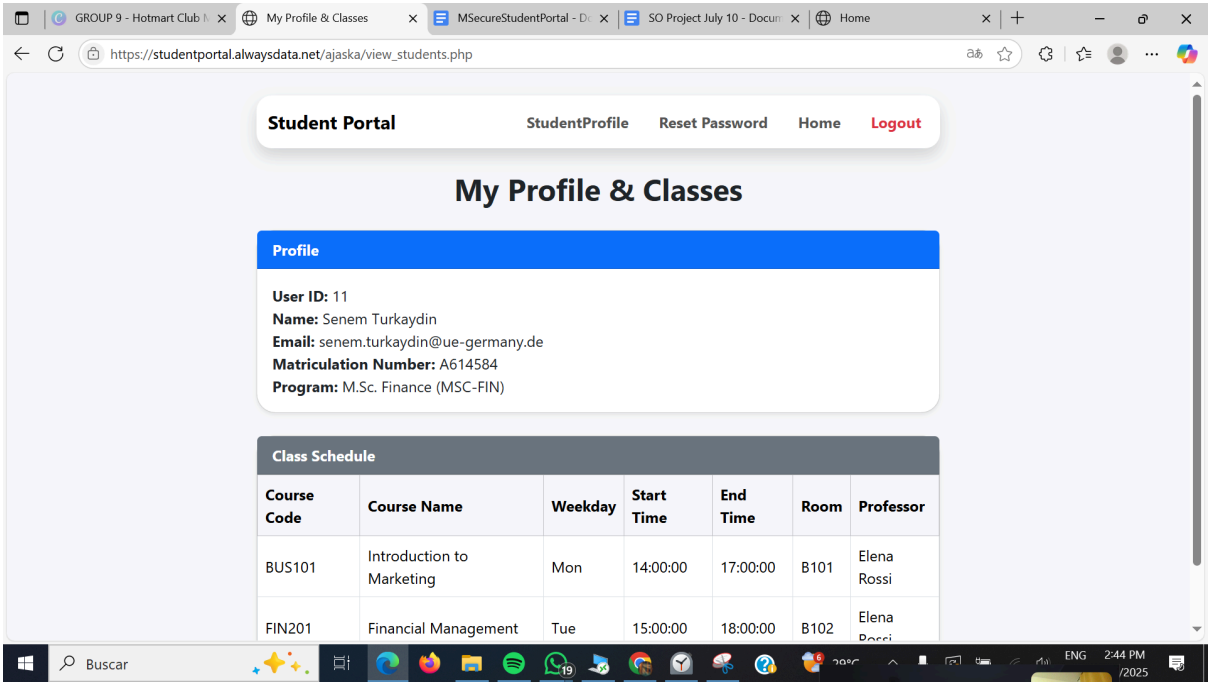
And if we try to navigate with other users id neither as : ?id=11

Input: https://studentportal.alwaysdata.net/ajaska/view_students.php?id=11

The page will not change or navigate in other user data

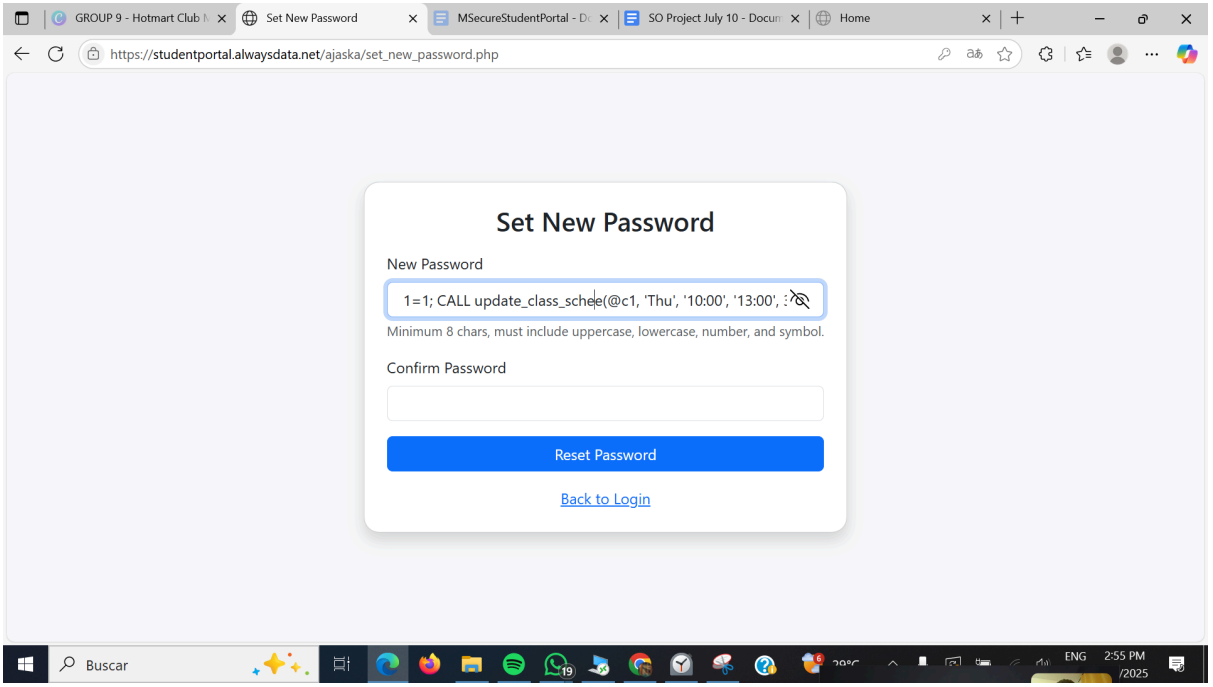


And the real data of user 11 if we login properly is this one.

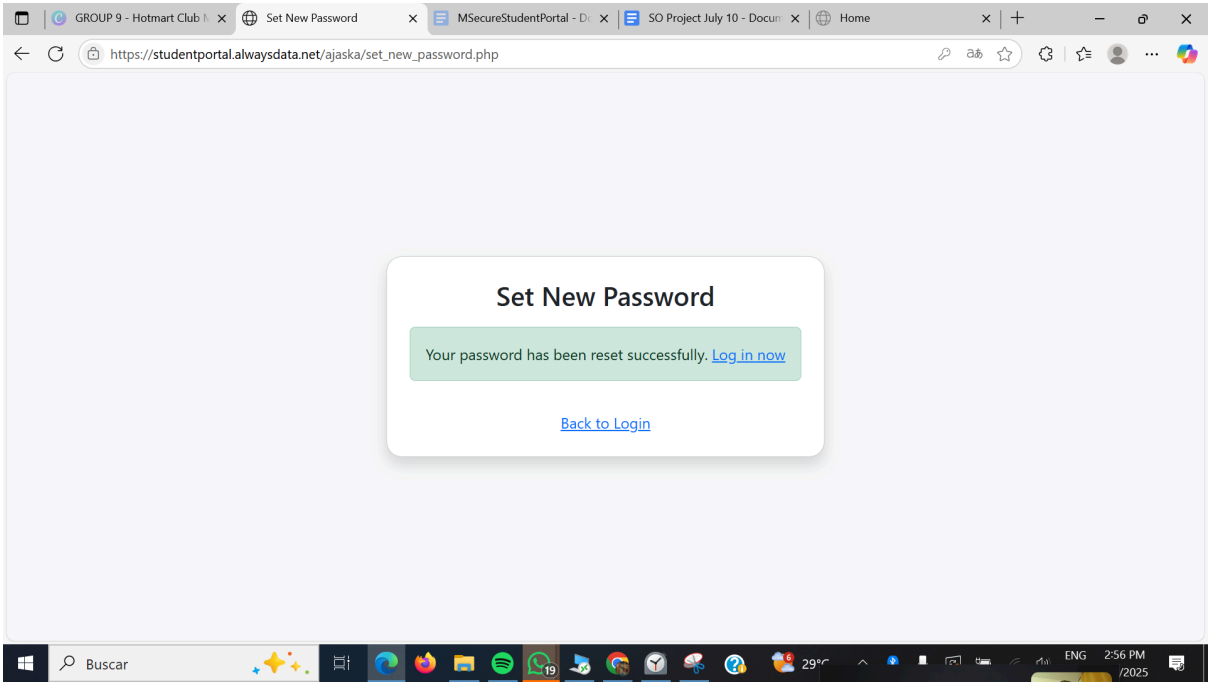


9 - TC_SQLI_09: SQLi via Stored Procedures

Input: '; password' OR 1=1; CALL update_class_schedule(@c1, 'Thu', '10:00', '13:00', 3);--

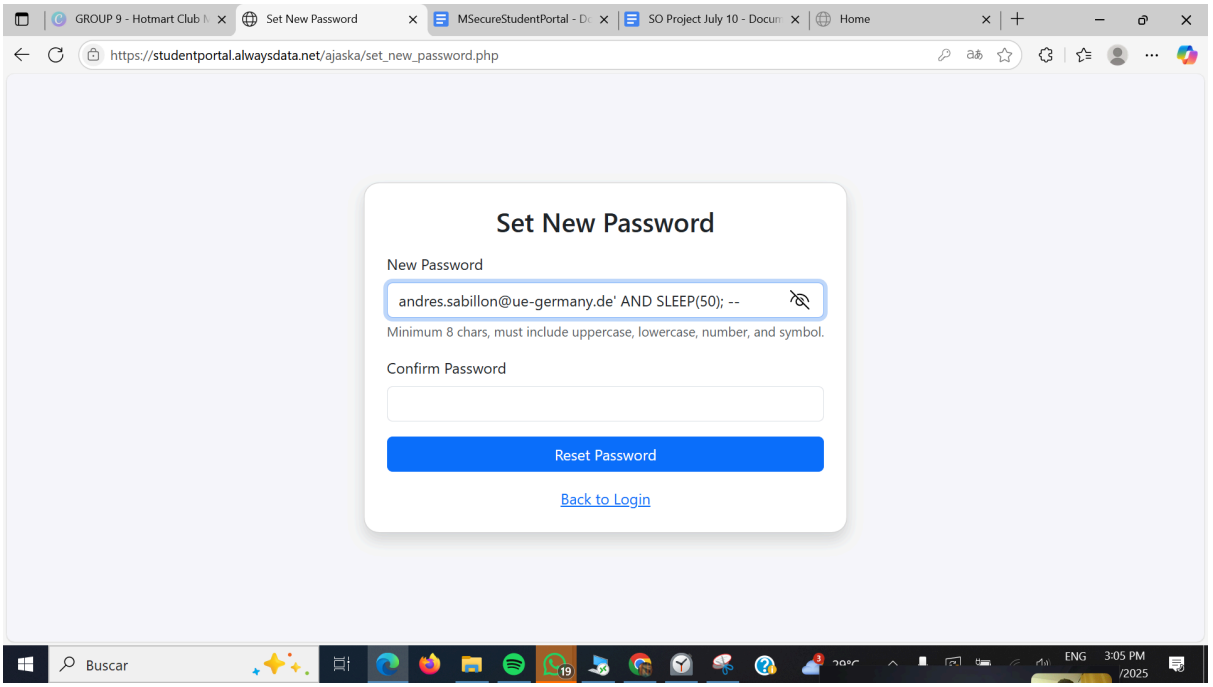


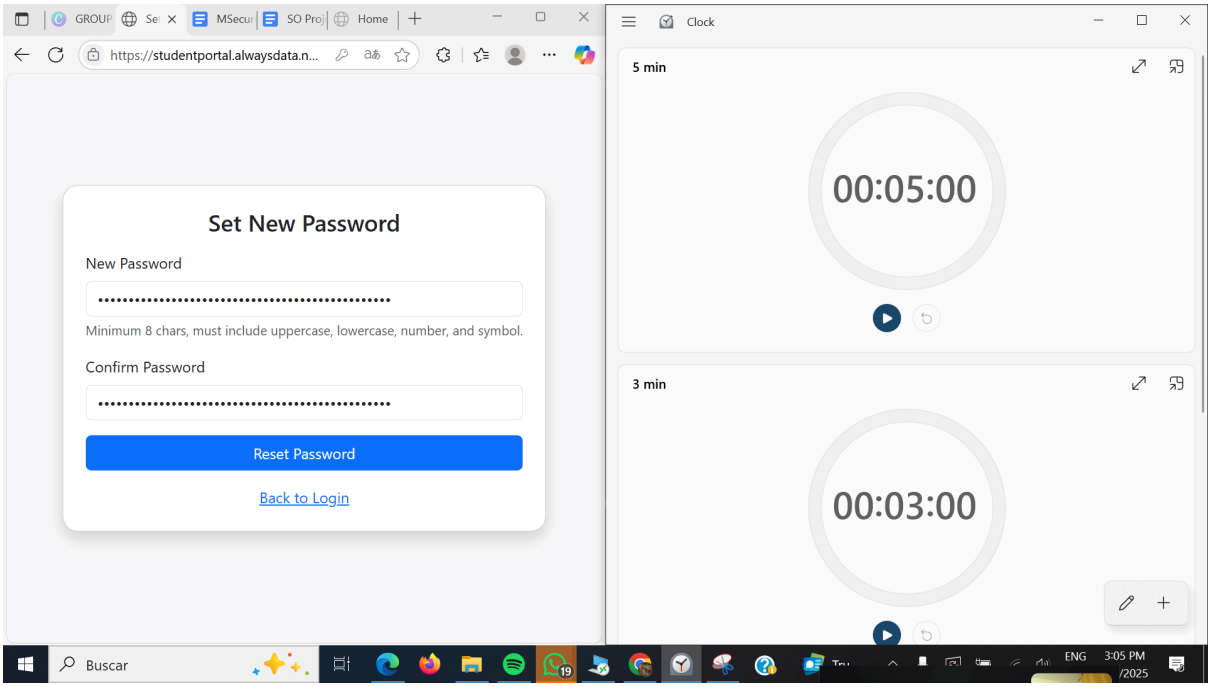
It updates the password, does not take the SQL injection.



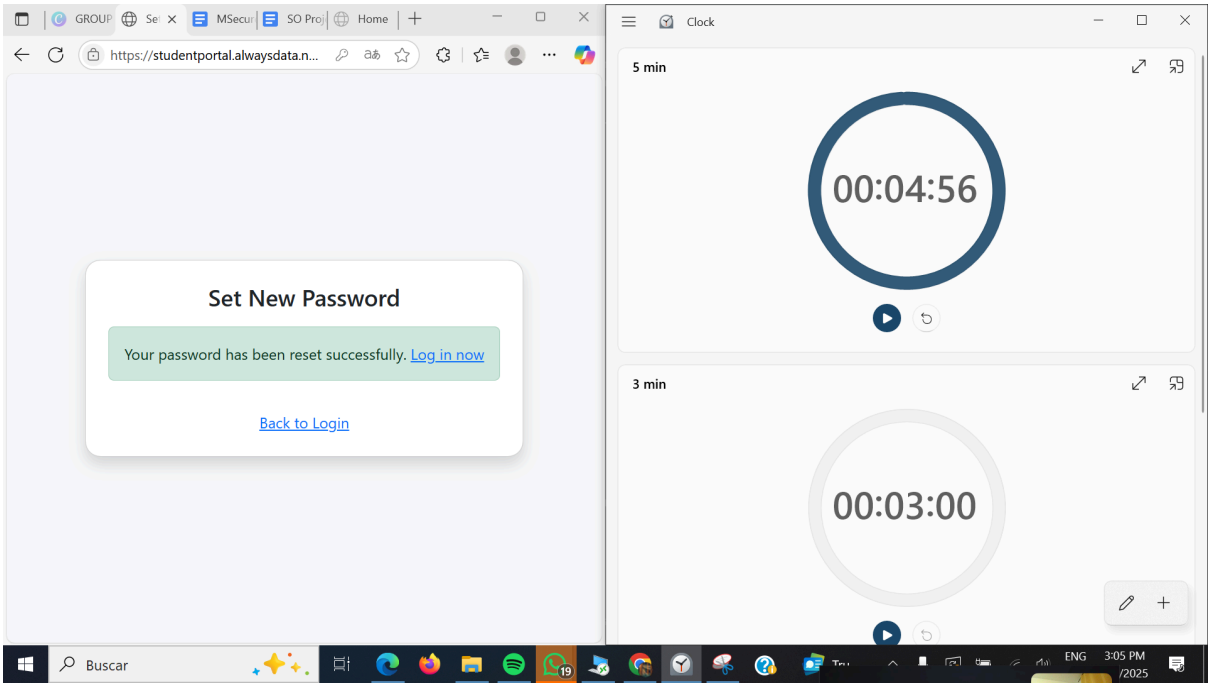
10 - TC_SQLI_10: Time-Based Blind SQL Injection

Username: andres.sabillon@ue-germany.de' AND SLEEP(50); --
Password: irrelevant



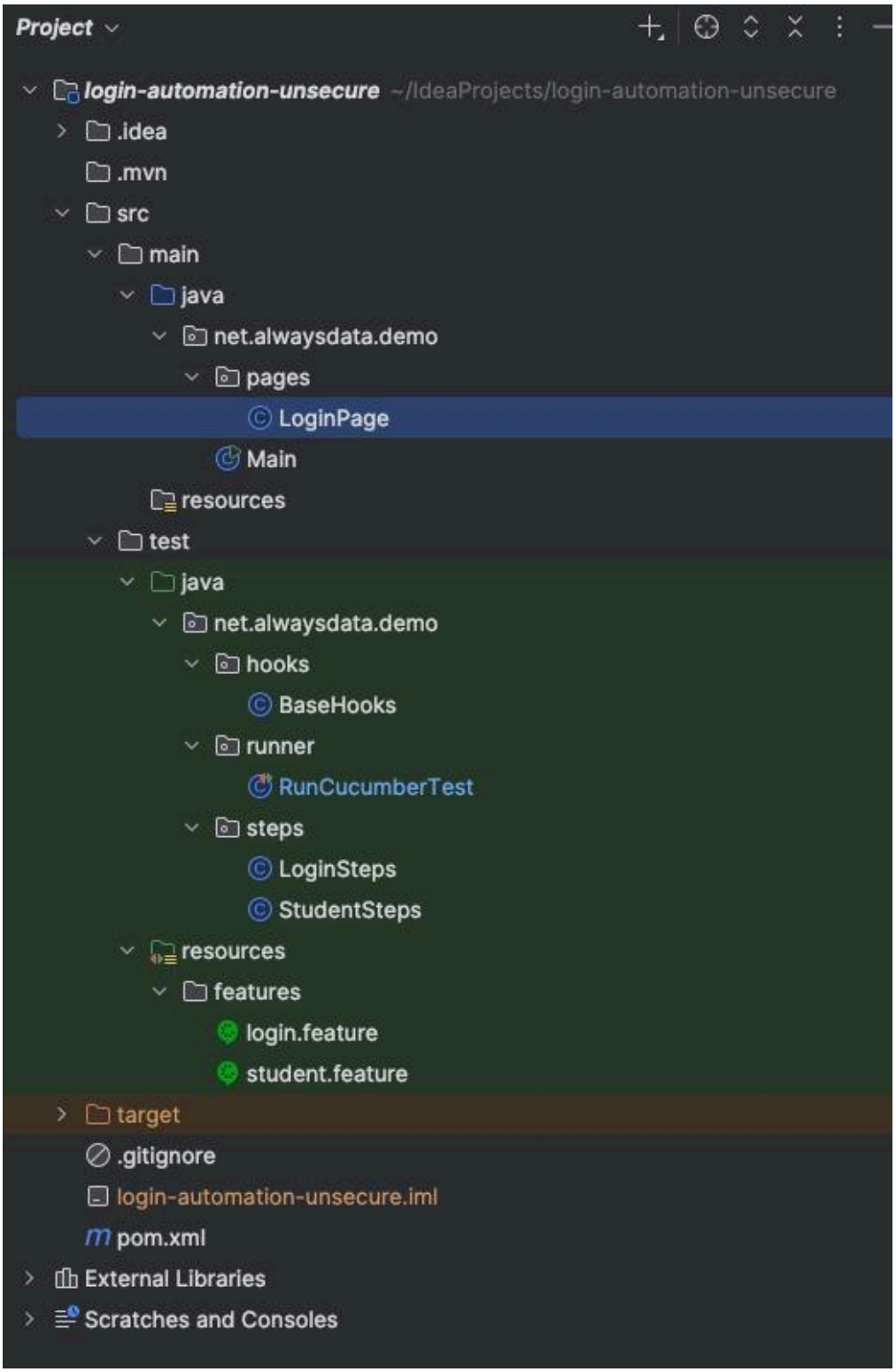


The password was reseted and he SQL injection was not successful:



All the manual tests were correct, the system is secure against SQL injections

Automatic


Project
login-automation-unsecure ~/IdeaProjects/login-automation-unsecure
.idea
.mvn
src
main
java
net.alwaysdata.demo
pages
LoginPage
Main
resources
test
java
net.alwaysdata.demo
hooks
BaseHooks
runner
RunCucumberTest
steps
LoginSteps
StudentSteps
resources
features
login.feature
student.feature
target
.gitignore
login-automation-unsecure.iml
pom.xml
External Libraries
Scratches and Consoles

```java
package net.alwaysdata.demo.pages;

import ...

public class LoginPage {  3 usages

    private final WebDriver driver;  7 usages

    // CSS locators supplied by you
    private final By email    = By.cssSelector("input[name='email']");  2 usages
    private final By password = By.cssSelector("input[name='password']");  2 usages
    private final By loginBtn = By.cssSelector("div.card form button");  1 usage

    public LoginPage(WebDriver driver) { this.driver = driver; }  1 usage

    public void open() {  1 usage
        driver.get("https://studentportal.alwaysdata.net/ajaska/");

    }

    public void loginAs(String user, String pass) {  1 usage
        driver.findElement(email).clear();
        driver.findElement(email).sendKeys(user);
        driver.findElement(password).clear();
        driver.findElement(password).sendKeys(pass);
        driver.findElement(loginBtn).click();
    }
}
```

```java
package net.alwaysdata.demo.hooks;

import ...

public class BaseHooks {  SenemTElmali

    private static WebDriver driver;  6 usages

    @Before  SenemTElmali
    public void setUp() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @After  SenemTElmali
    public void tearDown() {
        if (driver != null) {
            driver.quit();
            driver = null;
        }
    }

    public static WebDriver getDriver() { return driver; }
}
```

```java
package net.alwaysdata.demo.runner;

import org.junit.platform.suite.api.ConfigurationParameter;
import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;

import static io.cucumber.junit.platform.engine.Constants.GLUE_PROPERTY_NAME;
import static io.cucumber.junit.platform.engine.Constants.PLUGIN_PROPERTY_NAME;

@Suite  SenemTElmali *
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(
        key   = GLUE_PROPERTY_NAME,
        value = "net.alwaysdata.demo"
)
@ConfigurationParameter(
        key   = PLUGIN_PROPERTY_NAME,
        value = "pretty, html:target/cucumber-report.html"
)
public class RunCucumberTest {

}
```

```java
package net.alwaysdata.demo.steps;

import ...

public class LoginSteps {  SenemTElmali

    private WebDriver driver;   // Başta null  6 usages
    private LoginPage login;  3 usages

    @Given("the user is on the login page")  SenemTElmali
    public void theUserIsOnLoginPage() {

        driver = BaseHooks.getDriver();
        login  = new LoginPage(driver);
        login.open();
    }

    @When("the user logs in with {string} and {string}")  SenemTElmali
    public void theUserLogsIn(String email, String pass) { login.loginAs(email, pass); }

    @Then("the login should be {string}")  SenemTElmali
    public void loginShouldBe(String result) {
        String expectedUrl = "";
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(3));

        if ("success".equalsIgnoreCase(result)) {
            expectedUrl = "home.php";
            wait.until(ExpectedConditions.urlContains(expectedUrl));
            assertTrue(driver.getCurrentUrl().endsWith(expectedUrl));
        } else if ("fail".equalsIgnoreCase(result)) {
            expectedUrl = "index.php";
            wait.until(ExpectedConditions.urlContains(expectedUrl));
            assertTrue(driver.getCurrentUrl().endsWith(expectedUrl));
        } else if ("maria-db-redirect".equalsIgnoreCase(result)) {
            expectedUrl = "index.php";
            wait.until(ExpectedConditions.urlContains(expectedUrl));
            String src = driver.getPageSource().toLowerCase();
            assertTrue( condition: src.contains("mariadb") || src.contains("sql syntax") || src.contains("warning"));
        } else {
            fail("Unknown result type: " + result);
        }
```

```java
package net.alwaysdata.demo.steps;

import ...

public class StudentSteps {

    @When("the user navigates to student view page with ID {int}")
    public void theUserNavigatesToStudentViewPage(int id) {
        BaseHooks.getDriver()
                .get("https://studentportal.alwaysdata.net/ajaska/view_students.php?id=" + id);
    }

    @Then("the first list item should display ID {string}")
    public void theFirstListItemShouldDisplayId(String expectedText) {
        WebDriver driver = BaseHooks.getDriver();
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        String actual = wait.until(ExpectedConditions
                .presenceOfElementLocated(By.xpath( xpathExpression: "/html/body/div[1]/div[1]/div[2]/div[1]"))) WebElement
                .getText() String
                .trim();

        assertEquals(String.valueOf(expectedText), actual,
                message: "Student ID in first <li> is incorrect");
    }
}
```

```gherkin
# All rows share the same steps; only <result> changes
Scenario Outline: Login with different credential sets
  Given the user is on the login page
  When the user logs in with "<email>" and "<password>"
  Then the login should be "<result>"

  Examples: Login test matrix
    | case                    | email                       | password    | result  |
    | valid credentials       | senem.turkaydin@ue-germany.de | STur_2025pass!       | success |
    | wrong password          | senem.turkaydin@ue-germany.de | wrongpass  | fail    |
    | wrong email             | wrong@gmail.com             | STur_2025pass!       | fail    |
    | empty password          | senem.turkaydin@ue-germany.de |            | fail    |
    | empty email             |                             | STur_2025pass!       | fail    |
    | sql inj 1 (always true) | ' OR 1=1 --                 | ' OR 1=1 --    | fail |
    | sql inj 2 (always true) | ' or ''='                   | ' or ''='    | fail |
    | sql inj 3               | admin@domain.com' --          | anything    | fail  |
    | sql inj 4     | UNION SELECT creditCardNumber,1,'admin' FROM CreditCardTable -- | irrelevant | fail |
    | sql inj 5               | admin@domain.com' AND SLEEP(5)-- | any    | fail |
    | sql inj 6               | ' OR 1=1 --          | anything    | fail |
    | sql inj 7               | ' OR 1=1 --          | ' OR 1=0 --    | fail |
```

```
Scenario Outline: Login and view student
  Given the user is on the login page
  When  the user logs in with "<email>" and "<password>"
  And   the user navigates to student view page with ID <id>
  Then  the first list item should display ID "<Userid>"

  Examples:
    | email                        | password          | id | Userid        |
    | senem.turkaydin@ue-germany.de | STur_2025pass!   | 3  | User ID: 11   |
    | senem.turkaydin@ue-germany.de | STur_2025pass!   | 4  | User ID: 11   |
```

classpath:features/login.feature

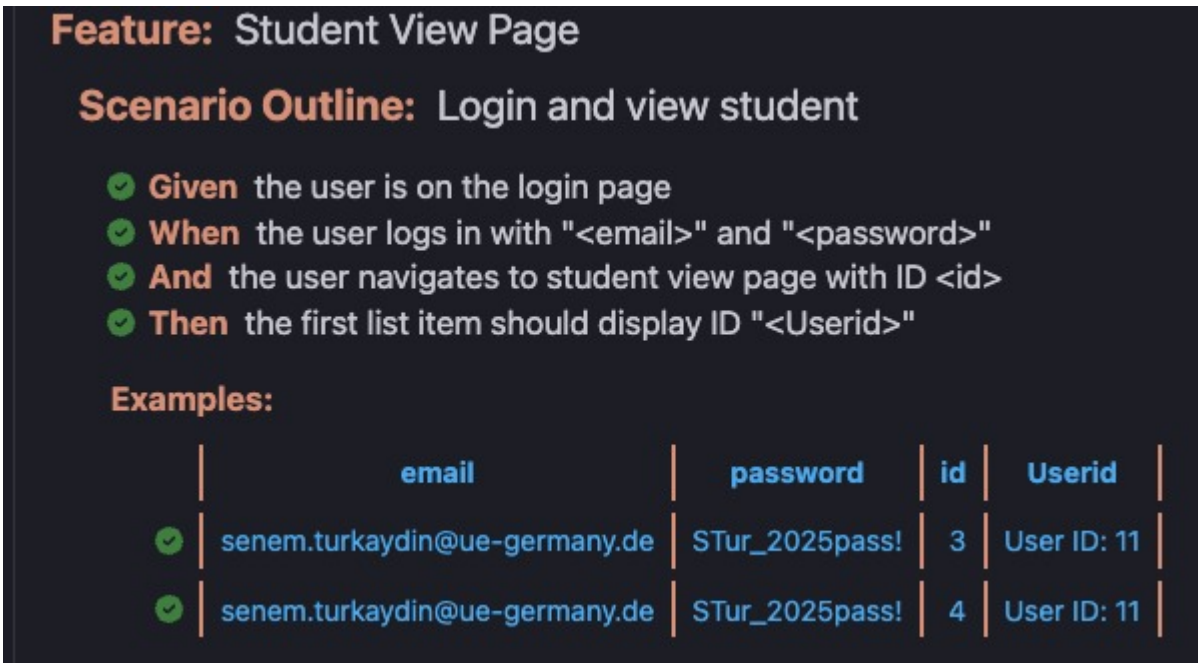
Feature: User authentication

Scenario Outline: Login with different credential sets

- **Given** the user is on the login page
- **When** the user logs in with "<email>" and "<password>"
- **Then** the login should be "<result>"

Examples: Login test matrix

case	email	password	result
valid credentials	senem.turkaydin@ue-germany.de	STur_2025pass!	success
wrong password	senem.turkaydin@ue-germany.de	wrongpass	fail
wrong email	wrong@gmail.com	STur_2025pass!	fail
empty password	senem.turkaydin@ue-germany.de		fail
empty email		STur_2025pass!	fail
sql inj 1 (always true)	' OR 1=1 --	' OR 1=1 --	fail
sql inj 2 (always true)	' or ''='	' or ''='	fail
sql inj 3	admin@domain.com' --	anything	fail
sql inj 4	UNION SELECT creditCardNumber,1,'admin' FROM CreditCardTable --	irrelevant	fail
sql inj 5	admin@domain.com' AND SLEEP(5)--	any	fail
sql inj 6	' OR 1=1 --	anything	fail
sql inj 7	' OR 1=1 --	' OR 1=0 --	fail



SQL Injection in non-secure WEB Pages

SQL Injection in web pages is a common tactic that occurs when the system expects the users to give an input in the authentication requests. The expected behavior from the users would be to provide the ‘username/email/ID” values along with the ‘password’ value. Often used in basic authentication, these two values will be included in the POST request, which is going to be sent against the system API. After the authentication and validation take place, the users can therefore continue accessing the system based on which roles and user permissions they are granted.
On the contrary, during an SQL injection attack, instead of the correct credentials, the people behind the attack will try to break the system and expose private information by triggering different errors. Those errors, when the database is poorly designed, are not properly handled, and they contain information that comes in handy for the attackers.

In the example used for testing purposes, after designing our database, in order to test its security, it was planned to inject SQL statements in order to achieve successful authentication
without having the correct credentials.

As shown in Figure 1. the student portal is requesting it’s users to provide two inputs, the email and password.

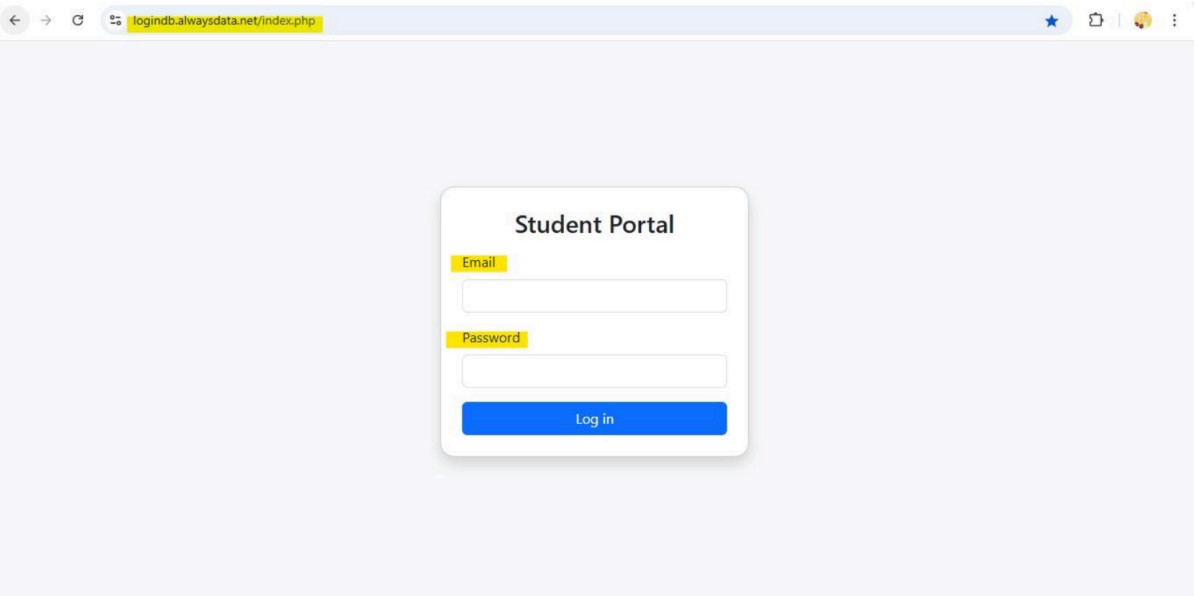


Figure 1. Authentication LogIn page

If we attempted to provide the incorrect data, an expected 'Invalid Credentials' error would get thrown, but these types of errors are already handled and do not provide any valuable insights which can be utilized in the SQL Injections attack.

In order to build the proper authentication request and bypass the browser security measures, we can gather the necessary information by observing the 'Headers' panel as illustrated in Figure 2. The information gathered is as follow:

Request URL: https://logindb.alwaysdata.net/index.php
Request Method: POST

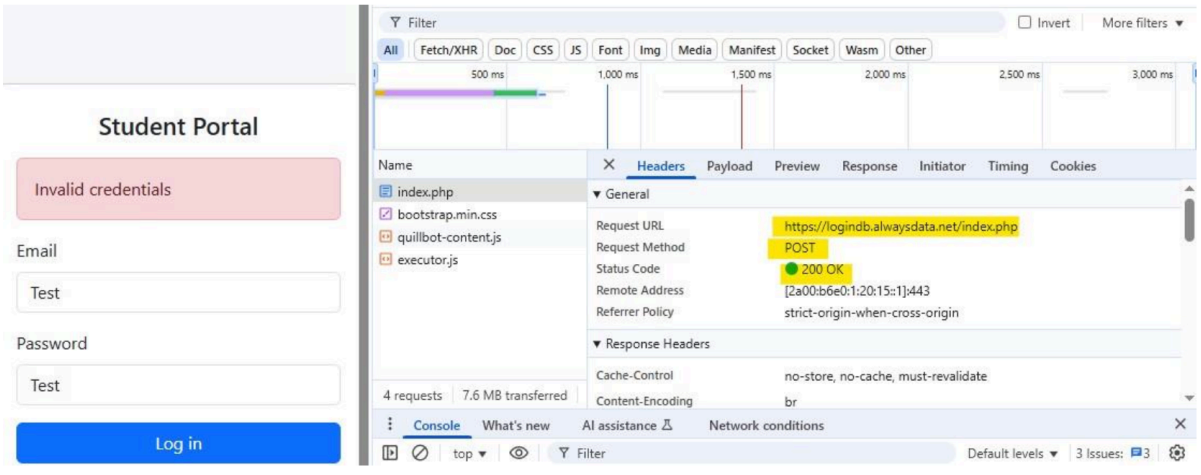


Figure 2. Illustration of a handled exception which does not break the syntax

By examining the Response tab using the Developer Tools in the Chrome browser as shown in Figure 3, we can obtain more insightful information on how to send the correct request body. In order for the SQL injection to take place successfully, the request must remain correctly sent at all attempts; therefore, the input parameters are Email/Password.

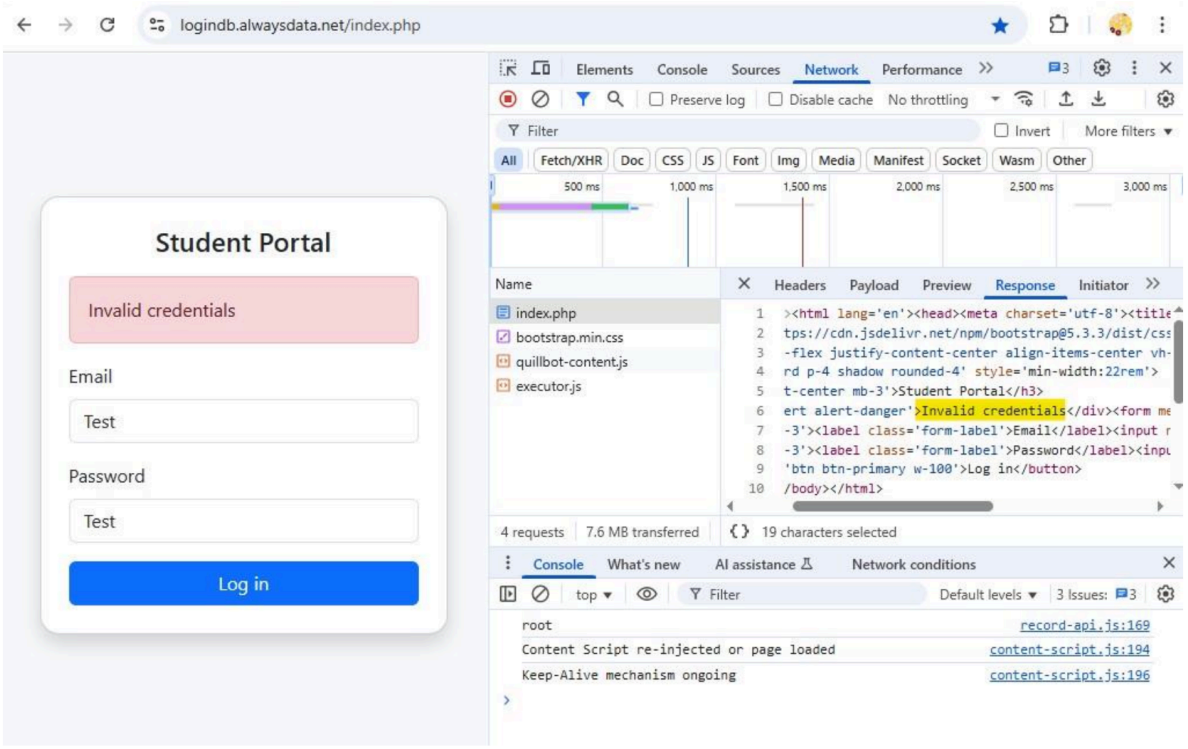


Figure 3. Response Body of the request

In this testing session, the chosen proxy tool to submit the HTTP request was Postman. At this step, we have the following information as shown in Table 1.

Request URL	https://logindb.alwaysdata.net/index.php
Request Type	POST
RequestBody	email/password

A known way on how to trigger non-handled error messages, is to include in the email or password value a ‘ (single quote) character. This is an attempt to read from the database, the expected syntax.

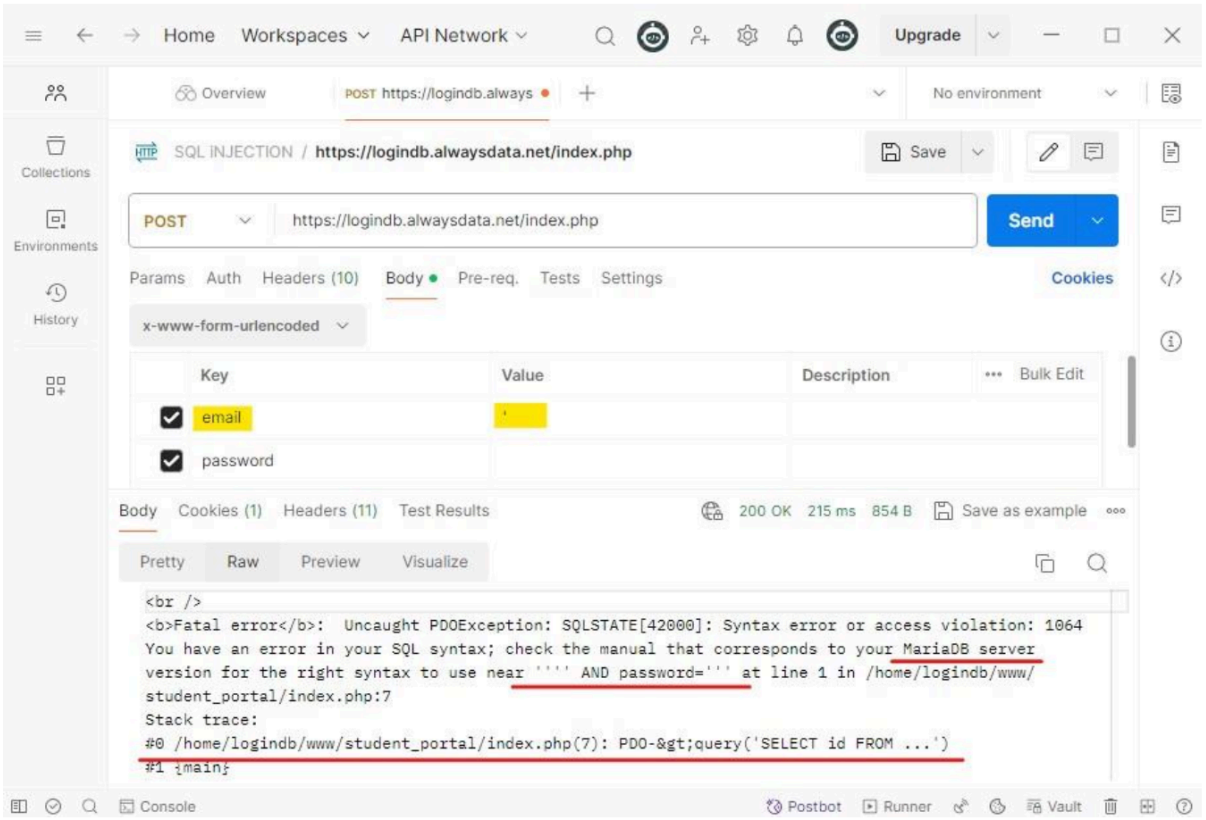


Figure 4. Triggering of unhandled error messages

As it is demonstrated in Figure 4. we have successfully broken the request, and the server response is all the details that are needed in order to further proceed with the SQL Injections attempts. This error reveals crucial information regarding the database. Based on the error message we triggered in the previous step, we know have some information about this website.

a. Is using MariaDB
b. Is expecting both conditions to be satisfied: email "" AND password=""

This information leads a hacker into re-creating the authentication code which should look similar to :

$sql = "SELECT id FROM users WHERE email = '$email' AND password = '$password'";
As it is demonstrated in the below example, there is nothing that can prevent the users not giving the wrong inputs when they are requested to complete a sign in form.
At this point, some of the classic ways of SQL Injections were tested to prove that the website does not fulfil the security requirements:

1. SQL Injection Based on 1=1 is Always True

At this attacking method, the user is attempting to validate the authentication criteria without

entering any correct credentials. To do so, it will be implemented the known 1=1 condition, which means that the system with go through the valid users until the first one is found. In order to build the correct syntax, we need to properly adjust it.

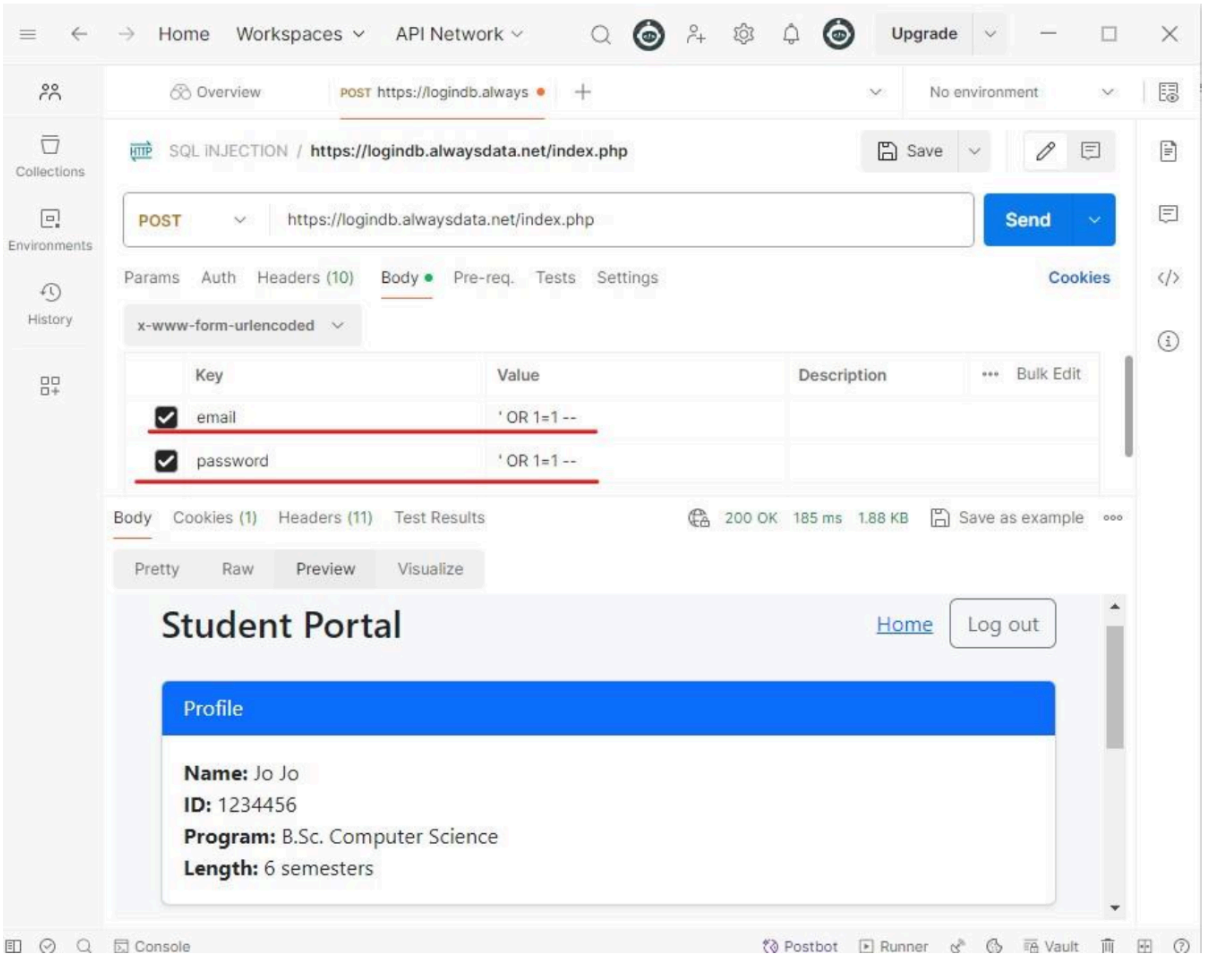
Starting by using a ‘ (single quote) character, this will make sure that the email variable is closed and the query becomes:
SELECT * FROM Users where ‘email’ .....

Here the Or syntax will serve as an OR condition since it is no longer part of the string given as an input for the email variable. Therefore the query becomes:
SELECT * FROM Users WHERE ‘email’ OR 1=1

When the same thing is done for the password field, the final query should look like:

SELECT * FROM Users WHERE ‘email’ OR 1=1 AND ‘password’ OR 1=1

This will result in logging in as the most recent user in the database as shown in picture 5



Picture 5. First SQL Injection attempt

2. SQL Injection Based on ""="" is Always True

When the user is requested to give as an input the authentication credentials, and they are correctly inserted then the query is transformed as:
SELECT * FROM Users WHERE ‘email’ = ‘correctemail’ AND ‘password’ = ‘Correctpassword’

A known form of SQL Injections is to get access to user names and passwords that are registered in the database by inserting " OR ""=" in the request body.
The query in that case would be transformed into:

SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""

These modifications will not break the request, and it will be successfully passed to the server as shown in Figure 6.

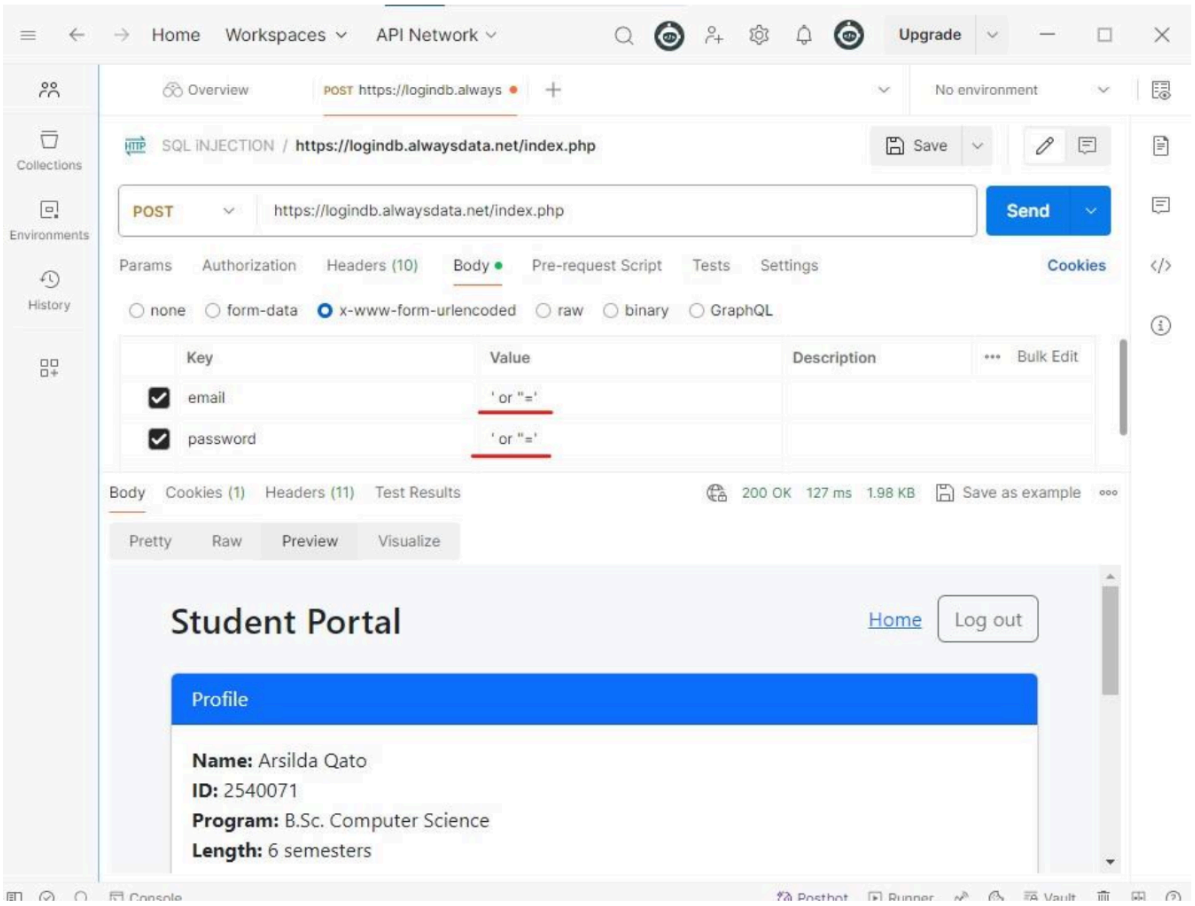


Figure 6. Second attempt of SQL Injection

3. SQL Injection Based on Batched SQL Statements

Based on MariaDB documentation, using Batch Operation, which consists in grouping multiple SQL commands to send against the database, is not only allowed but also serves as a recommended approach when it comes to improving overall performance, this could be done by using the character ; (semicolon)

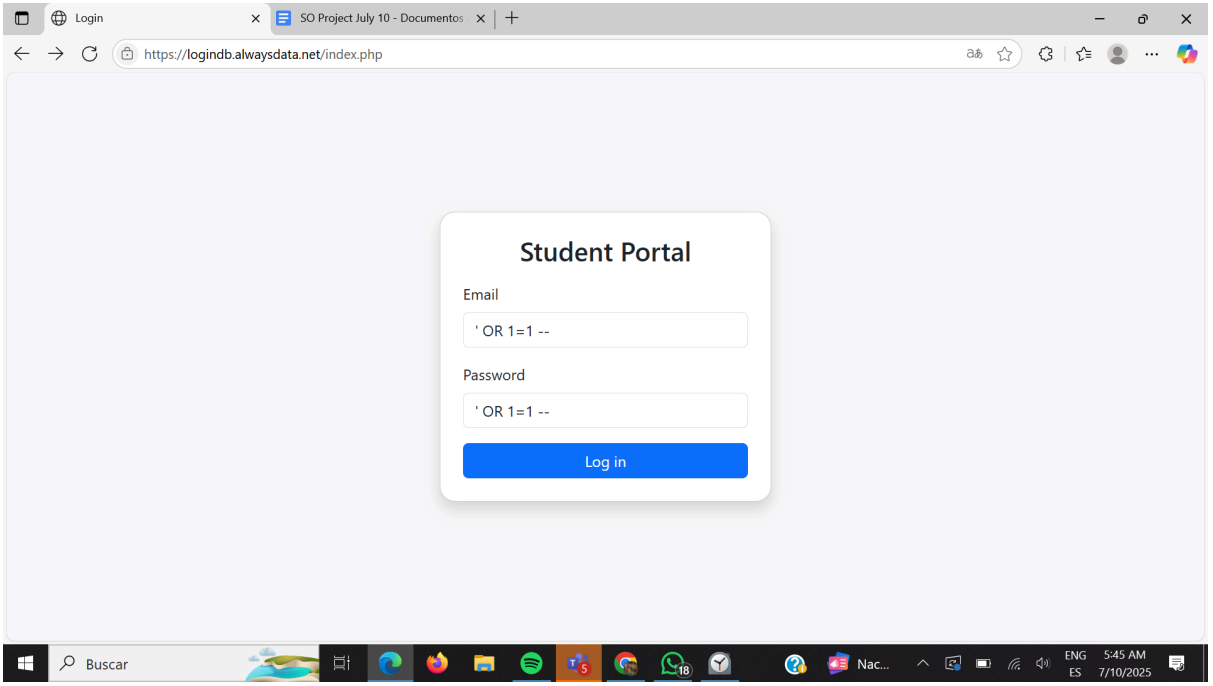
The SQL statement below use all the allowed potential without breaking the request, and will read all rows from the "Users" table until the condition is met, then will delete the "Exams" table.

SELECT * FROM Users WHERE ‘email' or ''='; DROP TABLE Exams.....

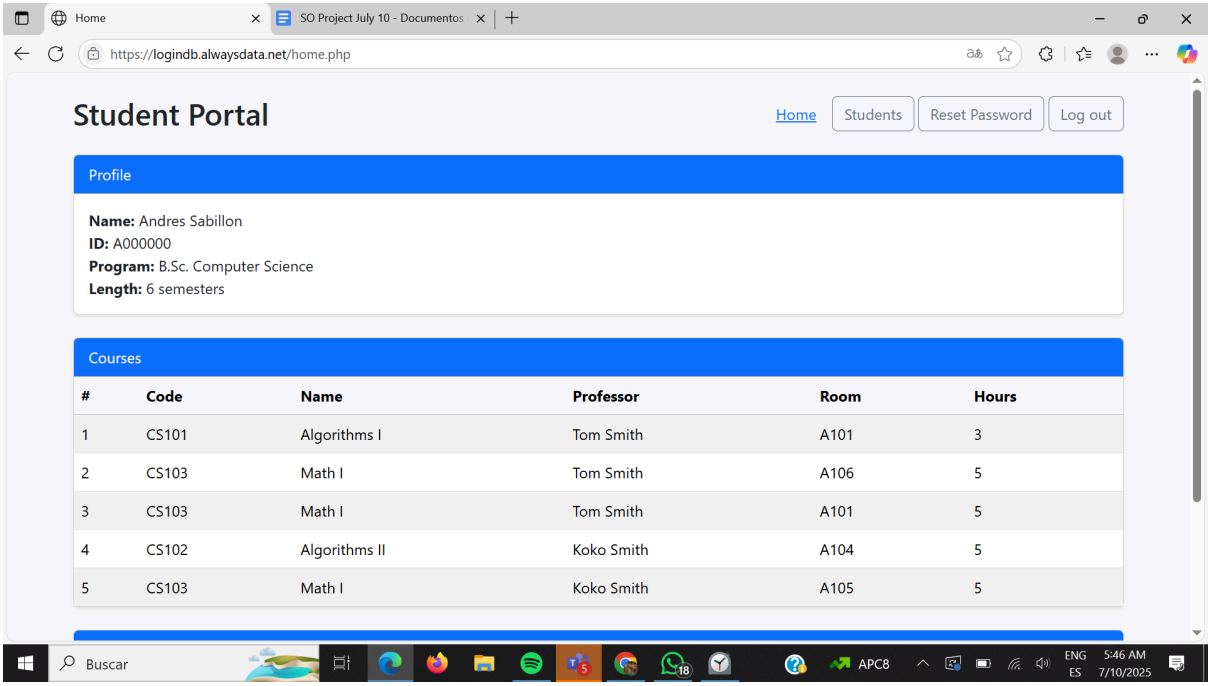
More SQL Injection in Unsecure WEB Pages Made by Andres with other scenarios.

This ones needed to be tested in the unsecure site to understand them and design

4 - TC_SQLI_04: Error-Based SQL Injection
username: ' OR 1=1 –
password: ' OR 1=1 –

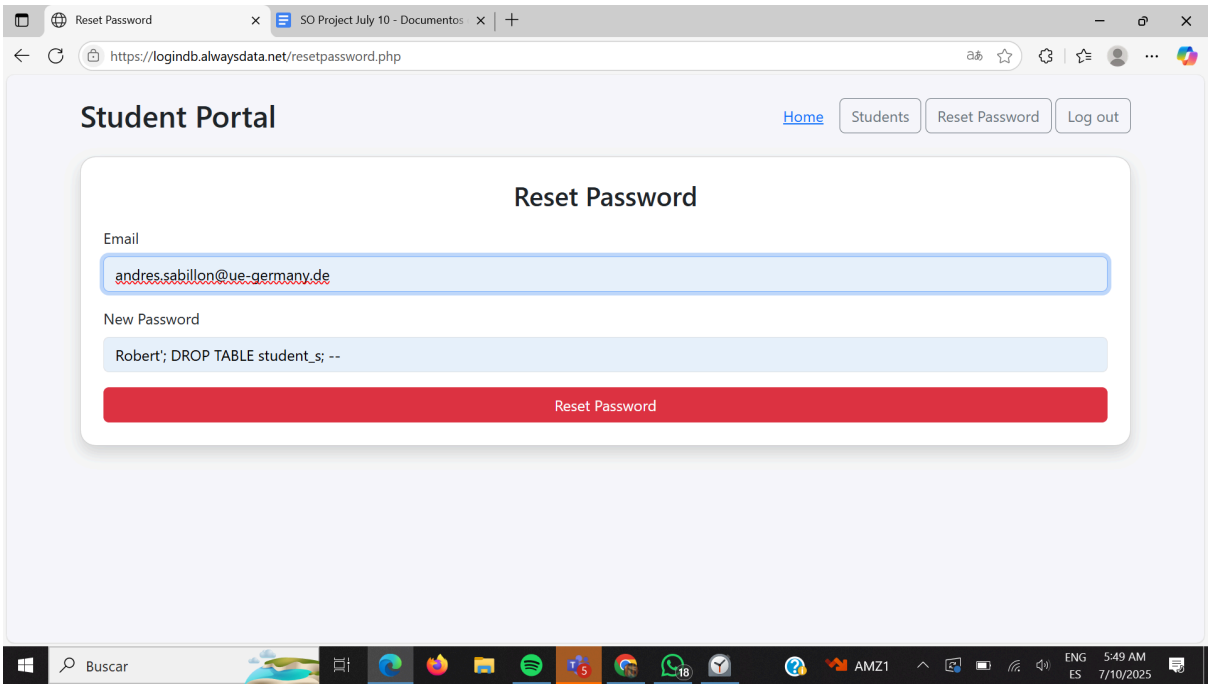


Logged in

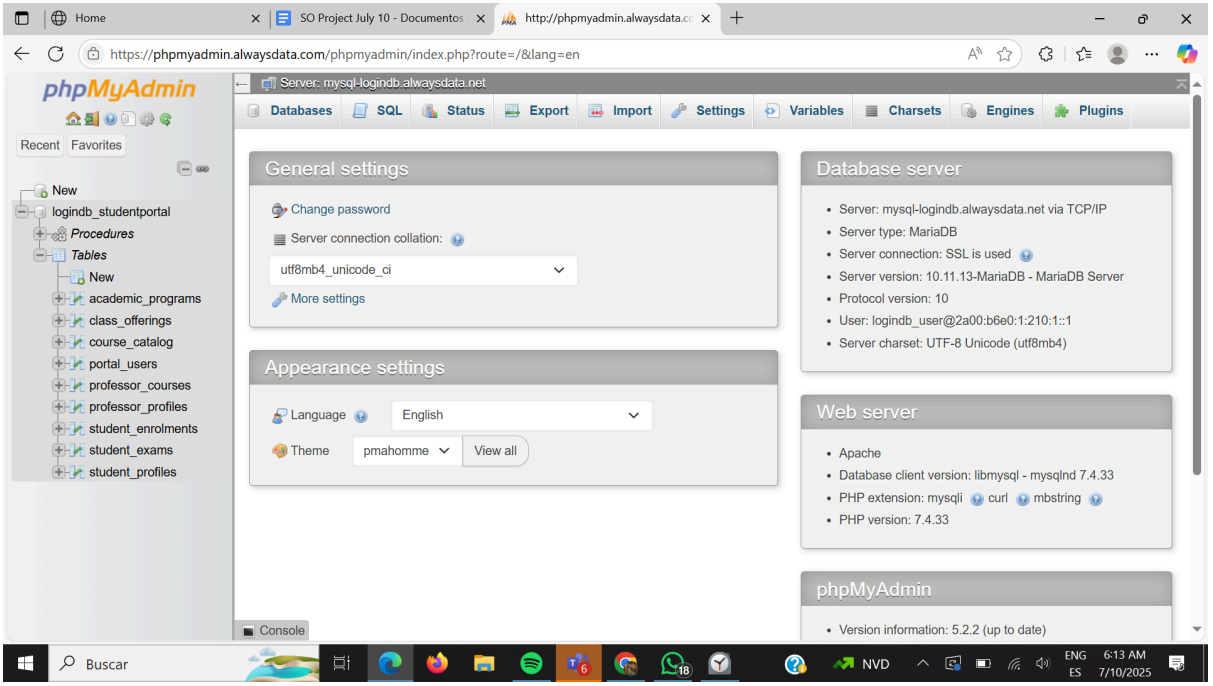


7 - TC_SQLI_07: Second-Order SQL Injection

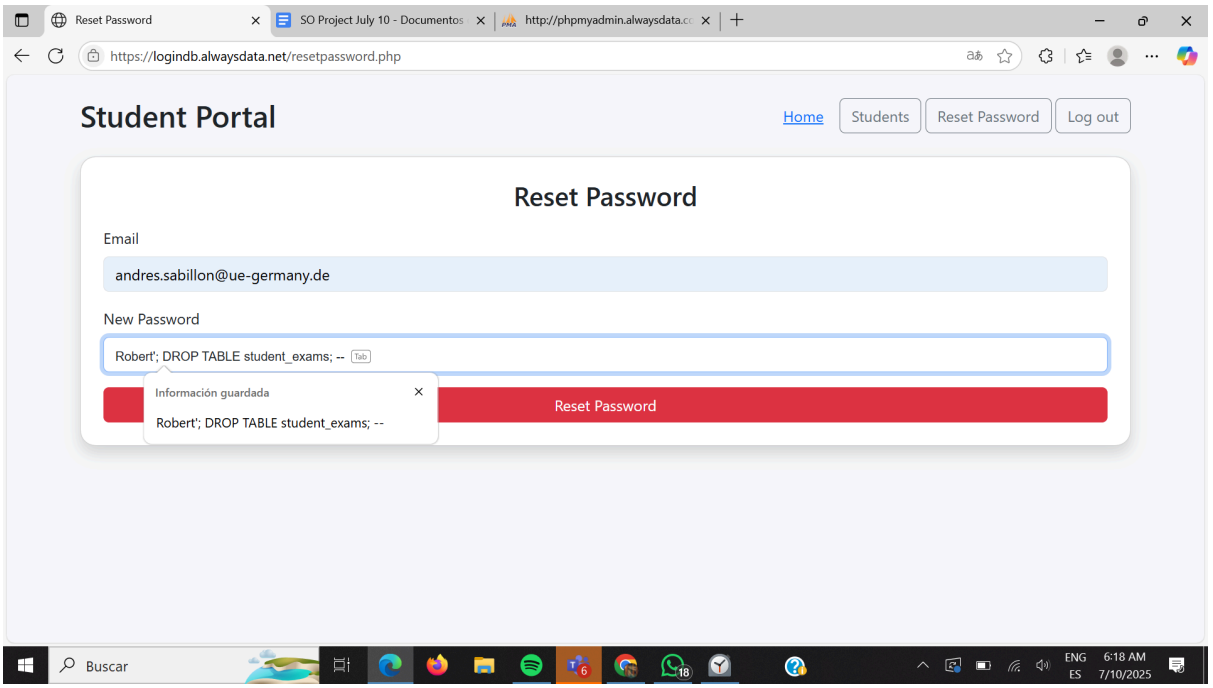
Robert'; DROP TABLE student_exams; --
Also
Robert1'; DROP TABLE stunt_exas; --



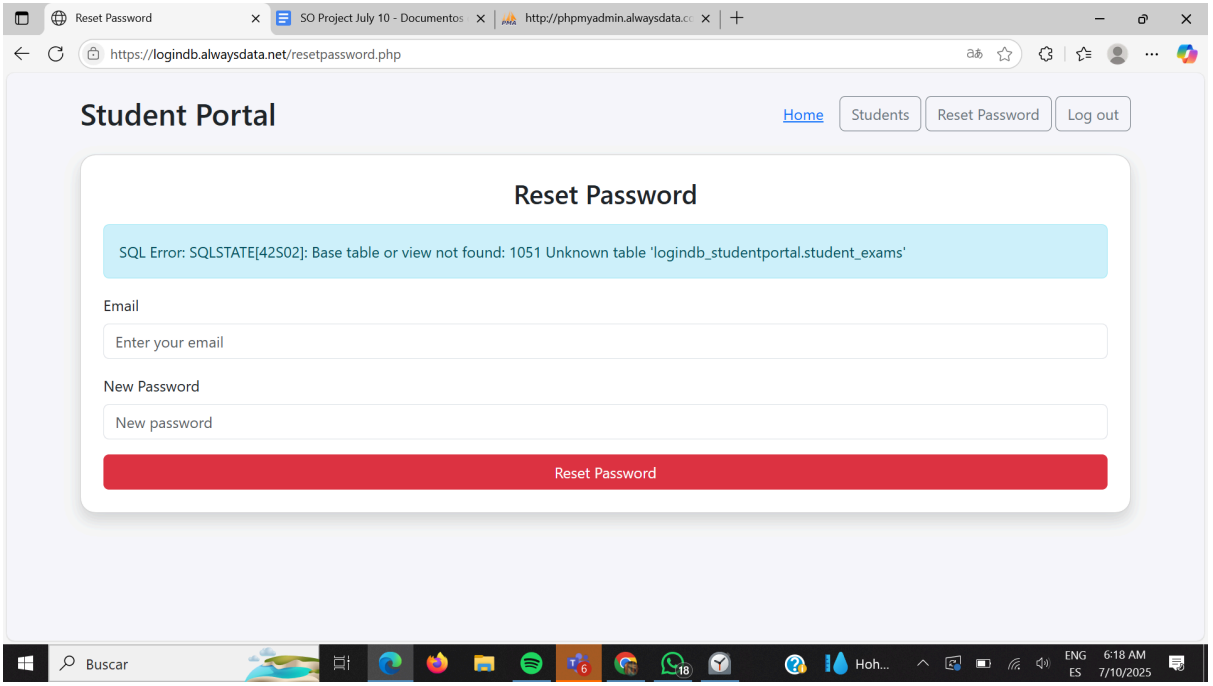
Now we are deleting the table student_exams
Here is the DB before the attack



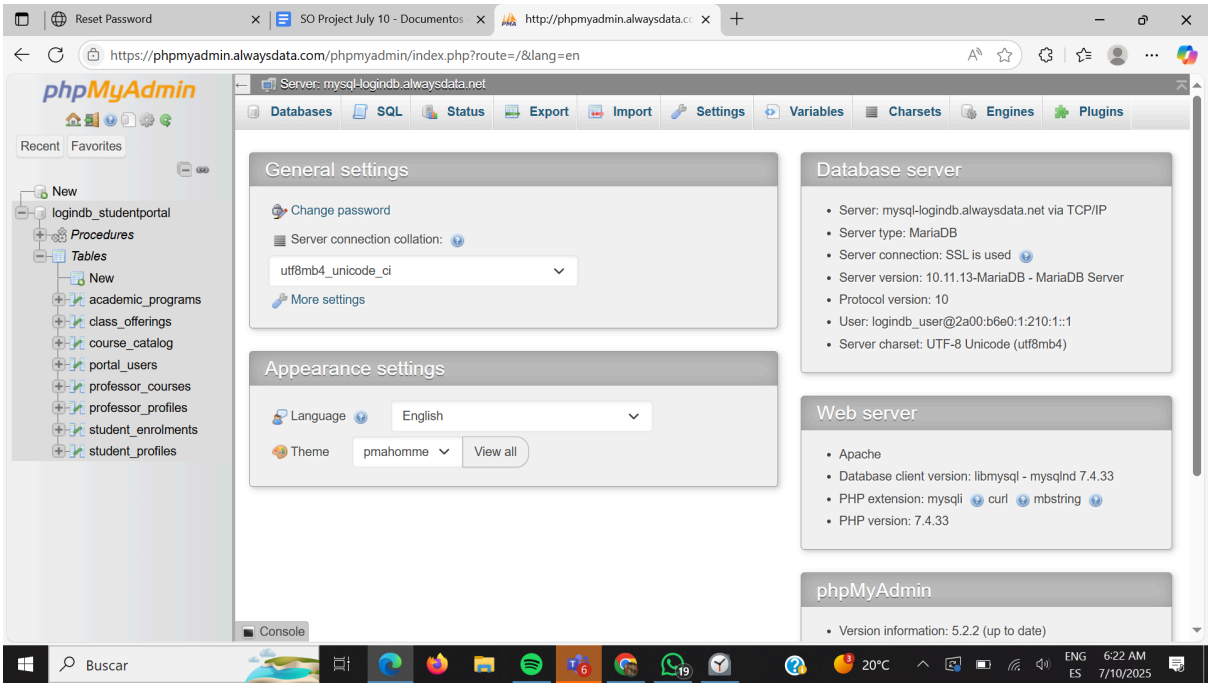
Doing the injection
With: Robert'; DROP TABLE student_exams; --



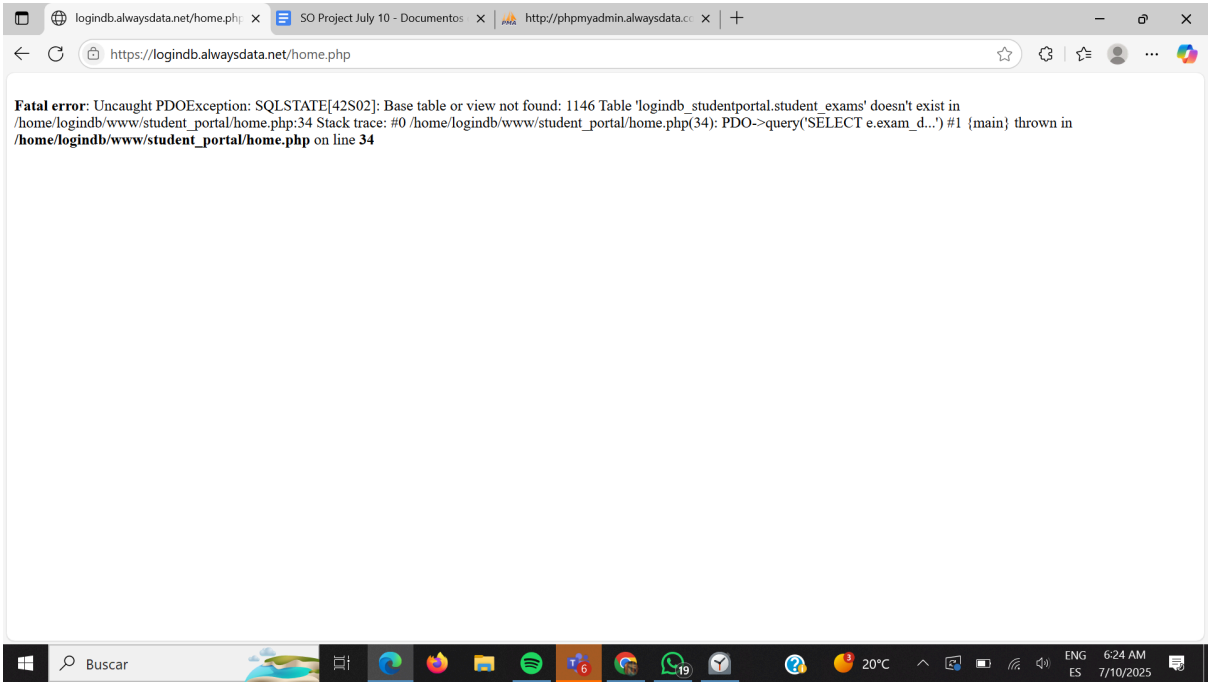
First show SQL injection, and in that moment the table is deleted
And is we try to do the injection again this will be shown



Database after the injection
Student_exams table is deleted



And home page does not load because of it

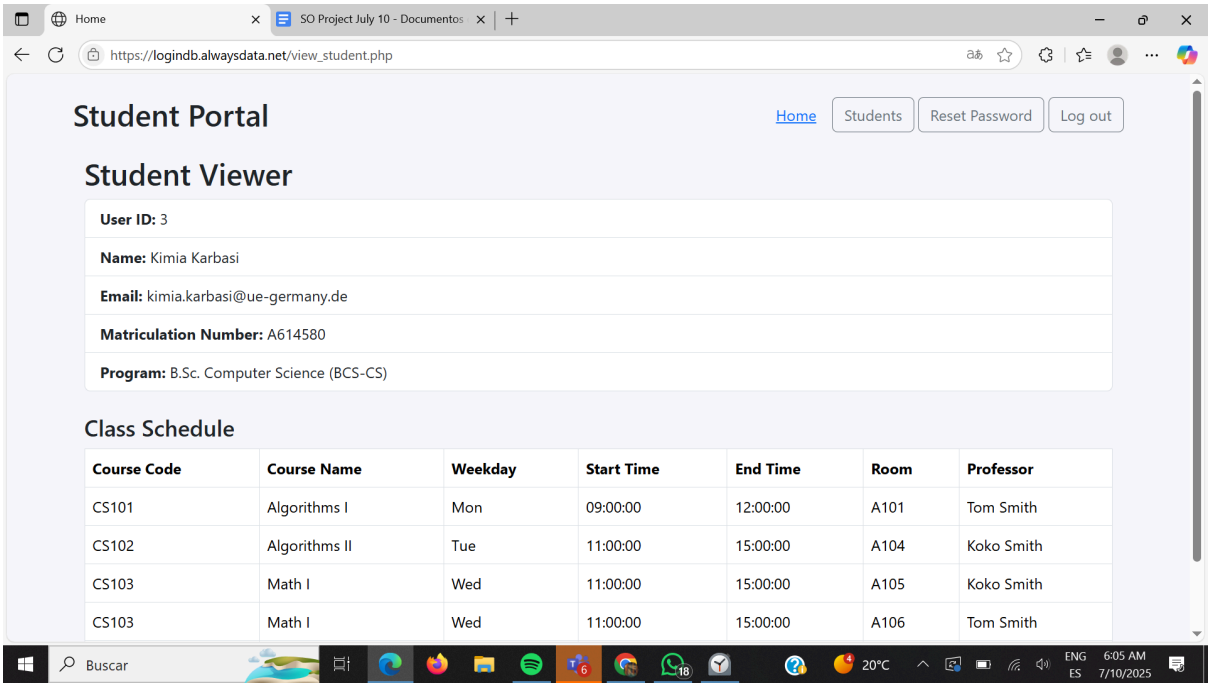


We have to restore the database to keep everything working

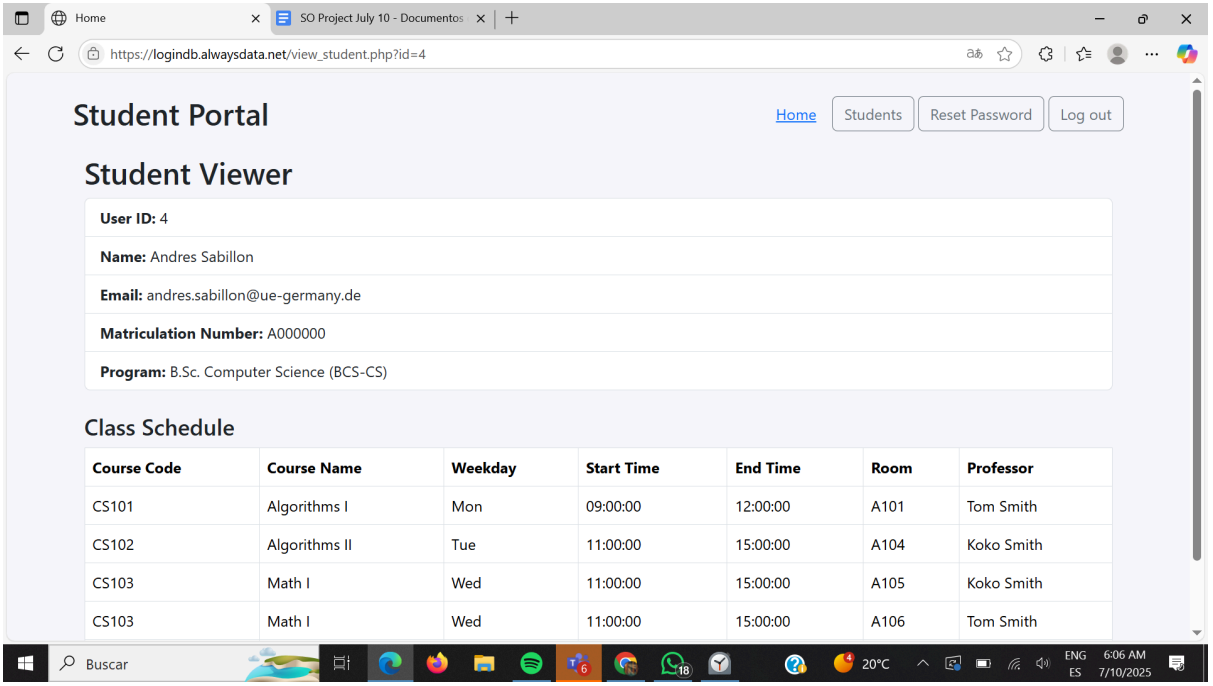
8 - TC_SQLI_08: SQL Injection via URL Parameters

Input in the URL example: /view-student.php?id=1' OR '1'='1

Before adding SQL injection in URL

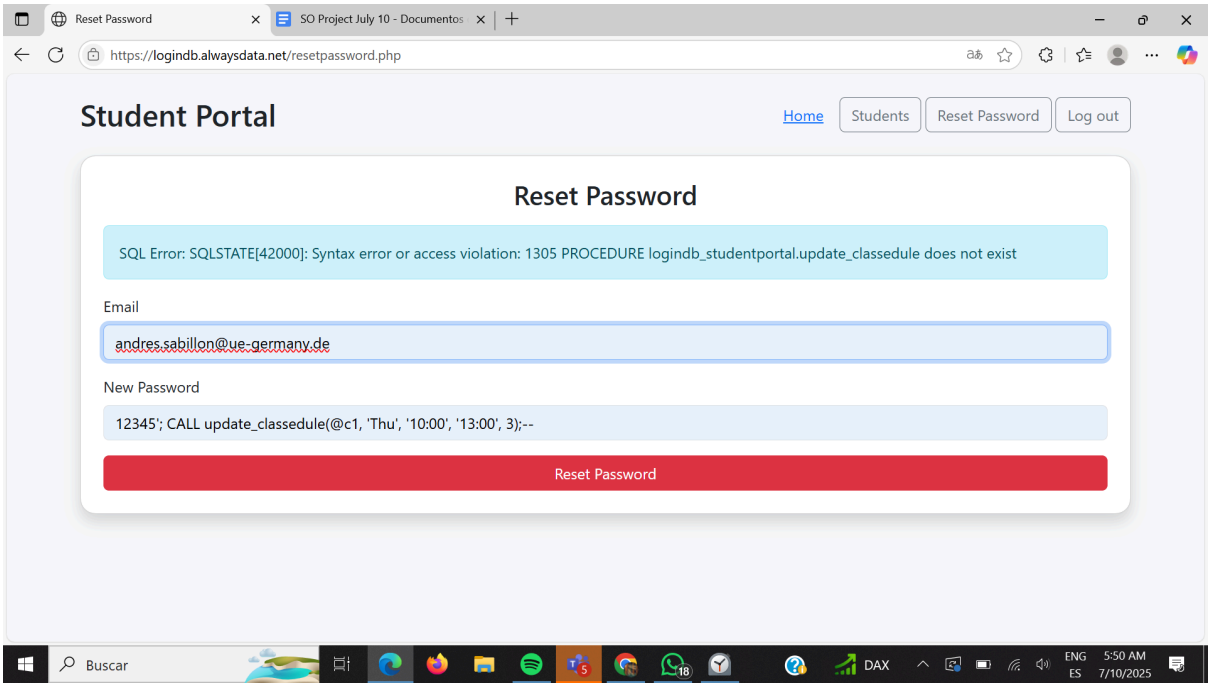


After the injection, we see the data of another user



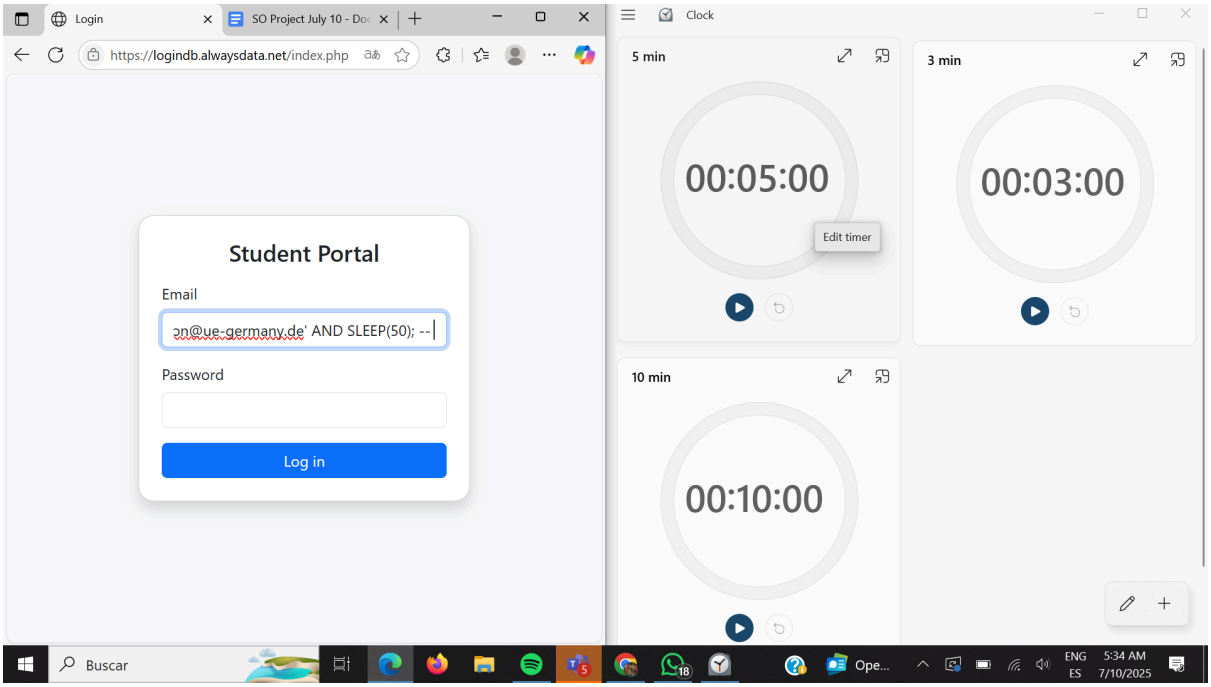
9 - TC_SQLI_09: SQLi via Stored Procedures

Input: 12345'; CALL update_classedule(@c1, 'Thu', '10:00', '13:00', 3);--

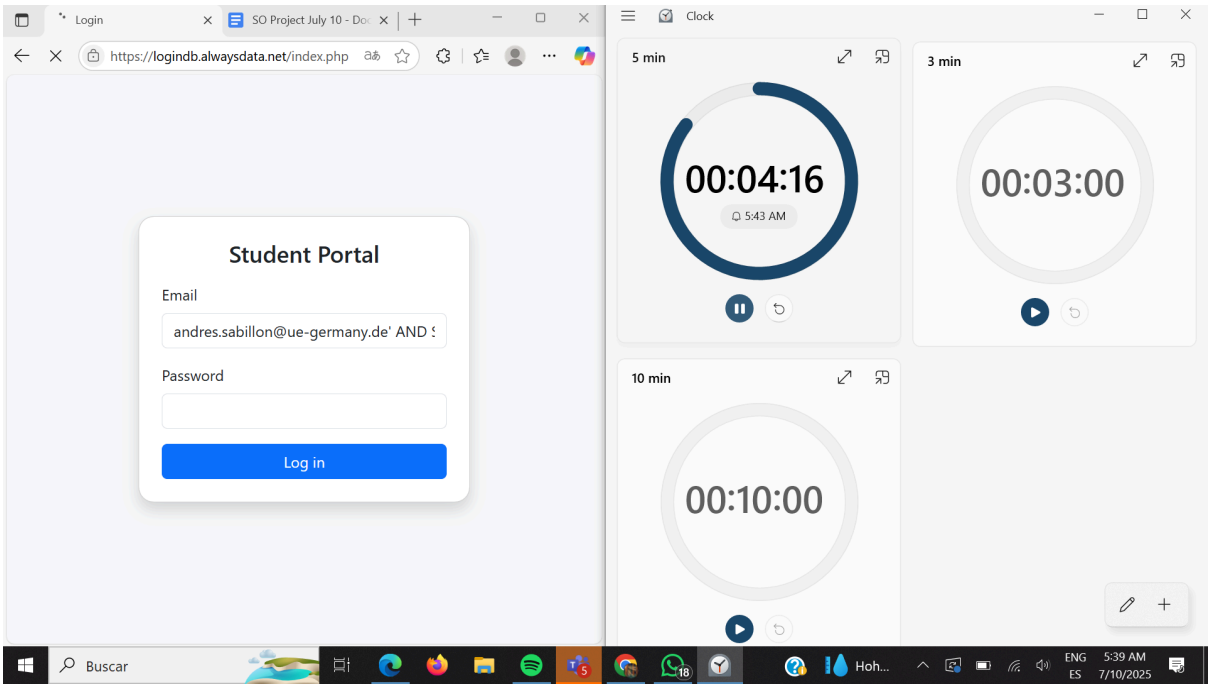


10 - TC_SQLI_10: Time-Based Blind SQL Injection

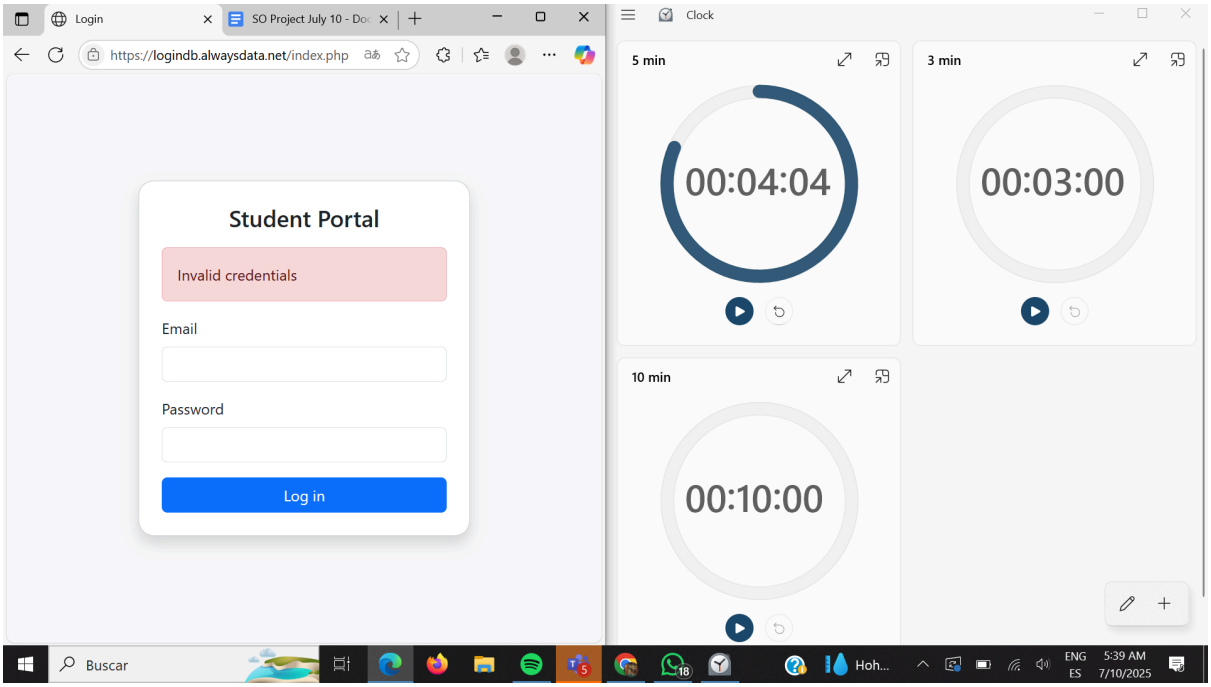
Username: andres.sabillon@ue-germany.de' AND SLEEP(50); --
Password: irrelevant
Before executing



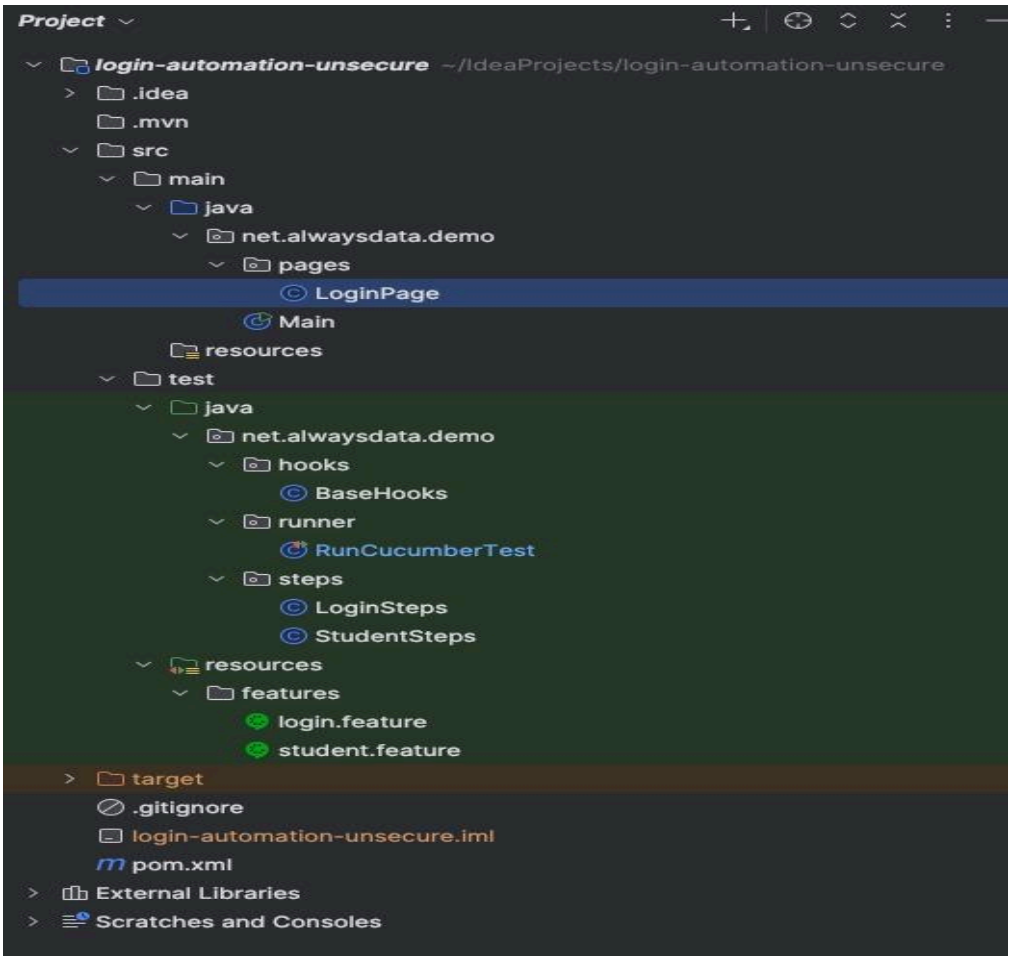
During the execution



After execution we got system response after 50 seconds


Login
SO Project July 10 - Doc
https://logindb.alwaysdata.net/index.php
Student Portal
Invalid credentials
Email
Password
Log in
Clock
5 min
00:04:04
3 min
00:03:00
10 min
00:10:00
Buscar
ENG
ES
5:39 AM
7/10/2025

Automatic


Project
login-automation-unsecure ~/IdeaProjects/login-automation-unsecure
.idea
.mvn
src
main
java
net.alwaysdata.demo
pages
LoginPage
Main
resources
test
java
net.alwaysdata.demo
hooks
BaseHooks
runner
RunCucumberTest
steps
LoginSteps
StudentSteps
resources
features
login.feature
student.feature
target
.gitignore
login-automation-unsecure.iml
pom.xml
External Libraries
Scratches and Consoles

```java
package net.alwaysdata.demo.hooks;

import ...

public class BaseHooks {

    private static WebDriver driver;  6 usages

    @Before
    public void setUp() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
            driver = null;
        }
    }

    public static WebDriver getDriver() { return driver; }
}
```

```java
package net.alwaysdata.demo.runner;

import org.junit.platform.suite.api.ConfigurationParameter;
import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;

import static io.cucumber.junit.platform.engine.Constants.GLUE_PROPERTY_NAME;
import static io.cucumber.junit.platform.engine.Constants.PLUGIN_PROPERTY_NAME;


@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(
        key   = GLUE_PROPERTY_NAME,
        value = "net.alwaysdata.demo"
)
@ConfigurationParameter(
        key   = PLUGIN_PROPERTY_NAME,
        value = "pretty, html:target/cucumber-report.html"
)
public class RunCucumberTest {

}
```

```java
package net.alwaysdata.demo.steps;

import ...

public class LoginSteps {  SenemTElmali

    private WebDriver driver;   // Başta null  6 usages
    private LoginPage login;  3 usages

    @Given("the user is on the login page")  SenemTElmali
    public void theUserIsOnLoginPage() {

        driver = BaseHooks.getDriver();
        login  = new LoginPage(driver);
        login.open();
    }

    @When("the user logs in with {string} and {string}")  SenemTElmali
    public void theUserLogsIn(String email, String pass) { login.loginAs(email, pass); }

    @Then("the login should be {string}")  SenemTElmali
    public void loginShouldBe(String result) {
        String expectedUrl = "";
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(3));

        if ("success".equalsIgnoreCase(result)) {
            expectedUrl = "home.php";
            wait.until(ExpectedConditions.urlContains(expectedUrl));
            assertTrue(driver.getCurrentUrl().endsWith(expectedUrl));
        } else if ("fail".equalsIgnoreCase(result)) {
            expectedUrl = "index.php";
            wait.until(ExpectedConditions.urlContains(expectedUrl));
            assertTrue(driver.getCurrentUrl().endsWith(expectedUrl));
        } else if ("maria-db-redirect".equalsIgnoreCase(result)) {
            expectedUrl = "index.php";
            wait.until(ExpectedConditions.urlContains(expectedUrl));
            String src = driver.getPageSource().toLowerCase();
            assertTrue( condition: src.contains("mariadb") || src.contains("sql syntax") || src.contains("warning"));
        } else {
            fail("Unknown result type: " + result);
        }
```

```java
package net.alwaysdata.demo.steps;

import ...

public class StudentSteps {  SenemTElmali

    @When("the user navigates to student view page with ID {int}")  SenemTElmali
    public void theUserNavigatesToStudentViewPage(int id) {
        WebDriver driver = BaseHooks.getDriver();
        new WebDriverWait(driver, Duration.ofSeconds(10))
                .until(ExpectedConditions.urlContains( fraction: "home.php"));

        driver.get("https://logindb.alwaysdata.net/view_student.php?id=" + id);
    }

    @Then("the first list item should display ID {string}")  SenemTElmali
    public void theFirstListItemShouldDisplayId(String expectedText) {
        WebDriver driver = BaseHooks.getDriver();
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        // <li> öğesini bul ve metnini al
        String actual = wait.until(ExpectedConditions
                        .presenceOfElementLocated(By.xpath( xpathExpression: "/html/body/div[1]/div/ul/li[1]")))  WebElement
                .getText()  String
                .trim();

        assertEquals(String.valueOf(expectedText), actual,
                message: "Student ID in first <li> is incorrect");
    }
}
```

```gherkin
Feature: User authentication

  # All rows share the same steps; only <result> changes
  Scenario Outline: Login with different credential sets
    Given the user is on the login page
    When the user logs in with "<email>" and "<password>"
    Then the login should be "<result>"

    Examples: Login test matrix
      | case                    | email                     | password    | result  |
      | valid credentials       | Kimia.karbasi@ue-germany.de | 1234        | success |
      | wrong password          | Kimia.karbasi@ue-germany.de | wrongpass   | fail    |
      | wrong email             | wrong@gmail.com           | 1234        | fail    |
      | empty password          | Kimia.karbasi@ue-germany.de |             | fail    |
      | empty email             |                           | 1234        | fail    |
      | sql inj 1 (always true) | ' OR 1=1 --               | ' OR 1=1 --    | success |
      | sql inj 2 (always true) | ' or ''='                 | ' or ''='    | success |
      | sql inj 3               | admin@domain.com' --        | anything    | maria-db-redirect  |
      | sql inj 4               | UNION SELECT creditCardNumber,1,'admin' FROM CreditCardTable -- | irrelevant    | maria-db-redirect |
      | sql inj 6               | ' OR 1=1 --           | anything    | maria-db-redirect |
      | sql inj 7               | ' OR 1=1 --           | ' OR 1=0 --    | success |
```

```
Feature: Student View Page

  Scenario Outline: Login and view student
    Given the user is on the login page
    When  the user logs in with "<email>" and "<password>"
    And   the user navigates to student view page with ID <id>
    Then  the first list item should display ID "<Userid>"

    Examples:
      | email                      | password       | id | Userid        |
      | Kimia.karbasi@ue-germany.de | 1234           | 3  | User ID: 3    |
      | Kimia.karbasi@ue-germany.de | 1234           | 4  | User ID: 4    |
      | ' OR 1=1 --                | ' OR 1=1 --    | 4  | User ID: 4    |
      | ' OR 1=1 --                | ' OR 1=1 --    | 3  | User ID: 3    |
```

classpath:features/login.feature

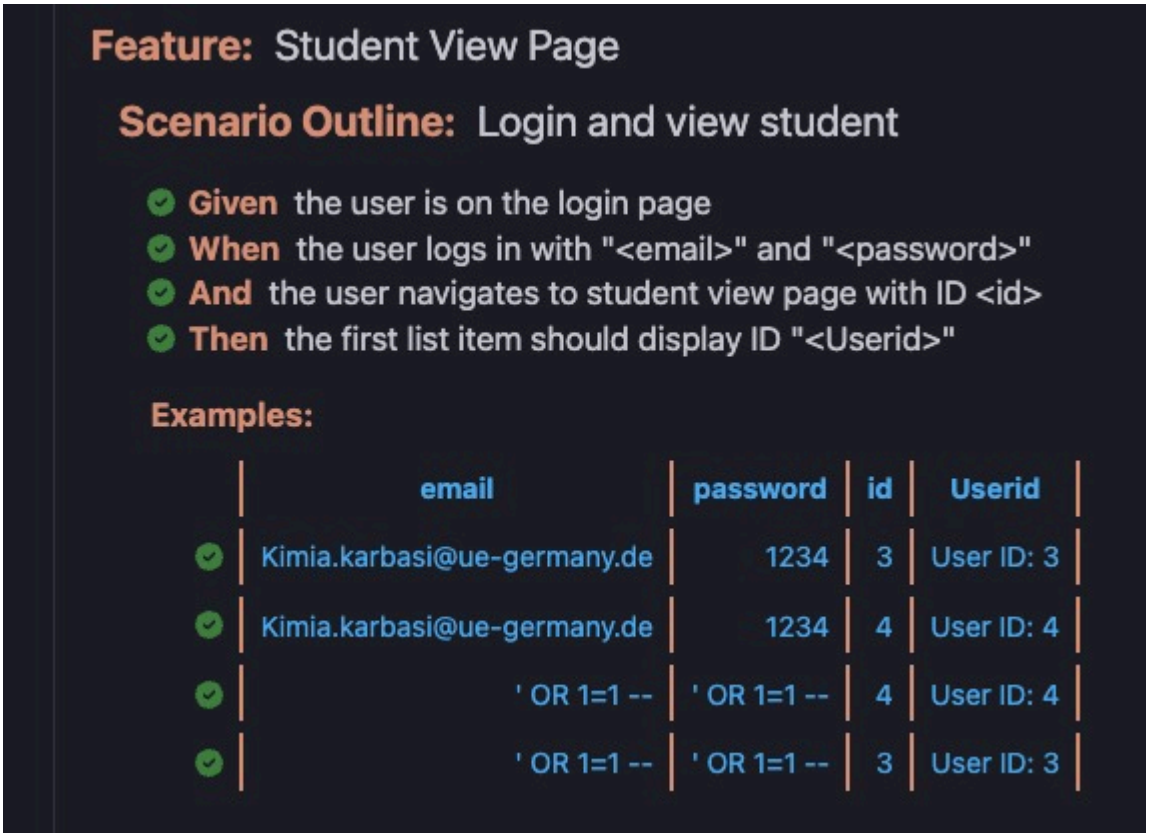
Feature: User authentication

Scenario Outline: Login with different credential sets

- **Given** the user is on the login page
- **When** the user logs in with "<email>" and "<password>"
- **Then** the login should be "<result>"

Examples: Login test matrix

case	email	password	result
valid credentials	Kimia.karbasi@ue-germany.de	1234	success
wrong password	Kimia.karbasi@ue-germany.de	wrongpass	fail
wrong email	wrong@gmail.com	1234	fail
empty password	Kimia.karbasi@ue-germany.de		fail
empty email		1234	fail
sql inj 1 (always true)	' OR 1=1 --	' OR 1=1 --	success
sql inj 2 (always true)	' or ''='	' or ''='	success
sql inj 3	admin@domain.com' --	anything	maria-db-redirect
sql inj 4	UNION SELECT creditCardNumber,1,'admin' FROM CreditCardTable --	irrelevant	maria-db-redirect
sql inj 6	' OR 1=1 --	anything	maria-db-redirect
sql inj 7	' OR 1=1 --	' OR 1=0 --	success



Contribution Table:

Student Name	Matriculation No.	Contribution
Kimia Sadat Karbasi	60393958	Create DB, Create Unsecure Site and Secure Site in web hosting and upload in FTP servers, Secure-WebPage document
Civan Arda Özel	45824733	General documentation, Creating login and password part, support in designing test cases,
Arsilda Qato	20540071	SQL Injection in non-secure WEB Pages
Andres Sabillon	79887864	Design Test Cases, Manual testing Secure Site, Coding Unsecure Website with

		Kimia.
Senem Türkaydın Elmalı	79873076	Automated Testing in Secure and Unsecure sites, support in designing test scenarios.
Sorasith Chormalee	53500842	Make unsecure website + update sample info and search for hosting that used for our project and define Major Non-Secure Issue

References

source codes:
https://github.com/cardaozel/Student_Portal
non-secure:
http://logindb.alwaysdata.net/index.php
secure:
https://studentportal.alwaysdata.net/ajaska